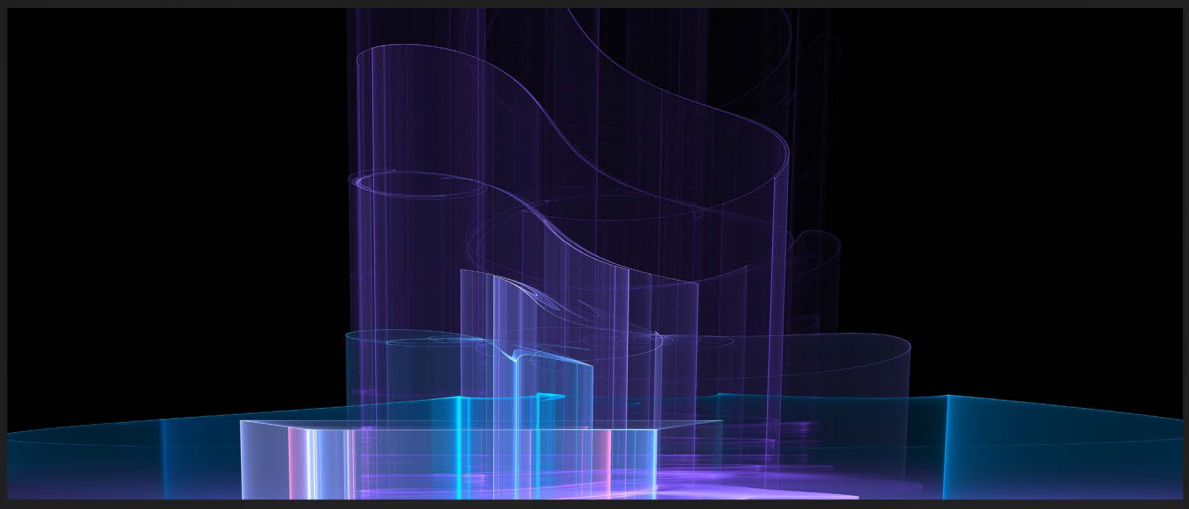
packt



1ST EDITION

GUI Programming with C#

Learn GUI development by building beginner-friendly apps with Blazor, MAUI, and WinUI 3

MARCELO GUERRA HAHN

GUI Programming with C#

Learn GUI development by building beginner-friendly apps with Blazor, MAUI, and WinUI 3

Marcelo Guerra Hahn

‹packt›

GUI Programming with C#

Portfolio Director: Kunal Chaudhari
Relationship Lead: Samriddhi Murarka
Project Manager: K. Loganathan
Content Engineer: Gowri Rekha
Technical Editor: Aditya Bharadwaj
Copy Editor: Safis Editing
Indexer: Tejal Soni
Proofreader: Gowri Rekha
Production Designer: Shankar Kalbhor
Growth Lead: Vinishka Kalra

First published: February 2026

Production reference: 2230226

Published by Packt Publishing Ltd.
Grosvenor House
11 St Paul's Square
Birmingham
B3 1RB, UK.

ISBN 978-1-83588-254-2

www.packtpub.com

Contributors

About the author

Marcelo Guerra Hahn is an educator, technologist, and software developer with experience in computer science and modern application development. He has worked extensively with C# and .NET technologies, focusing on building practical, industry-aligned learning experiences.

Marcelo combines academic rigor with real-world application, emphasizing clean architecture, performance optimization, and maintainable software design. His work bridges the gap between foundational computer science principles and modern development practices.

About the reviewers

Ricardo Peres is a Portuguese developer from Coimbra, Portugal, where he lives and works. He graduated in informatics engineering from the University of Coimbra and works as a tech lead at Evolve Software. Ricardo is a blogger, author, and contributor to open source projects, a book author and reviewer, and is currently doing a PhD in AI and microservices architectures. His interests include distributed services, architectures, enterprise application integration, design patterns, and LLMs.

Naga Santhosh Reddy Vootukuri is a principal software engineering manager at Microsoft, specializing in cloud computing and artificial intelligence within the C+AI division. Naga has authored and published numerous research articles in peer-reviewed and indexed journals. He has reviewed 10+ technical books on cloud computing and AI. He is a senior member of IEEE and teaches workshops on AI. He writes technical articles as a Core MVB member at DZone, engaging with millions of active readers. He has presented at 15+ WW conferences about the cloud, AI, and distributed systems. He mentors juniors on ADPList, helping with their career changes.

Michael Toner is an Azure-certified developer based in Belfast, Northern Ireland. With over 25 years of commercial programming experience, primarily in .NET and C#, he has been providing consultancy services to companies across various sectors for nearly 20 years.

Romain Ottonelli Dabadie, is a seasoned technical expert in .NET, shaping the IT landscape in both France and Canada. With over a decade of experience, he is a trusted professional known for crafting robust solutions using the .NET ecosystem. Throughout his career, Romain has taken on diverse roles, from developer to team leader. Beyond his technical skills, Romain is actively engaged on social media, sharing valuable insights with the tech community. He keeps up with the latest developments from Microsoft, showing a strong interest in their news and advancements. Furthermore, he is the author of the last edition of the book Mastering Visual Studio.

Table of Contents

Preface xiii

Free benefits with your book xviii

Chapter 1: Introduction to GUI Development in C# 1

Technical requirements 2

Developing GUIs in C# 2

Installing Visual Studio • 3

Using Visual Studio to create a GUI in C# • 6

Creating GUIs for different platforms in C# 10

Blazor • 10

.NET MAUI • 11

WinUI 3 • 11

Creating Blazor, MAUI, and WinUI 3 GUIs 12

Blazor • 12

MAUI • 14

WinUI 3 • 17

Summary 18

Chapter 2: Introduction to Blazor and WebAssembly 19

Technical requirements 20

Understanding Blazor 20

.NET runtime is loaded • 21

Blazor bootstraps the app • 22

Blazor's Management of Interactive UI and User Interactions • 22

Why use Blazor? • 23

Exploring WebAssembly **24**

What is WebAssembly? • 24

How Does WebAssembly Work? • 25

Why use WebAssembly? • 26

Blazor and WebAssembly Together **26**

Improving your Blazor Application **28**

Debugging and troubleshooting **32**

Displaying information for debugging • 32

Using diagnostic tools • 33

Issues with Running Pages • 34

Using debugging tools in Visual Studio • 35

Handling errors **37**

Best practices and tips **38**

Summary **39**

Chapter 3: Building Your First Blazor Web Application 41

Technical requirements **41**

Understanding the Program.cs file **42**

Understanding the other files in the solution **44**

Creating the application **47**

Component life cycle methods **52**

Using the component **52**

Routing in Blazor • 53

Setting up a sample API **54**

Summary **55**

Chapter 4: Using Razor Components and Data Binding 57

Technical requirements **58**

Understanding Razor components **58**

Benefits of using Razor components • 58

Understanding the development workflow • 59

Creating Razor components **59**

The basic structure of a Razor component • 59

Defining component parameters • 60

Reusing components across pages and applications • 61

Data binding in Razor components **61**

One-way data binding • 61

Two-way data binding • 62

Event binding • 63

Managing dynamic data **63**

Advanced component techniques **65**

Component life cycle methods • 65

Passing data between components • 66

Using templates with Razor components • 67

State management in Razor components **68**

Handling state in single components • 68

Sharing state across multiple components • 69

Using state management libraries • 70

Working with forms and validation **71**

Displaying validation messages • 71

Styling Razor components **72**

Applying CSS to components • 72

Scoped CSS for components • 72

Summary **73**

Chapter 5: Introduction to .NET MAUI 75

Technical requirements **76**

.NET MAUI **76**

Evolution from Xamarin.Forms • 76

Core components – .NET, C#, and XAML • 77

Benefits of using .NET MAUI • 77

Use cases and applications • 78

Creating a cross-platform application with .NET MAUI • 78

Creating a new project • 79

Project structure and files • 79

Building the UI with XAML 80

Layouts • 80

Controls • 81

Adding interactivity with C# 83

Building a cross-platform application with .NET MAUI 84

Summary 89

Chapter 6: Creating Native, Cross-Platform Apps with .NET MAUI 91

Technical requirements 92

Overview of the app and .NET MAUI 92

Benefits of using .NET MAUI for cross-platform development • 92

Setting up the development environment 93

Understanding the basics of .NET MAUI • 94

Key components and namespaces • 94

Developing the app – creating the project, UI, logic, and events 95

Designing the UI with XAML • 95

Using the Grid layout for responsive design • 95

Adding controls • 96

Styling and theming the app • 97

Implementing app logic and events 98

Handling user inputs and events • 99

Data binding and the MVVM pattern • 99

Advanced UI design techniques 101

Animations and transitions • 101

Accessibility features • 102

Testing and deploying the app 104

Configuring platform-specific settings • 104

Running the app on Windows, macOS, iOS, and Android • 105

Deploying the app to app stores • 106

Creating app packages • 106

Submitting to app stores • 107

Summary 108

Chapter 7: Implementing MVVM and Data Binding in .NET MAUI 109

Technical requirements 110

Introduction to MVVM and data binding in .NET MAUI 110

What are MVVM and data binding? • 111

Comparison with other design patterns (MVC and MVP) • 112

Benefits of using MVVM in app development • 113

Data binding concepts 114

Implementation in .NET MAUI • 115

View model (UserViewModel.cs) • 115

View (MainPage.xaml) • 116

Creating a view model and defining bindings 117

Creating a simple .NET MAUI app • 118

Creating the view model • 119

Define a ViewModel class • 119

Implement properties • 120

Initialize data • 120

Defining the view • 121

Binding to the view model • 121

Basic data binding 122

When and how to use each mode • 123

Converters • 123

Commands • 125

Collections • 126

Enhancing user experience 128

Summary 129

Chapter 8: Getting Started with WinUI 3 and Visual Studio 131

Technical requirements **132**

Understanding WinUI 3 and its benefits **132**

WinUI 3 versus WPF – key differences and advantages • 134

WinUI 3 versus UWP • 135

WinUI 3 versus WinForms • 135

Real-world applications and use cases **135**

Case study 1 – healthcare management system • 136

Case study 2 – financial trading platform • 136

Case study 3 – educational e-learning application • 136

Setting up the environment **136**

Compatible operating systems and versions • 137

Installing and configuring WinUI 3 ..... **138**

Creating a new WinUI 3 project **139**

Understanding project templates and initial settings • 140

Understanding the WinUI 3 project structure • 141

Choosing between packaged and unpackaged app types • 141

Building a WinUI 3 app **142**

Running and debugging a WinUI 3 app • 144

Data binding and the MVVM pattern in WinUI 3 ..... **144**

Summary **148**

Chapter 9: Building a Modern Desktop App with WinUI 3 149

Technical requirements **150**

Advanced WinUI 3 controls **150**

Learning management system example • 150

Step 0 – create a new WinUI project • 151

Step 1 – define the NavigationView control • 151

Step 2 – add navigation items (menu) • 152

Step 3 – display the main content • 152

Step 4 – add a border with welcome text • 153

ListView **154**

Adding the ListView control for Favorite Courses • 156

Displaying Favorite Courses dynamically in the code behind • 157

GridView **159**

Summary **162**

Chapter 10: Advanced WinUI 163

Technical requirements **164**

TreeViews **164**

Customizing controls and building complex interfaces **167**

The Background property • 167

The Padding property • 167

The Margin property • 168

The FontSize property • 169

The FontWeight property • 169

The Foreground property • 170

Data binding and MVVM – deep Dive into the MVVM pattern to decouple UI and business logic **171**

Understanding MVVM layers • 171

MVVM example – course details app • 171

Optimizing app performance **175**

Load time reduction • 176

Memory optimization • 176

Reducing UI latency • 176

Profiling and debugging techniques for ensuring smooth user experiences • 177

Debugging techniques • 177

Summary **178**

Why subscribe? **179**

Other Books You May Enjoy 181

Index 185

Preface

Graphical User Interfaces (GUIs) have fundamentally transformed the way humans interact with technology. From desktop applications to web platforms and cross-platform mobile solutions, modern software depends on intuitive, responsive, and visually engaging interfaces. This book is designed to guide you through the evolving landscape of GUI development using C#, equipping you with the tools and architectural understanding necessary to build contemporary applications across multiple platforms.

C# and the .NET ecosystem provide a powerful, unified foundation for building UIs. Whether targeting Windows desktop applications with WinUI 3, developing interactive web experiences with Blazor and Razor components, or creating native cross-platform applications using .NET MAUI, developers can leverage a consistent language, shared libraries, and a cohesive development model.

This book explores GUI development across three primary pillars:

Desktop development with WinUI 3

Web development with Blazor and Razor components

Cross-platform development with .NET MAUI

Rather than focusing on a single framework, this book emphasizes architectural principles, UI composition, data binding, event-driven programming, and performance optimization. By mastering these foundations, you will be prepared to build scalable, maintainable, and high-performance applications across different platforms.

Throughout the chapters, we move from foundational principles to applied development:

- We begin with the fundamentals of GUI development in C# and modern development workflows using Visual Studio
- We explore Blazor and WebAssembly to understand client-side and server-side web UI development

- We build practical web applications using Razor components and structured data binding techniques
- We learn how to create cross-platform native applications using .NET MAUI, leveraging a unified project structure and shared code base
- We develop modern Windows desktop applications using WinUI 3, incorporating Fluent design principles, MVVM patterns, and advanced UI controls

Each chapter balances conceptual understanding with hands-on examples. You will learn not only how to build UIs but also why certain architectural decisions matter. Topics such as data binding, MVVM, state management, responsive layouts, platform integration, and performance optimization are treated as essential skills for modern developers.

In writing this book, I wanted to combine academic clarity with real-world practicality. The examples are structured to reflect patterns used in production environments while remaining accessible to learners building foundational expertise.

Rather than focusing on a single framework, this book emphasizes understanding architectural principles, UI composition, data binding, event-driven programming, and platform integration. By mastering these foundations, you will be prepared to build scalable, maintainable, and high-performance applications in diverse environments.

Who this book is for

This book is for developers, students, and software engineers who want to build modern graphical applications using C# and the .NET ecosystem.

It is particularly suited for the following people:

- Students learning modern application development
- Developers transitioning into Blazor, .NET MAUI, or WinUI 3
- Engineers seeking to strengthen their understanding of MVVM and UI architecture
- Educators designing coursework around contemporary GUI frameworks

A foundational understanding of C#, object-oriented programming, and basic Visual Studio usage is recommended to get the most out of this book.

What this book covers

Chapter 1, *Introduction to GUI Development in C#*, introduces the fundamentals of GUIs and the role of C# in modern UI frameworks.

Chapter 2, *Introduction to Blazor and WebAssembly*, explains Blazor architecture and how C# can power interactive web applications.

Chapter 3, *Building Your First Blazor Web Application*, walks through the creation of a structured web application using components and layouts.

Chapter 4, *Using Razor Components and Data Binding*, explores component-based design and dynamic UI updates through data binding.

Chapter 5, *Introduction to .NET MAUI*, introduces cross-platform application architecture and development workflows.

Chapter 6, *Creating Native, Cross-Platform Apps with .NET MAUI*, demonstrates how to build and structure practical cross-device applications.

Chapter 7, *Implementing MVVM and Data Binding in .NET MAUI*, focuses on architectural best practices and separation of concerns.

Chapter 8, *Getting Started with WinUI 3 and Visual Studio*, introduces modern Windows desktop application development.

Chapter 9, *Building a Modern Desktop App with WinUI 3*, covers UI composition, layouts, controls, and application structure.

Chapter 10, *Advanced WinUI*, explores control customization, MVVM implementation, performance optimization, profiling, and debugging techniques.

To get the most out of this book

You should have the following:

- A basic understanding of C# programming
- Familiarity with object-oriented programming concepts
- Visual Studio installed on your machine
- The relevant .NET workloads for web, desktop, and cross-platform development

Download the example code files

The code bundle for the book is hosted on GitHub at `https://github.com/PacktPublishing/GUI-Programming-with-C-Sharp`. We also have other code bundles from our rich catalog of books and videos available at `https://github.com/PacktPublishing`. Check them out!

Conventions used

Several text conventions are used throughout this book.

`CodeInText`: Indicates code words in text, class names, filenames, namespaces, methods, or user input. For example: `INotifyPropertyChanged`.

A block of code is set as follows:

```
public class CourseViewModel : INotifyPropertyChanged
{
    public string CourseName { get; set; }
}
```

When we want to draw attention to a specific part of a code block, the relevant lines are bolded.

Any command-line input or output is written as follows:

```
dotnet build
```

Bold text indicates a new term, an important word, or words that appear onscreen.

Warnings or important notes appear in highlighted format.

Tips and best practices are presented separately where appropriate.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book or general feedback, please email us at `customercare@packt.com` and mention the book's title in the subject line of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you could report it to us. Please visit `http://www.packt.com/submit-errata`, click **Submit Errata**, and fill in the form.

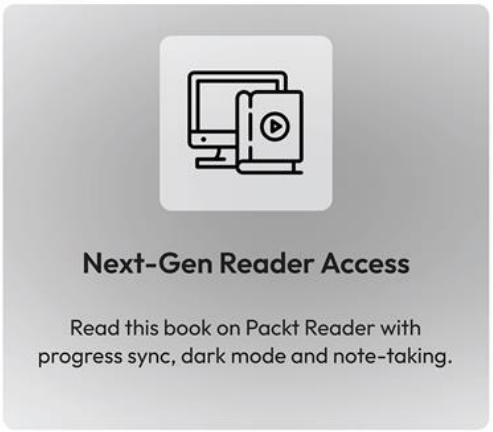
Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you could provide the location address or website name. Please get in touch with us at `copyright@packt.com` with a link to the material.

If you are interested in becoming an author, and you have expertise in a topic and would like to write or contribute to a book, please visit `http://authors.packt.com/`.

Free benefits with your book

This book comes with free benefits to support your learning. Activate them now for instant access (see the "*How to Unlock*" section for instructions).

Here's a quick overview of what you can instantly unlock with your purchase:



How to Unlock

Scan the QR code (or go to `packtpub.com/unlock`). Search for this book by name, confirm the edition, and then follow the steps on the page.

UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require one

Share your thoughts

Once you've read *GUI Programming with C#*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback.

`https://packt.link/r/1835882552`

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

1

Introduction to GUI Development in C#

Graphical User Interfaces (GUIs) have changed how humans and computers interact. With GUIs, users do not need to learn complex commands to perform actions. Instead, they can carry them out with simple visual features such as icons, windows, and buttons. This has made it easier for users to understand and utilize software, significantly improving its wide availability and convenience. The primary distinction between conventional text-based UIs and GUIs is that GUIs allow users of different backgrounds to interact with various applications using symbols and images to carry out commands. This flexibility has transformed the computing world by enabling users with limited technical knowledge to carry out different tasks easily. GUIs are a crucial part of application development due to their ability to influence its various aspects, such as the following:

- **User experience**: The convenient interfaces supplied by GUIs boost the user's software programs quickly
- **Accessibility**: The visual features, such as icons, windows, and buttons provided by GUIs allow individuals with limited technical knowledge to perform tasks quickly, making them more accessible
- **Throughput**: Proficient GUIs allow users to perform complex tasks swiftly, streamlining the workflow and enhancing efficiency and productivity while developing software
- **Competitive advantage**: Well-designed and accessible GUIs can differentiate products from competitors, giving them a significant edge in the market and contributing to success

In this chapter, we will cover the following main topics:

- Developing GUIs in C#
- Creating GUIs for different platforms in C#
- Using Visual Studio to develop GUIs for different platforms in C#

Technical requirements

This chapter requires access to **Visual Studio Community** (`https://visualstudio.microsoft.com/downloads/`) with the following workloads installed:

- ASP.NET and Web Development
- .NET **Multi-platform App UI** (**MAUI**) Development
- .NET desktop development
- **Universal Windows Platform** (**UWP**) Development

The code shown in the examples can be found here: `https://github.com/PacktPublishing/GUI-Programming-with-C-Sharp/tree/main/Chapter%201`

Developing GUIs in C#

Modern software uses GUIs. GUIs allow users to perform their tasks effortlessly through user-friendly interactions. Microsoft developed the object-oriented programming language C# (pronounced C Sharp) as its preferred language for developing its products across various platforms.

The easy-to-use, contemporary, and versatile C# works in the context of the wide-ranging set of technologies offered by Microsoft's platform. This platform is designed to support, among other features, cross-platform development. The simplicity of the C# syntax allows developers to use it for different programming tasks. Developers used to C, C++, or Java can easily transition to C# due to all of them being C-style languages with similar syntax basics, such as the use of curly braces.

C# is an object-oriented language supporting event-driven programming and has a rich library support that makes it appropriate for UI development. Let's explore some of them:

- **Rich library support**: C# is part of the .NET platform. This platform contains an extensive range of libraries. These libraries include large quantities of pre-written code. This code is exposed in the form of classes and interfaces that can be accessed through pre-compiled public APIs. This pre-existing code allows developers to focus on developing their application's distinctive functionalities instead of rewriting already-available solutions.

- **Object-oriented programming**: As an object-oriented programming language, C# includes classes, inheritance, and polymorphism. In C#, GUIs are represented as objects that contain and are near other objects and, in turn, allow users to invoke methods. This sequence of events enables developers to write modular code to handle the UI interactions and their effects.
- **Event-driven programming**: Events represent user actions, sensor inputs, or commands from other programs. Events enable the fluid execution of a program through the use of event-driven programming. This programming style simplifies the development of GUI-driven functionality because user actions trigger events that lead to the execution of the corresponding code. To fully appreciate the benefits of event-driven programming, it is helpful to understand the concepts of tight and loose coupling. Event-driven programming supports loose coupling, where components interact through well-defined interfaces, enhancing modularity, flexibility, and maintainability.

To use these features to start creating GUIs, we will need a tool that can facilitate the process. The tool of choice for C# development is Visual Studio. In the next section, we will install Visual Studio and develop our first GUIs.

Installing Visual Studio

The first step to be able to create the application is to install Visual Studio. Different options for this are available, including **Visual Studio Code**, **Visual Studio Community**, **Visual Studio Professional**, and **Visual Studio Enterprise**. In this book, we will use Visual Studio Code and Community as they are free and include the features we will explore. To download Visual Studio, navigate to `https://visualstudio.microsoft.com/` and follow these steps:

1. Run the installer once the download is complete. The installation can be customized by picking specific workloads and components.
2. Select **C# Workload**. Be careful when choosing the C# development workload during the installation. Make sure that you select the following workloads:
    1. **.NET Multi-platform App UI development**
    2. **.NET desktop development**
3. Follow the prompts to complete the installation process. Once installed, click the **Launch** button to start.

Once in Visual Studio, we can explore its main components as they appear on the screen:

- **Solution Explorer**: Displays the structure of the solution and projects. Files, folders, and project dependencies can be navigated from here.

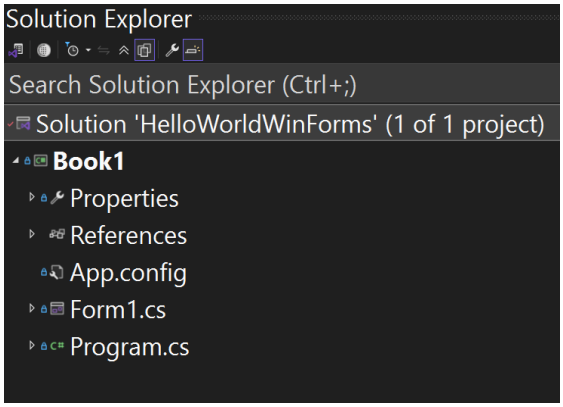


Figure 1.1 – Solution Explorer

- **Editor**: The main area where the actual coding is done. It provides code completion, syntax highlighting, and other helpful tools to boost productivity.

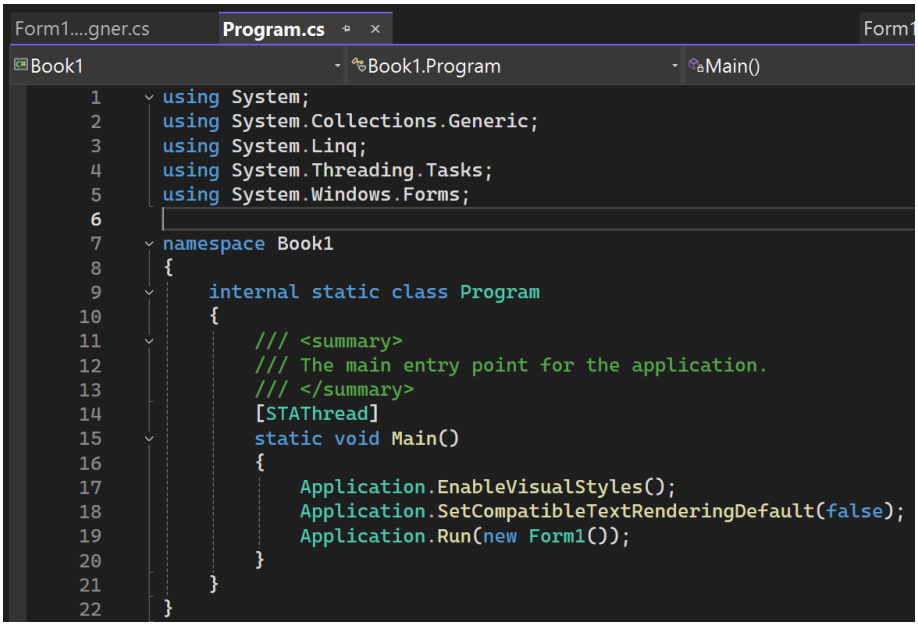


Figure 1.2 – Code editor

- **Toolbox**: Offers several elements, controls, and other features that may be dropped onto windows or forms.

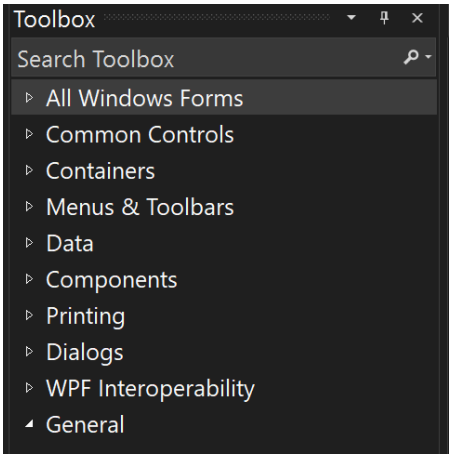


Figure 1.3 – Toolbox

- **Properties**: Displays the properties of the currently selected item in the designer or code editor. You can use it to modify various properties of controls and components.

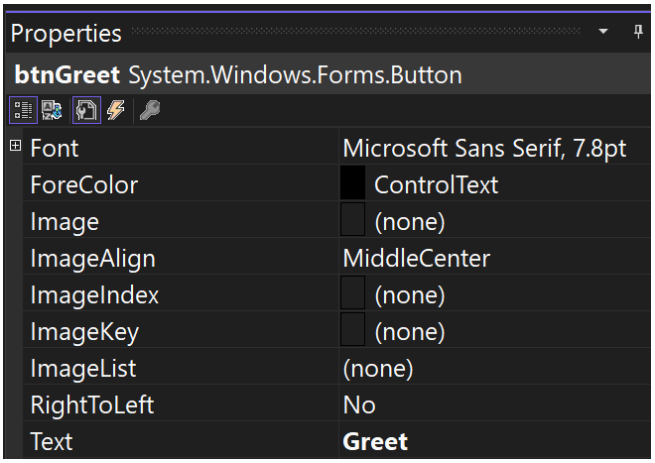


Figure 1.4 – Properties window

- **Designer**: Developers can drag and drop controls across a form or window to build the user experience graphically. Developers can flip through the code view and design view to see the code.

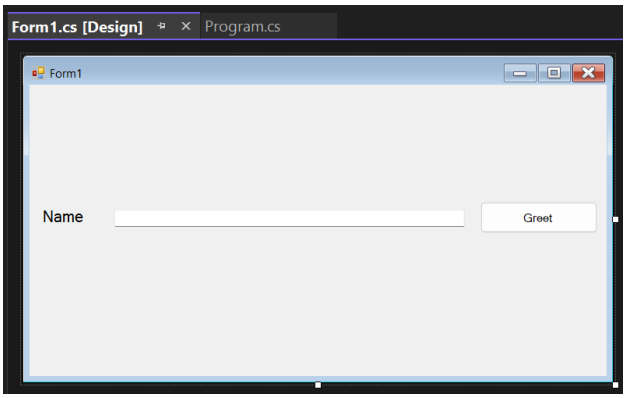


Figure 1.5 – Designer window

Now that Visual Studio is installed, we can create our first UI.

Using Visual Studio to create a GUI in C#

We can create the first project by following these instructions.

1. From the **Start** menu, select **Create New Project**.
2. From the **All languages** drop-down menu, select **C#**.
3. From **All Platforms**, select **Windows**.
4. From **All Project Types,** select **Desktop**.
5. From the list, choose **Windows Form App (.NET)**.

Note

The .NET framework is now outdated and is only included here to show the foundations for the other technologies.

6. Enter `Hello World` for **Project Name**.
7. Click the **Create** button.

Visual Studio offers a smooth programming experience, which makes it the best option for C# GUI development. A blank Windows form is shown in *Figure 1.6*.

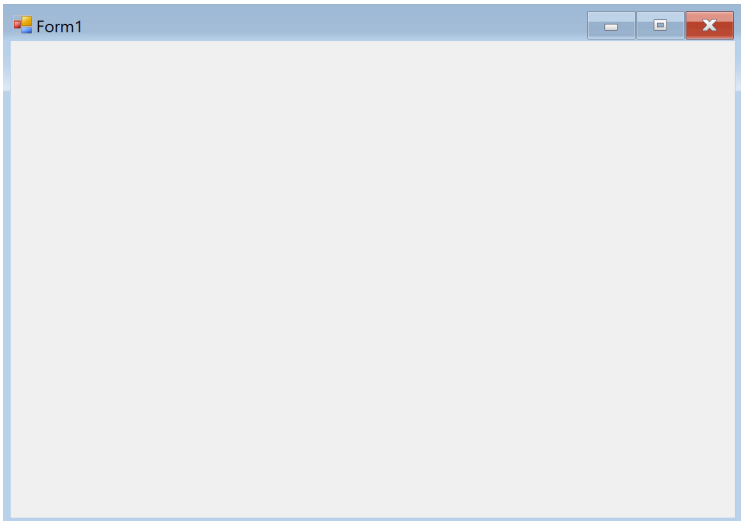


Figure 1.6 – Blank Windows form in Visual Studio

The next step is to add controls to the form. Those controls are added by dragging and dropping from **Toolbox**. Follow the steps below to complete the form.

1. Select **View** and then **Toolbox**. From here, start adding controls to the form. For this example, drag and drop a label, a text box, and a button so the form looks like *Figure 1.7*. These elements are under the **Common Controls** section of the **Toolbox** and can also be found using the search box in it.

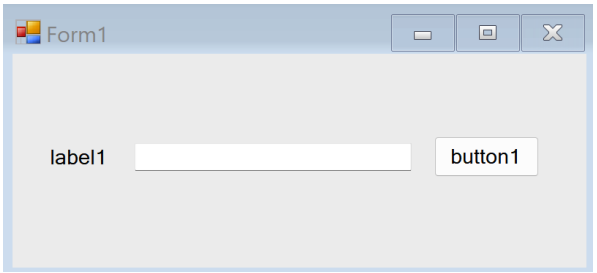


Figure 1.7 – Form with a label, text box, and button

2. Once the elements are in place, you can use the **Properties** window to change their properties. Make the following changes:

    1. Label:

        - Text: `Name`

    2. Textbox:

        - (Name): `txtName`

    3. Button

        - Text: `Greet`
        - (Name): `btnGreet`

    After making those changes, the form should look like this:

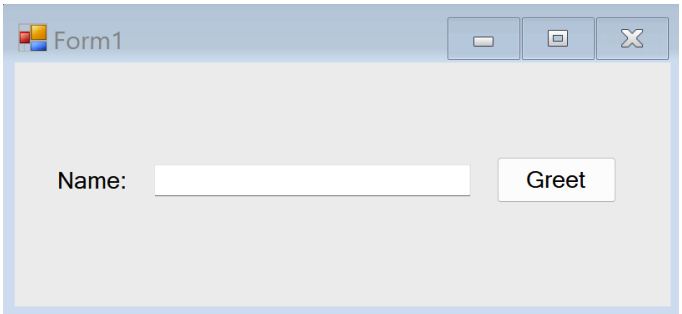


Figure 1.8 – Controls with updated properties

Note that each of the elements in the form, including the form, are objects in an object-oriented hierarchy where all of them inherit from a class named `Control`, and the label, textbox, and button are contained in the form.

As mentioned before, C# uses events to manage the interactions, so we need to access the code for the event that indicates that the button has been clicked on to add the code that produces the interaction. To do that, double-click on the **Greet** button. The code editor shown in *Figure 1.9* will be displayed.

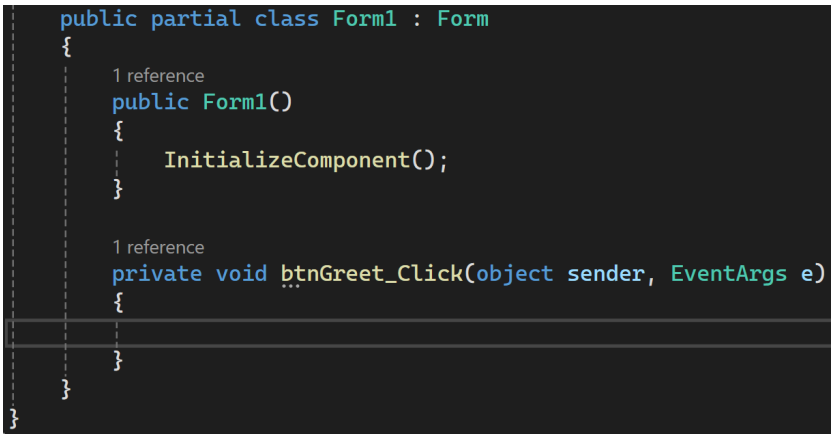


Figure 1.9 – Code editor window showing the button click method

Complete the code by showing a popup saying `'Hello'` followed by the name inserted in the text box. The code should look as follows:

```
        private void btnGreet_Click(object sender, EventArgs e)
        {
            MessageBox.Show($"Hello {txtName.Text}");
        }
```

This code works as follows:

- `private void btnGreet_Click(object sender, EventArgs e)`: This event handler method gets called when the `btnGreet` button is clicked. The `private` keyword means this method is only accessible within the same class. The `void` term means this method doesn't return any value. The parameter's `sender` object and `EventArgs e` represent the control that fired the event and the event data, respectively.
- `MessageBox.Show($"Hello {txtName.Text}");`: This line of code shows a message box with a greeting message. The `MessageBox.Show()` method displays a message box to the user. The message is `"Hello"` concatenated with the text from `txtName`. The `txtName.Text` part is the text property of a `TextBox` control named `txtName`, which contains the text entered by the user.

So, when the `btnGreet` button is clicked, a message box will appear saying **"Hello"** followed by whatever name the user has entered into the `txtName` text box. For example, if the user entered `John` into the `txtName` text box and then clicked the `btnGreet` button, a message box would appear saying **"Hello John"**.

This example, though simple, shows the main ideas behind designing a GUI in C#. Those steps include selecting the platform/technology to be used, creating the interactions by placing the UI elements in the desired locations, and programming the interactions by writing code to respond to user actions when interacting with the UI components.

Creating GUIs for different platforms in C#

As mentioned in the previous section, libraries play a key role in UI development. Many libraries have been used for this, and the libraries have evolved. Some older libraries include **Windows Forms** and **Windows Presentation Foundation (WPF)**. In this book, we will not focus on those. Instead, we will expand on modern libraries, including **Blazor**, **WebAssembly**, **.NET MAUI**, and **Windows UI Library 3 (WinUI 3)**.

Blazor

Microsoft offers a powerful frontend framework called Blazor. This framework can construct interactive client-side web GUIs. Blazor distinguishes itself from other frameworks because it allows developers to use C# and .NET to write the client-side part of the code without needing JavaScript. Developers familiar with C# and .NET can use their current skills and knowledge without the need to learn other GUI technologies. Blazor supports two hosting models: Blazor Server (SignalR-based) and Blazor WebAssembly (client-side in the browser). As a result, the learning period for developers new to web development is reduced, and the productivity of existing developers who are well versed in .NET and C# is increased.

Additionally, developers can make the most of the vast libraries offered by .NET, which boosts their productivity even more and increases the capabilities of their applications. Developers use the library SignalR, which allows them to streamline the process of adding real-time web functionality to their applications. Using this process, any changes happening on the server can be notified to the client using push notifications, which gives their applications the appearance of real-time responsiveness.

.NET MAUI

.NET MAUI allows developers to create mobile and desktop applications using C# and **Extensible Application Markup Language (XAML)**.

XAML

XAML is a declarative XML-based language used to define the UI in .NET applications, particularly those developed with WPF and UWP. It allows developers to design UI elements and their properties in a structured and readable format, which can then be seamlessly integrated with the application's backend logic written in languages such as C#.

.NET MAUI is an evolution of Xamarin.Forms and supports different operating systems, including Android, iOS, macOS, and Windows. The main advantage of using MAUI is that applications can be repackaged on any device, such as a smartphone, tablet, or desktop. As a result, developers no longer need to write platform-specific code for their applications, which helps them save precious time and resources. This also allows developers to concentrate all their effort on creating high-quality applications that could flawlessly work on all major platforms with the help of .NET MAUI.

WinUI 3

WinUI 3 builds modern native Windows applications. Although WinUI 3 is part of the Windows App SDK (formerly known as Project Reunion), it is shipped separately from the Windows operating system. As the next generation of the WinUI framework, WinUI 3 develops WinUI into a complete UX framework. Thus, WinUI is accessible as a UI layer for all desktop Windows apps. As it is separate from Windows SDKs, it can provide developers with a combined set of APIs and tools that they could utilize to develop production desktop apps. These apps were designed for Windows 10 and later versions and published in the Microsoft Store.

A vital feature of WinUI 3 is that developers can develop packaged and unpackaged C# and C++ WinUI 3 desktop apps with its help. The packaged applications include the package manifest and additional resources to let the developers turn the app into an MSIX package. Unpackaged applications, on the other hand, do not require a manifest or additional packaging, allowing developers to run and distribute their apps more flexibly without the constraints of the MSIX packaging process.

Creating Blazor, MAUI, and WinUI 3 GUIs

In this section, you'll create the same Hello World form you created using a Windows form in Blazor and MAUI. We'll implement it in WinUI 3 in a future chapter.

Blazor

To create the Hello World app in Blazor, create a new project in C# for Windows under the **Cloud** category and select **Blazor Web App** by following these steps:

1. Select **Create New Project** and mark the options shown in the following screenshot:

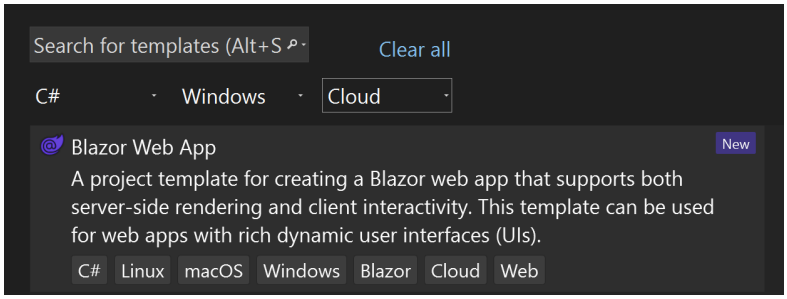


Figure 1.10 – New Blazor Web App project options

2. In the **Project Configuration** window, click **Next**.
3. Find the **Home.razor** page in **Solution Explorer**, as shown here:

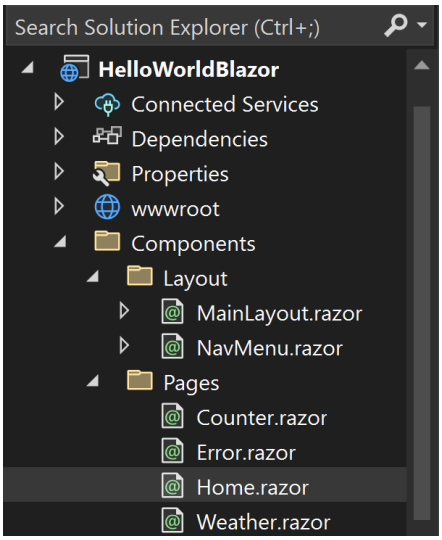


Figure 1.11 – Blazor solution

4. Modify the code within the page so that it looks like the following:

```
@page "/greet"

<h3>Greet</h3>

<input @bind="Name" placeholder="Enter your name" />

<button @onclick="ShowGreeting">Greet</button>

@if (!string.IsNullOrEmpty(Greeting))
{
    <p>@Greeting</p>
}

@code {
    string Name { get; set; }
    string Greeting { get; set; }

    void ShowGreeting()
    {
        Greeting = $"Hello {Name}";
    }
}
```

5. Run the project. The screen in *Figure 1.12* should display. At this point, you may receive one or more popups related to SSL certificates. If this happens, accept the requests. The component can be accessed through the `/greet` URL.

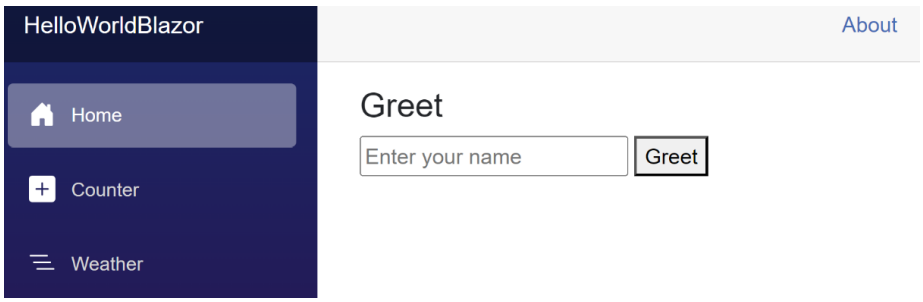


Figure 1.12 – Final Blazor GUI

We will dedicate multiple chapters to Blazor. However, here is a short overview of the code:

- `@page "/greet"` sets the route for this component. In this context, a route defines the URL path (`"/"greet"`) that maps to and displays the specified component when accessed by the user.
- The input element is data-bound to the `Name` property, updating the name whenever the user enters the field.
- The button element has an `@onclick` directive that calls the `ShowGreeting` method when the button is clicked.
- The `ShowGreeting` method sets the `Greeting` property to a string that includes the name entered by the user. This will in turn cause the string being displayed to update. This line will become important when debugging the code as it is the spot where the breakpoint can be set.
- The `@if` block renders the greeting message only if it's not null or empty.

When running the application, you can navigate to this component by entering `/greet` in your browser's address bar. When you enter a name and click the **Greet** button, you should see the greeting message appear below the button. This is similar to the functionality of the Windows Forms code, but it uses Blazor and WebAssembly.

MAUI

To create the same form in .NET MAUI, follow these steps:

1. Select **Create New Project** and select the following options:

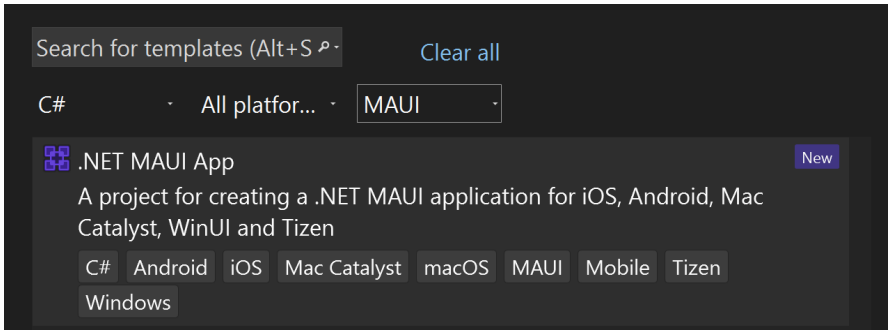


Figure 1.13 – New MAUI project options

2. Find the `MainPage.xaml` and `MainPage.xaml.cs` files in **Solution Explorer**, as seen in *Figure 1.14*.

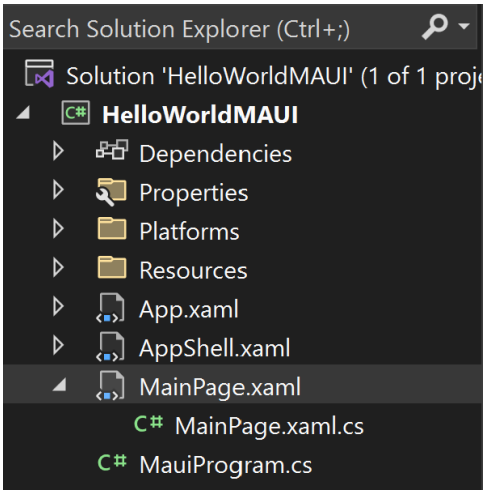


Figure 1.14 – MAUI solution

3. Modify the code in `MainPage.xaml` to look like the following code.

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class= "HelloWorldMAUI.MainPage">

    <StackLayout Margin="20">
        <Entry x:Name="NameEntry" Placeholder="Enter your name" />
        <Button Text="Greet" Clicked="OnGreetButtonClicked" />
        <Label x:Name="GreetingLabel" />
    </StackLayout>

</ContentPage>
```

In this code, note the following:

- The `Entry` element is where the user can enter their name
- The `Button` element has a `Clicked` event that calls the `OnGreetButtonClicked` method when the button is clicked
- The `Label` element is where the greeting message will be displayed

4. Modify the `MainPage.xaml.cs` code to look like the following one:

```
namespace HelloWorldMAUI
{
    public partial class MainPage : ContentPage
    {
        public MainPage()
        {
            InitializeComponent();
        }

        void OnGreetButtonClicked(object sender, EventArgs e)
        {
            GreetingLabel.Text = $"Hello {NameEntry.Text}";
        }
    }
}
```

5. Run the project. The form should look as shown in *Figure 1.15*:

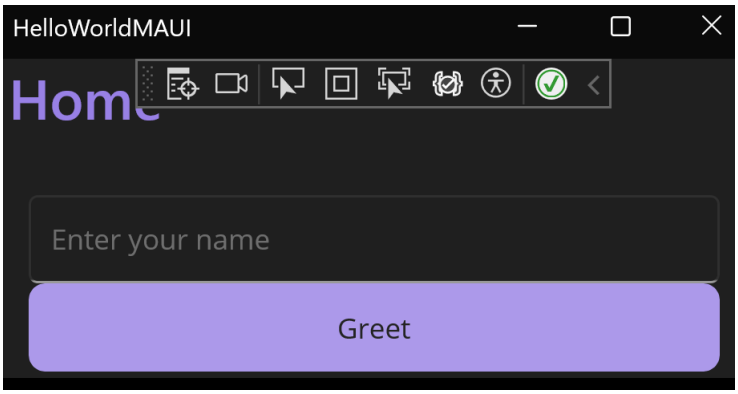


Figure 1.15 – Final MAUI GUI

In this code, the `OnGreetButtonClicked` method sets the `Text` property of the `GreetingLabel` to a string that includes the text entered in `NameEntry`.

So, when you enter a name and click the **Greet** button, you should see the greeting message on the label. This is similar to the event-driven implementation of the Windows form example.

WinUI 3

Chapter 8 will discuss the steps needed to run a WinUI 3 project. For comparison, the code looks as follows:

```
<Page
    x:Class= "HelloWorld.GreetPage"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">

    <StackPanel Margin="20">
        <TextBox x:Name="NameTextBox" PlaceholderText="Enter your name" />
        <Button Content="Greet" Click="OnGreetButtonClicked" />
        <TextBlock x:Name="GreetingTextBlock" />
    </StackPanel>

</Page>
```

In this code, note the following:

- The `TextBox` element is where the user can enter their name
- The `Button` element has a `Click` event that calls the `OnGreetButtonClicked` method when the button is clicked
- The `TextBlock` element is where the greeting message will be displayed

Next, you have to implement the `OnGreetButtonClicked` method in the `GreetPage.xaml.cs` code-behind file. Here's what the code might look like:

```
using Microsoft.UI.Xaml;
using Microsoft.UI.Xaml.Controls;

namespace YourNamespace
{
    public sealed partial class GreetPage : Page
    {
        public GreetPage()
        {
            this.InitializeComponent();
```

```
        }

        private void OnGreetButtonClicked(object sender, RoutedEventArgs e)
        {
            GreetingTextBlock.Text = $"Hello {NameTextBox.Text}";
        }
    }
}
```

In this code, the `OnGreetButtonClicked` method sets the `Text` property of `GreetingTextBlock` to a string that includes the text entered in `NameTextBox`.

So, when you enter a name and click the **Greet** button, you should see the greeting message appear in the text block. This is similar to the functionality of the Windows Forms, Blazor WebAssembly, and .NET MAUI code you provided, but in a WinUI 3 context.

Summary

In this chapter, we introduced the concept of GUI development using C#, a powerful and versatile programming language. We explored GUI development through various frameworks, namely, Windows Forms, Blazor, .NET MAUI, and WinUI 3. Each framework offers unique features and capabilities for creating robust and interactive UIs. Windows Forms, for instance, is a tried-and-true framework that has been used for many years to develop desktop applications. On the other hand, Razor is a newer framework designed for building dynamic web pages with C#. .NET MAUI and WinUI 3 are exciting frameworks that allow developers to create cross-platform applications with a single code base. You can write your application once and run it on multiple platforms, including Windows, macOS, iOS, and Android.

Throughout the chapter, we focused on practical implementation, demonstrating the creation of a simple Hello World application that illustrates key concepts in GUI development. By looking at the different implementations of this application across the four frameworks, we explored their similarities and differences. These comparisons will serve as the foundation for a deeper exploration of each framework and its functionality in future chapters.

The Blazor implementation is the one that deviates the most from a traditional C# implementation. MAUI and WinUI 3 are very similar and continue to evolve from their roots in Windows Forms.

In the next chapter, we will explore Blazor and WebAssembly in detail. These technologies are revolutionizing how we develop web applications, and we believe they hold exciting possibilities for the future of GUI development.

2

Introduction to Blazor and WebAssembly

In the previous chapter, we explored the pivotal concepts of interfaces and polymorphism in C#. We learned how interfaces define the contracts for methods and properties, allowing for flexible and reusable code, while polymorphism enables treating objects of different types as instances of a common interface, thus promoting code generalization and reusability. These principles are foundational for creating modular and scalable software systems in C#. In this chapter we explore two important GUI technologies: **Blazor** and **WebAssembly**. We discuss how they work and how they support the development of web applications. Blazor is a free, open-source web framework built by Microsoft. It allows developers to use C# to create interactive web interfaces without writing JavaScript. Using the .NET platform, Blazor will enable developers to create advanced and interactive web user interfaces sharing code and libraries across servers and clients. WebAssembly, also known as "wasm," is a binary format for instructions created for a stack-based virtual machine. It allows high-level languages like C, C++, and Rust to be compiled and run efficiently in web applications on the client. Integrating Blazor and WebAssembly introduces a fresh approach to web application development, providing a comprehensive full-stack development experience utilizing C# and . NET. This enables developers to leverage their existing .NET expertise and efficiently reuse code and libraries from the server-side aspects of their applications.

In this chapter, we will begin by examining the basic principles of Blazor and WebAssembly, their architecture, and how they collaborate. Then, we will create a simple Blazor application running on WebAssembly. Upon concluding this chapter, you will possess a comprehensive knowledge of these technologies and will be equipped to initiate your application development.

To achieve this, we'll cover the following topics:

- Understanding Blazor
- Exploring WebAssembly
- Blazor and WebAssembly Together
- Setting Up Your Development Environment
- Creating Your First Blazor Application
- Debugging and Troubleshooting
- Best Practices and Tips

Technical requirements

This chapter requires access to **Visual Studio Community** (`https://visualstudio.microsoft.com/downloads/`) with the following workloads installed:

- ASP.Net and Web Development
- .Net Multi-platform App UI Development
- .Net desktop development
- Universal Windows Platform Development

The code shown in the examples can be found here: `https://github.com/PacktPublishing/GUI-Programming-with-C-Sharp`.

Understanding Blazor

The term "Blazor" is the result of merging "Browser" and "Razor," the latter being a syntax for generating dynamic web pages using C#. Blazor signifies a notable advancement in web development pioneered by Microsoft. It functions as a freely accessible web framework, enabling the creation of dynamic web user interfaces using C# without needing JavaScript. This integration permits the utilization of C# for client and server coding, facilitating the sharing of code and libraries. Blazor has access to several tools and libraries of the .NET developer platform for developing web applications because it is a part of the widely known web development framework called ASP.NET Core.

Blazor applications operate directly within web browsers through the use of WebAssembly. Since it is the actual .NET working on WebAssembly, developers can reuse code and libraries from the server-side components of their applications. Additionally, Blazor supports server-side execution of .NET, managed by a SignalR connection, handling tasks such as UI updates, JavaScript calls, and event handling.

Blazor ensures a single-page app experience by adhering to the latest web standards. This is achieved as browsers run a combination of .NET, Razor, and HTML via WebAssembly technology.

The following steps are followed when a Blazor app is built and run in a browser:

1. **.NET runtime is loaded**: The browser downloads a small .NET runtime along with your compiled project. This .NET runtime is based on Mono, an open-source version of the .NET Framework that Microsoft maintains.
2. **Blazor bootstraps the app**: The runtime boots the .NET environment, loads and runs your app.
3. **Blazor sets up interactive UI**: UI event handlers are set up automatically, and UI updates are handled efficiently via a sophisticated diffing algorithm.
4. **Blazor handles user interactions**: When the user interacts with the app, Blazor interoperates with JavaScript to update the DOM.

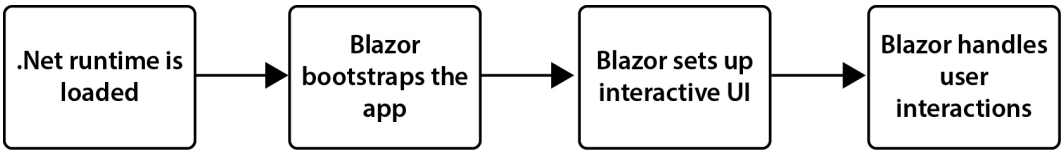


Figure 2.1 – Blazor loading steps

Let us explore each of these steps in more detail.

.NET runtime is loaded

The initial step in launching a Blazor application in a web browser involves loading the .NET runtime. This action is essential because Blazor applications are constructed using .NET and operate within the .NET platform. The .NET runtime serves as a software environment offering various services to applications. These services include managing memory, ensuring type and memory safety, and offering an extensive class library to fulfill diverse programming requirements. Within the realm of Blazor, the .NET runtime is constructed upon Mono, an open-source version of .NET managed by Microsoft. Upon the initial loading of the Blazor application, the browser downloads the runtime alongside the compiled project.

Upon loading the .NET runtime, it initializes the .NET environment and loads the Blazor application before finally running it. So, the Blazor application operates directly within the browser through WebAssembly, allowing developers to utilize C# to write both client and server code. Moreover, sharing code and libraries across servers and clients is also possible.

Blazor bootstraps the app

Once the .NET runtime is loaded in the browser, the second step is the process where the Blazor framework begins initializing and starting an application, which is known as "Blazor bootstraps the app."

Here's a breakdown of what happens during this bootstrap process:

- **Application Initialization**: The Blazor framework sets up the application, configuring its services and settings.
- **Component Initialization**: Blazor determines the components required to render the page. Components in Blazor are self-contained parts of the **user interface (UI)**, such as pages, dialogs, or forms.
- **Render the UI**: Blazor then renders these components onto the page. During this phase, Blazor establishes the required JavaScript interop handlers to enable dynamic updates as a response to user actions or events.
- **Event Handling**: Blazor creates event handlers for user interactions like button clicks or text input. When these events occur, Blazor updates the UI accordingly.

Once the bootstrap process is finished, the Blazor application is fully operational and responsive in the browser. It can interact with user input, send requests to the server, and update the UI dynamically.

Blazor's Management of Interactive UI and User Interactions

The UI rendering and event handling happens iteratively. The process through which Blazor readies the app's UI to be interactive and adaptable to user input is referred to as *Blazor sets up interactive UI* and is a fundamental step in the Blazor bootstrap sequence. After this step comes the *Blazor handles user interactions* part. This is the operation where Blazor responds to the inputs or actions of users, such as clicking buttons, text input, and more. Therefore, it is clear how Blazor plays a pivotal role in putting together an interactive UI and managing user interactions within applications. This encompasses several crucial processes integral to the Blazor framework's functionality:

- **Component Rendering**: Blazor identifies and renders components, which represent UI parts such as pages, dialogs, or forms, onto the web page.

- **JavaScript Interop Setup**: Blazor establishes the required handlers for JavaScript interoperability, facilitating communication between .NET code and JavaScript functions. Blazor needs a route to communicate with the APIs because the browser's **Document Object Model (DOM)** APIs are based on JavaScript, and Blazor facilitates writing client-side logic in C#. Once it has a way to interact, it can update the UI dynamically.
- **Event Binding**: Blazor connects DOM events, such as button clicks or text input, and corresponding .NET methods. As a result, when users interact with the UI, the associated .NET methods are triggered, facilitating the execution of relevant actions or operations.
- **UI Updates and State Management**: When user interactions invoke the linked .NET methods, modifications may be made to the UI linked to them. Utilizing an efficient diffing algorithm, Blazor calculates the differences between the existing and new UI and applies necessary changes to the DOM, ensuring optimal performance.
- **JavaScript Interoperability**: Blazor enables seamless interoperability between .NET and JavaScript, allowing .NET methods to invoke JavaScript functions and vice versa. This capability is invaluable for interacting with browser APIs and performing tasks requiring JavaScript functionality.

Through these processes, Blazor prepares a dynamic and interactive user experience, enabling applications to respond effectively to user input and provide rich functionality.

Why use Blazor?

Using Blazor has several advantages. Here are some of them:

- **Full-stack development with .NET**: Utilizing Blazor allows developers to use their current .NET expertise for client-side development. They can seamlessly share code and libraries between server and client components, streamlining the development process. Additionally, debugging tools commonly utilized for server-side .NET applications apply to Blazor, enhancing development efficiency.
- **Interoperability with JavaScript**: Even though minimizing the reliance on JavaScript is what Blazor strives for, it still integrates effortlessly. This allows developers to leverage the extensive array of JavaScript libraries for client-side UI development while utilizing C# to write logic.
- **Strong typing with C#**: C# aids in identifying and preventing errors during compile-time as it is a statically typed language. Moreover, the developers benefit from IntelliSense functionality in Visual Studio, facilitating quicker and more accurate coding.

- **Performance**: Blazor operates on WebAssembly, enabling near-native speed execution within the browser. This enhances the performance of Blazor applications, contributing to a seamless user experience.

Blazor's combination of these features makes it a powerful tool for modern web development. Another key technology that powers Blazor's impressive capabilities is WebAssembly.

Exploring WebAssembly

WebAssembly, frequently called wasm, embodies a binary instruction structure tailored for a stack-oriented virtual machine. Its primary goal is to act as a portable destination to compile high-level programming languages such as C, C++, and Rust, facilitating their adaptation on the web for various applications ranging from client-side operations to server-side solutions.

What is WebAssembly?

Wasm is a groundbreaking technology revolutionizing web development. It introduces a binary instruction format as a low-level equivalent for executing code on a stack-based virtual machine. Even though it appears complex, WebAssembly enables the execution of code written in multiple languages on the web at speeds nearly comparable to native applications.

Introduced in 2015, WebAssembly swiftly gained popularity due to its performance and efficiency advantages. Its compact binary format is designed for minimal size and fast execution. Both are crucial factors in enhancing user experience on the web.

Beyond its binary structure, WebAssembly is engineered as a low-level virtual machine, running code at near-native speed. This advancement is pivotal in web development, which is traditionally reliant on high-level interpreted languages like JavaScript. WebAssembly can pave the way for web development because it can run demanding web applications like games, video editing, computer-aided design, and scientific visualization in browsers at speeds akin to native applications.

A distinguishing characteristic of WebAssembly is its support for multiple programming languages. While JavaScript has been the predominant language for web development, it may not always be the most suitable option. Developers often prefer to use languages they are most familiar with or better suited to specific tasks. WebAssembly addresses this as a compilation target for C, C++, and Rust. Developers can write code in these languages and then compile it into WebAssembly for web execution.

Additionally, WebAssembly integrates seamlessly with JavaScript, allowing both languages to work harmoniously. This integration is valuable due to JavaScript's extensive collection of libraries and frameworks. With WebAssembly, developers can continue leveraging their preferred JavaScript libraries while implementing performance-sensitive code in other languages.

How Does WebAssembly Work?

WebAssembly works uniquely to deliver performance similar to that of native applications. Here's an overview of how it operates:

- **Compilation**: Source code in C, C++, C#, or Rust is compiled into WebAssembly bytecode—a low-level binary format designed for swift decoding and execution.
- **Delivery**: The compact binary bytecode is then transmitted to the browser via the web. **Decoding and Validation**: Upon reaching the browser, the bytecode gets decoded into a format that the browser's WebAssembly engine understands and also gets validated by the engine to decide whether it is safe to execute.
- **JIT Compilation**: The browser's WebAssembly engine employs **Just-In-Time (JIT)** compilation to convert WebAssembly code into machine code, enabling near-native speed execution.
- **Execution**: At last, the operation of the web application is initialized once the machine code is processed.
- **Interaction with JavaScript**: WebAssembly can seamlessly interact with JavaScript within the same web page, which facilitates the exchange of functions and data and enables developers to utilize the specific benefits of each language.

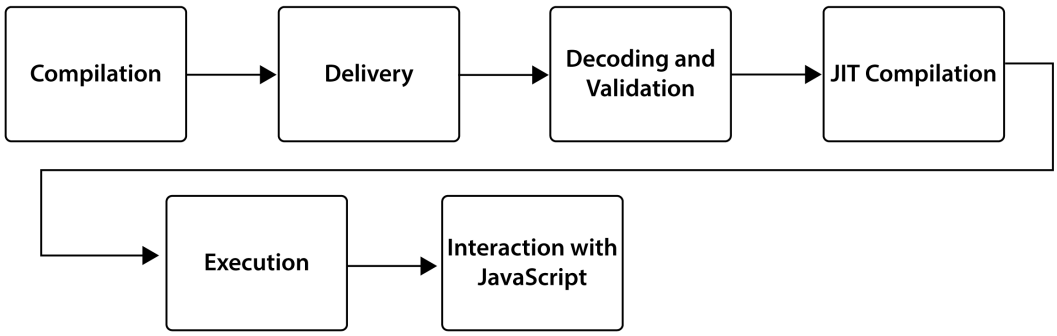


Figure 2.2 – WebAssembly process

An essential aspect of WebAssembly is its focus on security. The validation process ensures adherence to safety standards, while sandboxing prevents unauthorized operations, enhancing overall security within web environments.

Why use WebAssembly?

There are numerous excellent benefits provided by WebAssembly that make it an effective instrument for the development of the web:

- **Performance**: Web assembly code runs at near-native speed by leveraging standard hardware features. It is a noteworthy upgrade over traditional JavaScript, especially beneficial for performance-critical applications such as games and computer-aided design tools.
- **Language Flexibility**: Developers can write code in C, C++, C#, and Rust and then compile that code into WebAssembly. So, more developers who may be proficient in other languages can access web development.
- **Sandboxing**: WebAssembly operates in a sandboxed environment and abides by the browser's same-origin policy and permission model,.
- **Efficient Load Time**: WebAssembly's binary format is smaller and faster to load than corresponding JavaScript code, leading to remarkably faster load times for web applications
- **Integration with the Web Platform**: WebAssembly works alongside JavaScript, sharing APIs and capabilities, enabling developers to use WebAssembly modules like JavaScript modules.
- **Portability**: WebAssembly does not rely on a specific platform, allowing the same code to run across different browsers and operating systems without modifications.

These attributes make WebAssembly a great asset in contemporary web development, enabling browsers to host more sophisticated and performance-intensive apps.

Blazor and WebAssembly Together

Blazor and WebAssembly, when combined, introduce a significant enhancement to web development. Let's take a closer look at how these two technologies collaborate. The framework of Blazor is designed to build dynamic web UIs by utilizing C# rather than JavaScript. Enabling developers to write client and server code in C# promotes code sharing and using a single language throughout the application.

On the other hand, as a low-level binary instruction format, WebAssembly achieves almost native speed on the web and is a compilation target for languages like C, C++, and Rust, which allows browsers to run codes written in these languages. The synergy between Blazor and WebAssembly offers a compelling approach to web application development:

- **Full-stack Development with .NET**: Blazor and WebAssembly enable full-stack development. NET. Developers can utilize C# for client- and server-side code, promoting code sharing and enhancing development efficiency.
- **Near-native Performance**: C# code in Blazor is compiled into WebAssembly bytecode as Blazor operates on WebAssembly and is executed at near-native speed in the browser. This significantly enhances the performance of web applications.
- **Direct Access to Browser APIs**: Blazor facilitates browser APIs, allowing interaction with the DOM and other web APIs directly from C# code, eliminating the need to write JavaScript.
- **Interoperability with JavaScript**: While Blazor and WebAssembly offer robust functionality, there are still areas where JavaScript may still be necessary. Both technologies are designed to seamlessly interoperate with JavaScript, enabling two-way communication between C# and JavaScript functions.
- **Strong Typing and Tooling**: Using C# and .NET provides the benefits of strong typing and comprehensive tooling. Developers can detect errors at compile-time, utilize features like generics and LINQ, and take advantage of solid IDEs such as Visual Studio and Visual Studio Code.

In conclusion, the integration of Blazor and WebAssembly forms a potent combination, merging the productivity of C# and .NET, the performance of WebAssembly, and the extensive ecosystem of JavaScript. Whether transitioning from .NET development to the front end or seeking a robust approach to web application construction, Blazor and WebAssembly present plenty of opportunities.

Improving your Blazor Application

Let's revisit the Blazor App from *Chapter 1*. First, let's delve into the structure of the solution as seen in the following figure.

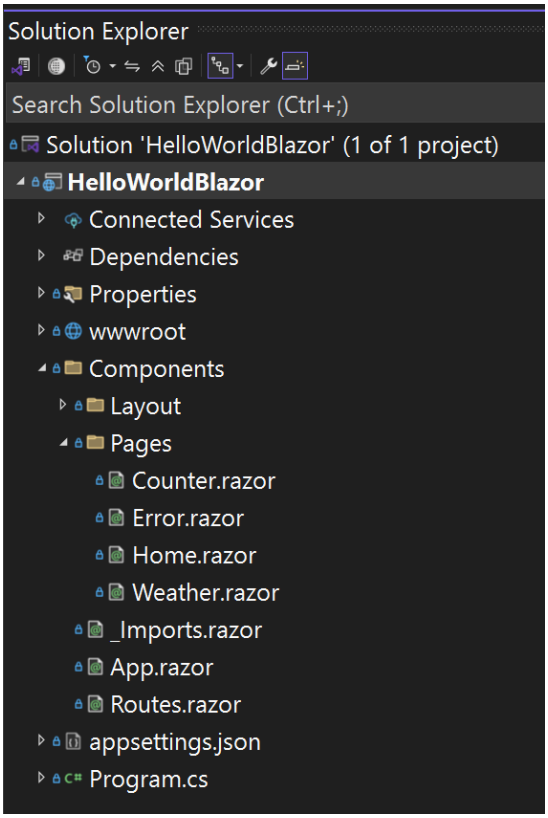


Figure 2.3 – Blazor Project

A typical Blazor application consists of several parts:

- **Project File**: The project file of a Blazor app is relatively straightforward and mainly includes references to any NuGet packages that the app may require. For a Blazor WebAssembly app, the project file targets the `Microsoft.NET.Sdk.BlazorWebAssembly` SDK.
- **Program.cs**: This is the entry point of the Blazor application, akin to a Console app.
- **Startup.cs**: This file is used to configure the app's services and the app's request pipeline.
- **wwwroot Folder**: This folder consists of static files, such as HTML, CSS, images, and JavaScript.

- **Pages Folder**: This folder comprises the app's routable components pages. These components are rendered based on the current URL.
- **Shared Folder**: This folder contains components shared across numerous app pages, such as the layout, navigation menu, and so on.
- **_Imports.razor**: This file contains standard namespace imports utilized across several components, saving you from importing them in every component.
- **App.razor**: This is the root component of the app. It sets up routing.

In a more complex solution, there might also be extra undertakings, including a shared library consisting of a shared object model, an API project for data access, and one or more projects for testing. As each of these parts plays a vital role in the app, understanding them may help you navigate and sort out your Blazor projects in a helpful manner.

Let's modify the app to calculate the division of two numbers. To achieve that, modify the `Pages/Home.razor` code to look as follows:

```
@page "/"
@rendermode InteractiveServer

<h3>Simple Division Calculator</h3>
<input type="number" @bind="Number1" placeholder="Enter first number" />
/
<input type="number" @bind="Number2" placeholder="Enter second number" />
<button @onclick="Calculate">Calculate</button>
<p>@Result</p>

@code {
    double Number1 { get; set; }
    double Number2 { get; set; }
    string Result { get; set; }

    void Calculate()
    {
        var res = Number1 / Number2;
        Result = $"The result is {res}";
    }
}
```

When you run it using the run button:

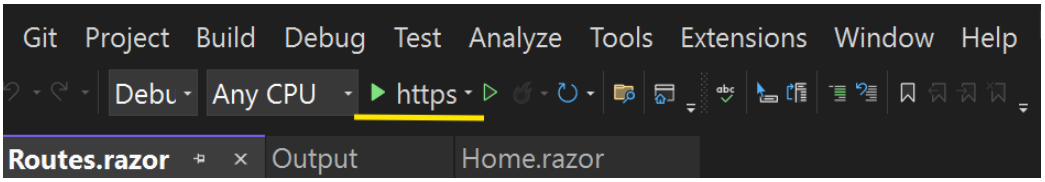


Figure 2.4 – Blazor Application Run Button

The application running should look like this:

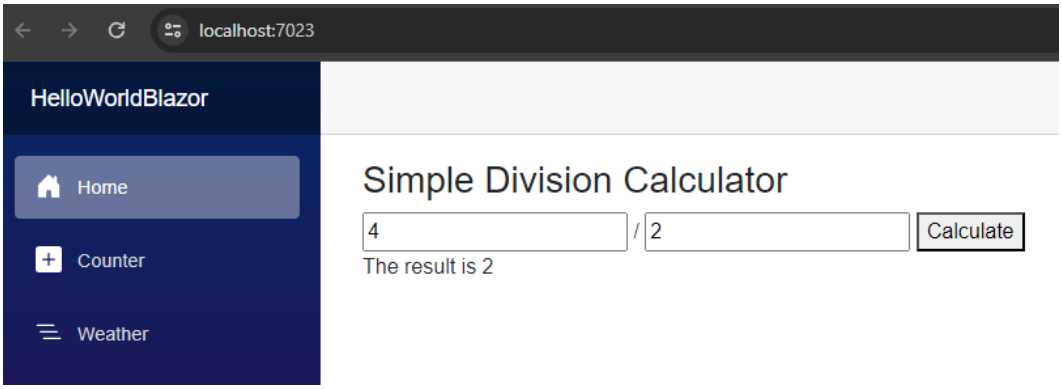


Figure 2.5 – Blazor Application Running

Now, let's explore the code line by line.

```
@page "/"
```

This directive specifies the route for this component. In this case, it's the root ("/") route, which means this component will be served when the user navigates to the application's root.

```
<h3>Simple Division Calculator</h3>
```

This simple HTML heading will display `Simple Division Calculator` on the page.

```
<input type="number" @bind="Number1" placeholder="Enter first number" />
/
<input type="number" @bind="Number2" placeholder="Enter second number" />
```

These are two HTML input elements for the type `"number"`. The `@bind` directive creates a two-way data binding between the input elements and the `Number1` and `Number2` properties. When the user enters a number in these fields, the corresponding properties are updated with those values.

```
<button @onclick="Calculate">Calculate</button>
```

This is a button that, when clicked, will call the `Calculate` method.

```
<p>@Result</p>
```

This paragraph element will display the value of the `Result` property. The `@` symbol outputs the value of a C# expression in the HTML.

```
@code {
    double Number1 { get; set; }
    double Number2 { get; set; }
    string Result { get; set; }

    void Calculate()
    {
        var res = Number1 / Number2;
        Result = $"The result is {res}";
    }
}
```

This code block defines the properties and methods for this component. `Number1` and `Number2` are properties that hold the numbers entered by the user. The result is a property that has the integer result of the division.

The `Calculate` method is called when the button is clicked. It calculates the division of `Number1` and `Number2`, and stores the result in the `Result` property. The result is then displayed on the page.

Ensuring that this functionality works correctly requires effective debugging and troubleshooting techniques. Let's explore the tools and methods available for debugging and troubleshooting in a Razor-based application.

Debugging and troubleshooting

Debugging and troubleshooting play crucial roles in the development process when using Razor, a markup syntax for integrating server-based code into web pages. Here's a comprehensive overview of how debugging and troubleshooting can be conducted in a Razor-based application.

Displaying information for debugging

You can display information that helps analyze and debug pages. This can be done by embedding output expressions to display page values:

```
@page "/"
@rendermode InteractiveServer
@inject ILogger<Home> Log

<h3>Simple Division Calculator</h3>

<input type="number" @bind="Number1" placeholder="Enter first number" />
/
<input type="number" @bind="Number2" placeholder="Enter second number" />

<button @onclick="Calculate">Calculate</button>

<p>@Result</p>

@code {
    double Number1 { get; set; }
    double Number2 { get; set; }
    string Result { get; set; }

    void Calculate()
    {
        var res = Number1 / Number2;
        Log.LogInformation(res.ToString());
        Result = $"The result is {res}";
    }
}
```

This new code works as follows.

```
@inject ILogger<Home> Log
```

This line is injecting an `ILogger` service into the component. `ILogger` is a built-in logging interface in .NET. The `<Home>` specifies the category name for the logger, which is usually the class where the logger is being used. In this case, it's the `Home` component.

```
Log.LogInformation(res.ToString());
```

This line is using the `Log` object to log an informational message. The `LogInformation` method is used to log information that may be useful in tracking the flow of the application and diagnosing issues. In this case, it's logging the result of the division operation. This information appears in the Visual Studio Output Window by default.

The `ILogger` interface provides several methods for logging, such as `LogDebug`, `LogError`, `LogWarning`, and `LogCritical`, each representing a different severity level.

Using diagnostic tools

Razor offers diagnostic tools such as the `ServerInfo` helper, this tool provides an overview of the web server environment hosting your page. It also presents HTTP request information when a browser requests the page.

In Razor Pages, the `@addTagHelper` directive integrates helpers and reusable components containing code and markup for tasks. However, Razor doesn't have a built-in server info helper. Typically, server information is retrieved within your code and displayed on your Razor page if you wish to display server information.

You can use the `@addTagHelper` directive to add in a custom server info helper if you have built or installed one from a NuGet package. This would usually be added to a `_ViewImports.cshtml` file, designed to contain common Razor directives you wish to be available on all Razor pages in your application.

Here's an example of how you might exhibit server information in your Razor page:

```
@page "/"
@rendermode InteractiveServer
@inject ILogger<Home> Log
@inject Microsoft.AspNetCore.Hosting.IWebHostEnvironment Environment

<h3>Simple Division Calculator</h3>
```

```
<input type="number" @bind="Number1" placeholder="Enter first number" />
/
<input type="number" @bind="Number2" placeholder="Enter second number" />

<button @onclick="Calculate">Calculate</button>

<p>@Result</p>

<p>Server Information: @Environment.EnvironmentName running on @
Environment.ApplicationName</p>
```

This modified version showcases the information regarding the web hosting environment where the application is being run as it is injected with `IWebHostEnvironment`. This is used to show the name of the environment and the application on the page.

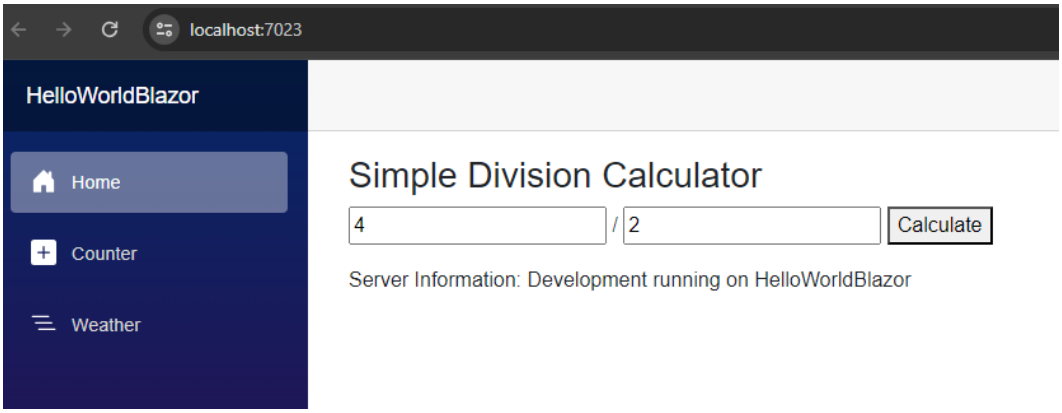


Figure 2.6 – Blazor Application UI

Issues with Running Pages

`.cshtml` and `.vbhtml` pages may not run correctly due to several issues. Reasons include server configuration problems, troubles with Razor code, security and membership issues, and more.

Remember that it is necessary to have a systematic approach during debugging and troubleshooting. Begin by determining the issue, and afterwards attempt to reproduce it steadily. Once you recreate it, you can examine the reason for the problem and try out possible solutions.

Using debugging tools in Visual Studio

Visual Studio offers robust debugging tools that facilitate the debugging process for your Razor pages. These tools allow you to establish breakpoints, navigate your code step by step, examine variables, and more. To use this tool, create a breakpoint by clicking on the left side of the code, as shown in line 23 in *Figure 2.7*.

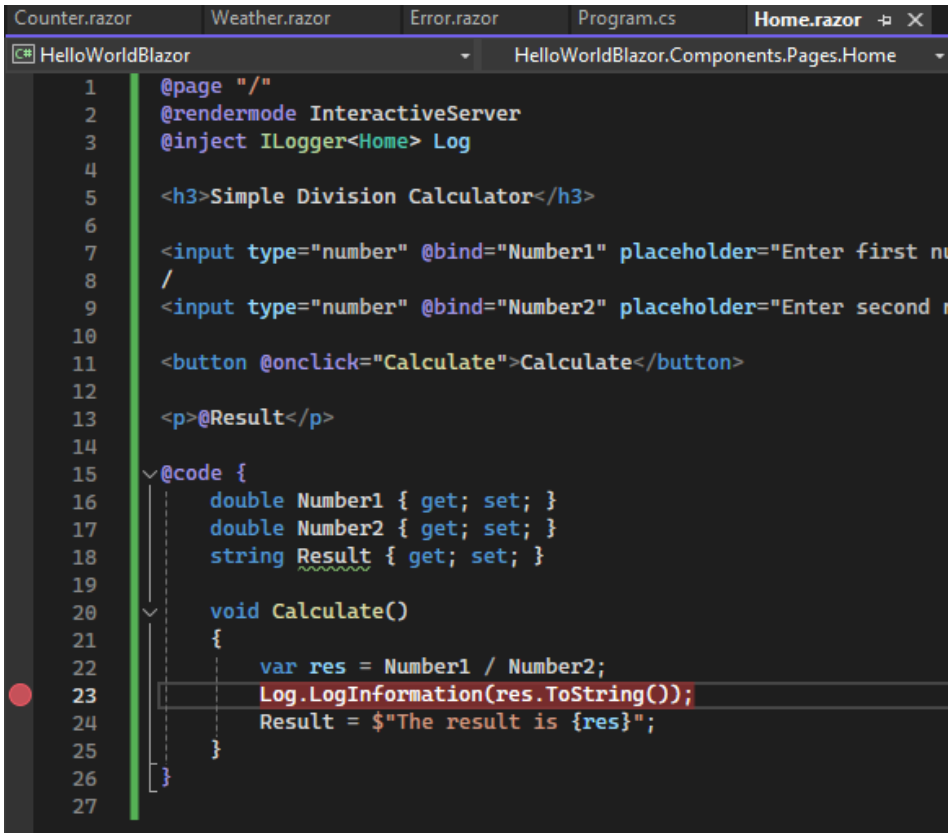


Figure 2.7 – Blazor Debugging with Breakpoint

Then run the project, input data, and click **Calculate**. The code execution will stop at the breakpoint, as shown in the following figure.

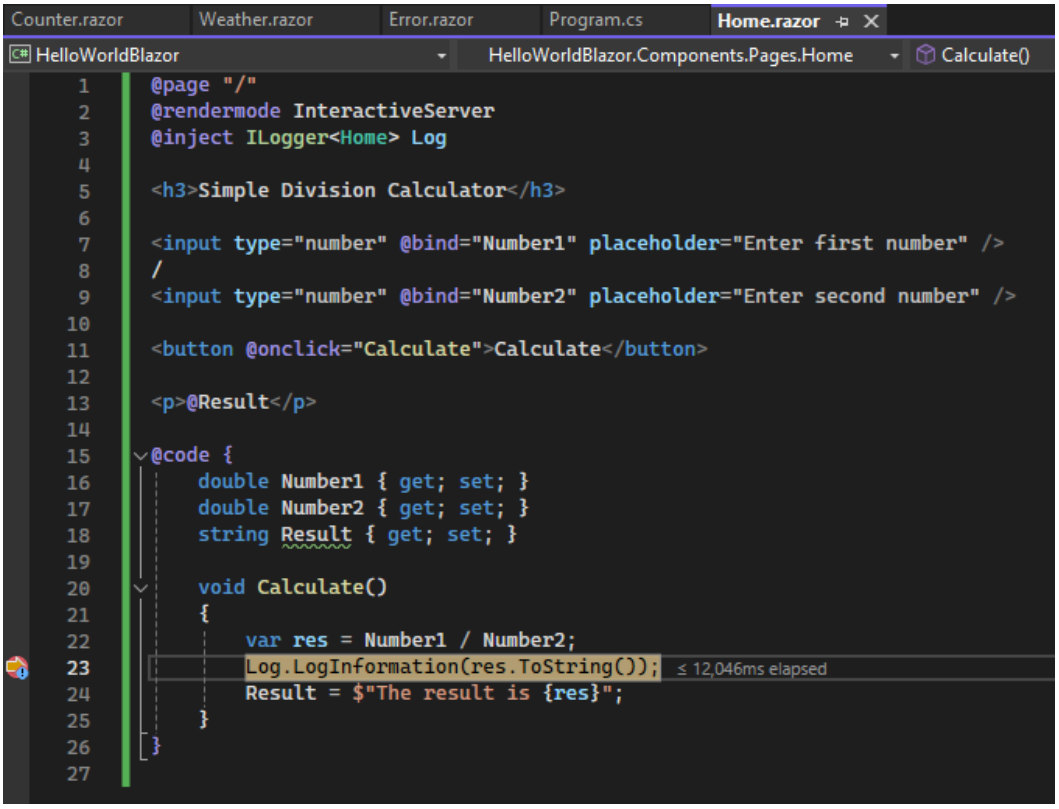


Figure 2.8 – Code stopped at a breakpoint

Identify the value of the `res` variable by hovering over it, as shown in the following figure.

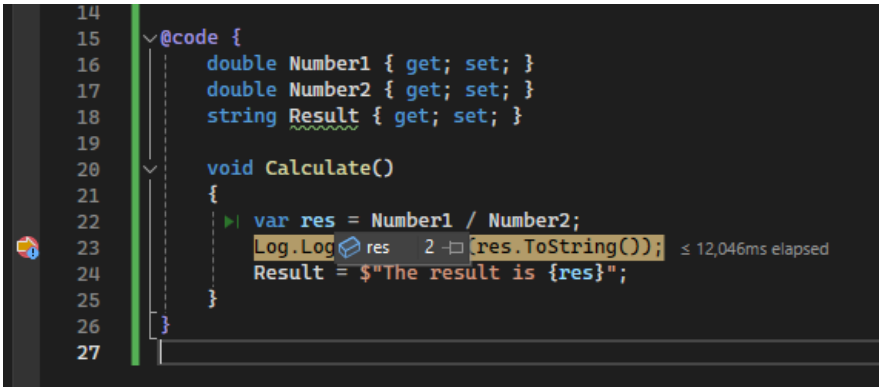


Figure 2.9 – Looking at a variable value while debugging

Now that we are able to identify debug errors, let's explore how to handle the errors when we are not debugging.

Handling errors

Avoiding mistakes right from the beginning is one of the crucial attributes of troubleshooting errors and issues in your code. You can do that by placing sections of your code expected to cause mistakes into try/catch blocks. The try/catch block is utilized to catch and handle exceptions that come about during program execution. Here's how you can alter the `Calculate` method:

```
void Calculate()
{
    try
    {
        var res = Number1 / Number2;
        Log.LogInformation(res.ToString());
        Result = $"The result is {res}";
    }
    catch (DivideByZeroException)
    {
        Log.LogError("Division by zero attempted.");
        Result = "Error: Division by zero is not allowed.";
    }
    catch (Exception ex)
    {
        Log.LogError(ex, "An error occurred while calculating.");
        Result = "Error: An unexpected error occurred.";
    }
}
```

The division operation is placed inside a try block in this modified version. If `Number2` is zero, a `DivideByZeroException` will be thrown. This exception is handled in the first catch block, where an error message is logged, and the `Result` is set to an error message.

If any other exception occurs (for example, if an invalid value is entered), it will be caught by the second catch block. This block catches exceptions of type Exception, the base class for all exceptions. Again, an error message is logged, and the Result is set to an error message.

Best practices and tips

Razor is a markup syntax commonly utilized in ASP.NET MVC and ASP.NET Web Pages to embed server-based code into webpages for generating dynamic web content. The following are several best practices and tips for using Razor effectively:

- **Keep Logic Minimal**: Razor views should strive for simplicity, minimizing the inclusion of intricate logic. It's advisable to relocate complex calculations or business logic to the model or controller rather than embedding it within the view.
- **Use Layouts**: Razor supports layout pages identical to master pages in ASP.NET WebForms. These layouts enable the creation of consistent site templates, ensuring uniformity across all application views.
- **Leverage HTML Helpers**: HTML Helpers in Razor efficiently generate HTML markup in a more neat and organized manner. They considerably reduce the need to type the same HTML tags manually. As a result, it saves a lot of time.
- **Use Partial Views**: Partial views in Razor offer a convenient means to segment views into smaller components, encouraging cleaner views and facilitating code reuse.
- **Encode Output for Security**: Razor automatically encodes all output expressions to repel **cross-site scripting** (**XSS**) attacks. However, manually crafted output expressions should be encoded using the `Html.Encode` method to bolster security.
- **Comment Your Code**: Despite Razor syntax's clean and minimalistic nature, code commenting is vital, particularly for documenting complex expressions or custom helper methods.
- **Organize Your Files**: Maintain the organization of your views by situating them within the designated Views folder. As the application grows, this practice would facilitate ease of navigation and management.

Effective Razor coding revolves around maintaining simplicity in views and prioritizing data presentation over complex logic or computations.

Summary

This chapter presents an in-depth discussion of Blazor and WebAssembly, two leading technologies transforming the future of web development. Initially, it introduces Blazor as a framework enabling the creation of dynamic web interfaces using C# rather than JavaScript, emphasizing the advantages of code sharing and language consistency across applications. Following this, the discussion moves on to WebAssembly, explaining its function as a low-level, binary instruction format that enhances web application performance by running at near-native speeds.

Furthermore, the chapter explores the collaboration between Blazor and WebAssembly, highlighting their combined potential in facilitating full-stack development with .NET, enabling client-side coding in C#, and achieving enhanced performance through WebAssembly.

Additionally, practical guidance is provided for improving Blazor applications, covering functionality improvement, performance optimization, and usability enhancement. The chapter also offers valuable insights into debugging and troubleshooting for both Blazor and WebAssembly, addressing common issues, methods of identification, and effective resolution strategies.

Lastly, a collection of best practices and tips is presented, offering guidance on maintaining clean and efficient code, organizing projects systematically, and more. This chapter is a comprehensive educational resource for individuals seeking to understand and effectively utilize Blazor and WebAssembly in their web development projects.

With this solid foundation in Blazor and WebAssembly, we are now ready to dive deeper into the specifics of GUI development in C#. This chapter will introduce you to the powerful features of graphical user interfaces, enabling you to create visually appealing and highly functional web applications using Blazor.

3

Building Your First Blazor Web Application

In this chapter, we will begin by examining the basic principles of Blazor and WebAssembly, their architecture, and how they collaborate. Then, we will create a simple Blazor application running on WebAssembly. Upon concluding this chapter, you will possess a comprehensive knowledge of these technologies and be equipped to initiate your application development. To achieve this, we'll cover the following topics:

- Understanding the `Program.cs` file
- Understanding the other files in the solution
- Creating the application
- Component life cycle methods
- Using the component
- Setting up a sample API
- Introducing Blazor server-side state management

Technical requirements

This chapter requires access to **Visual Studio Community** (`https://visualstudio.microsoft.com/downloads/`) with the following workloads installed:

- ASP.NET and web development
- .NET Multi-platform App UI development
- .NET desktop development
- Universal Windows Platform Development

The code shown in the examples can be found here: `https://github.com/PacktPublishing/GUI-Programming-with-C-Sharp`.

Understanding the Program.cs file

To better understand its structure and functionality, let's explore the `Program.cs` file in a Blazor application. To access the `Program.cs` file, create a Blazor web application following the steps in *Chapter 1*. The Blazor solution looks like the following figure.

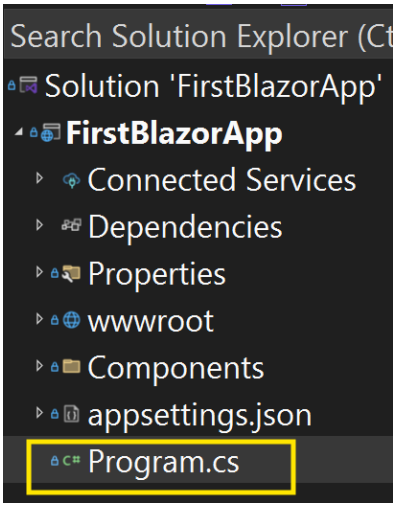


Figure 3.1 – Location of the Program.cs file in the solution

The structure of the files is explained in *Figure 3.1*.

Now that we have identified the location of the `Program.cs` file, let's look at its content.

The `Program.cs` file starts with importing the necessary namespace for the application. The `FirstBlazorApp.Components` namespace contains the application's main component and is imported using this code:

```
using FirstBlazorApp.Components;
```

After including the necessary libraries, the file initializes a new instance of the `WebApplicationBuilder` class, which provides default configurations, including configuration and services. The code looks like this:

```
var builder = WebApplication.CreateBuilder(args);
```

Once the builder is created, the code completes two steps:

- `AddRazorComponents()`: This registers services required for Razor components
- `AddInteractiveServerComponents()`: This adds support for interactive server components, enabling server-side rendering and interactivity

The code is as follows:

```
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();
```

These tasks are followed by building the `WebApplication` host, which is used to configure the HTTP request pipeline and other middleware. The code is as follows:

```
var app = builder.Build();
```

Now, it is time to configure the HTTP request pipeline; this step consists of the following:

- **Environment check**: This block checks if the application runs in a non-development environment
- `app.UseExceptionHandler("/Error", createScopeForErrors: true)`: This sets up a global exception handler to redirect to the `/Error` page in case of an error
- `app.UseHsts()`: This enforces **HTTP Strict Transport Security** (**HSTS**) to add a layer of security to the application

The code is as follows:

```
if (!app.Environment.IsDevelopment())
{
    // The default HSTS value is 30 days
    app.UseExceptionHandler("/Error", createScopeForErrors: true);
    app.UseHsts();
}
```

After the pipeline is configured, the next step is to configure the middleware by completing these steps:

- `app.UseHttpsRedirection()`: This redirects HTTP requests to HTTPS
- `app.UseStaticFiles()`: This serves static files from the `wwwroot` folder
- `app.UseAntiforgery()`: This adds anti-forgery token validation to protect against **cross-site request forgery** (**CSRF**) attacks

The code looks as follows:

```
app.UseHttpsRedirection();

app.UseStaticFiles();
app.UseAntiforgery();
```

Once the routing and middleware are configured, the razor components are mapped following these steps:

- `app.MapRazorComponents<App>()`: This maps the `App` component as the root component for the Razor components
- `AddInteractiveServerRenderMode()`: This adds support for server-side rendering and interactivity for the Razor components

The code looks as follows:

```
app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();
```

At this point, the application is ready to run by starting the web server and listening for incoming HTTP requests. The code looks as follows:

```
app.Run();
```

Understanding the other files in the solution

When working with Blazor, it's essential to comprehend the complete structure of a Blazor solution. Each file and folder within a Blazor project serves a specific purpose; understanding these roles is crucial for effectively developing and maintaining your applications.

In this section, we will explore the various files and directories that constitute a Blazor solution. By examining the entire structure, as illustrated in *Figure 3.2*, you will gain a comprehensive overview of the components of a Blazor project. This knowledge will help you navigate the project more efficiently, understand how different parts interact, and manage and expand your applications with greater ease.

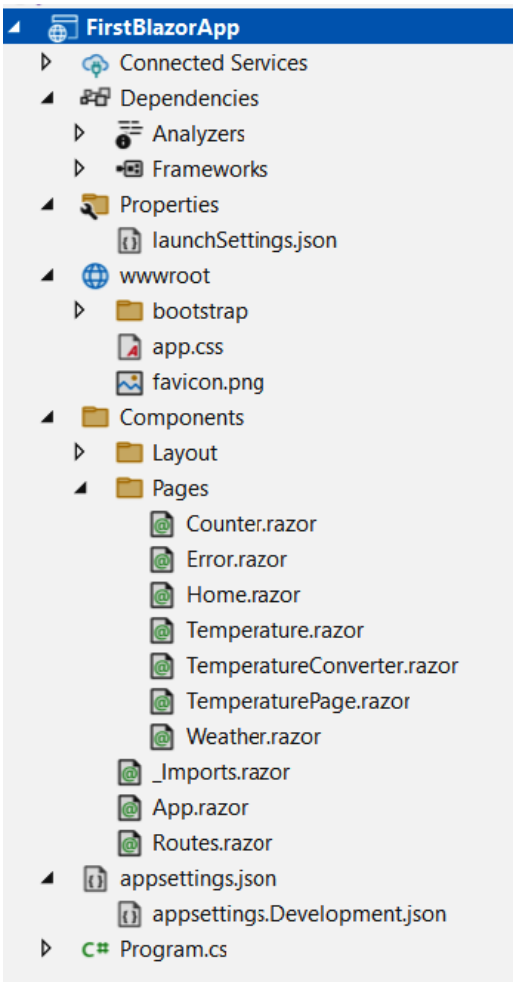


Figure 3.2 – Structure overview of a Blazor application

Here is an explanation of each of the components in *Figure 3.2*:

- **FirstBlazorApp:** This is the root of your Blazor project. It encompasses all the files and folders necessary for your application to run.
- **Connected Services**: A feature in Visual Studio that allows you to connect your application to various cloud services, such as Azure. This can include services such as databases, authentication, and other cloud-based resources.

- **Dependencies**:
    - **Analyzers**: Tools that provide code analysis to help you write better code. These can offer real-time feedback and suggestions to improve code quality and adherence to best practices.
    - **Frameworks**: The .NET frameworks your project depends on. This includes all necessary libraries and packages required for your application to function.
- **Properties**:
    - `launchSettings.json`: Configuration settings for launching your application. It defines profiles for different environments (e.g., IIS Express, Kestrel) and settings, such as application URL and environment variables.
- **wwwroot:**
    - This folder is the web root for your Blazor application, containing static files that will be served directly to clients. This includes assets such as images, CSS, and JavaScript files.
    - `bootstrap`: This contains Bootstrap CSS files for styling, providing a responsive and modern design framework.
    - `app.css`: Custom CSS file for your application. Here, you can define styles that are specific to your application.
    - `favicon.png`: Icon file for the application, displayed in the browser tab.
- **Components**: This folder contains reusable UI components. Components are building blocks of Blazor applications:
    - **Layout**: This contains layout components that define the overall structure of your application
    - `MainLayout.razor`: The main layout component typically includes the structure of the main page, such as headers, footers, and sidebars
    - `NavMenu.razor`: A component for the navigation menu, which provides links to different pages in the application

- **Pages**: This folder contains Razor components representing individual application pages. Each `.razor` file corresponds to a page users can navigate:
    - `Counter.razor`: A page with a counter component, often used as a simple example to demonstrate Blazor functionality
    - `Error.razor`: A page to display errors, providing user-friendly error messages
    - `Home.razor`: The home page of your application, usually the first page users see
    - `Weather.razor`: A page displaying weather information, typically used as a sample data-driven page
- `_Imports.razor`: A file to include common Razor directives for all components in the folder, reducing redundancy.
- `App.razor`: The root component of your application, which contains the `Router` component to handle navigation.
- `Routes.razor`: This defines the routes for your application, mapping URLs to components.
- `appsettings.json`: A configuration file for your application settings, such as connection strings, API keys, and other configuration data.
- `appsettings.Development.json`: A configuration file for development-specific settings. This file can override settings in `appsettings.json` when running in the development environment. This file is hidden and can be seen by clicking on the arrow beside `appsettings.json` to expand and see this file.
- `Program.cs` is the entry point of your application, where you set up the application host, services, and middleware. It configures the HTTP request pipeline and starts the web server.

Creating the application

Razor is a markup syntax that facilitates the inclusion of the embedding of server-based code into web pages. Blazor adopts this syntax to generate dynamic web content with the programming language C#. To explain this, we'll create a temperature converter. To make the component, right-click on **Pages** in the solution and then select **Add**, and then **Razor Component...**, as seen in *Figure 3.3*.

Name the file temperature `TemperatureConverter.razor`.

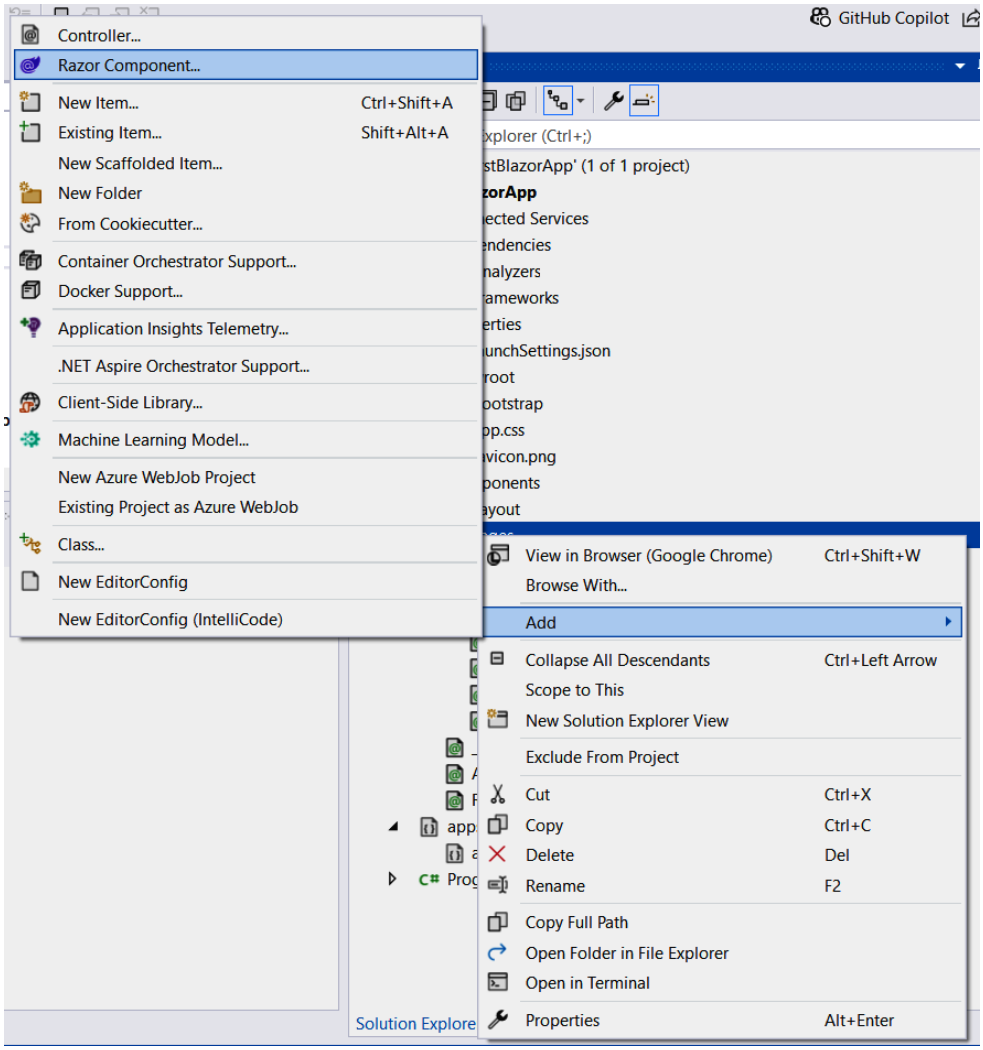


Figure 3.3 – Adding a Razor component to the Pages folder

Overwrite the existing code and add this code to the file:

```
@page "/temperatureconverter"
@rendermode InteractiveServer

<h3>Temperature Converter</h3>
<p>Enter temperature in Celsius:</p>
```

```
<input type="number" @bind="CelsiusTemp" @bind:event="oninput" />
<p>Temperature in Fahrenheit: @fahrenheitTemp</p>

@code {
    private double celsiusTemp;
    private double fahrenheitTemp;

    protected override void OnParametersSet()
    {
        ConvertTemperature();
    }

    private void ConvertTemperature()
    {
        fahrenheitTemp = (celsiusTemp * 9.0 / 5.0) + 32;
    }

    private double CelsiusTemp
    {
        get => celsiusTemp;
        set
        {
            celsiusTemp = value;
            ConvertTemperature();
        }
    }

}
```

Run the solution by pressing *F5* or clicking on **Run**.

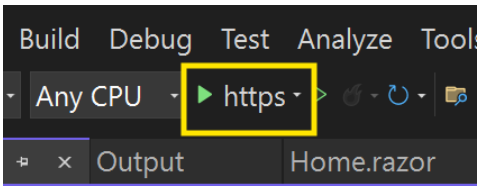


Figure 3.4 – Running a Blazor solution

Navigate to the new component by adding `temperatureconverter` to the URL, as shown in *Figure 3.5*.

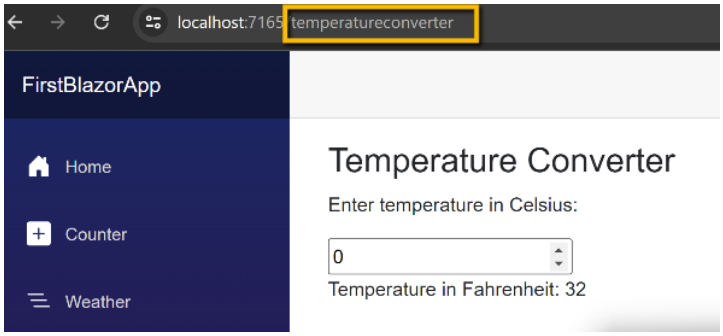


Figure 3.5 – Running a Blazor solution

We will review the code line by line to understand how it works.

Let's explore a Blazor component for the temperature converter application, breaking down each section to understand its purpose and functionality.

At the very top of the file, we encounter the `@page` directive:

```
@page "/temperatureconverter"
```

This directive is crucial as it specifies the URL route for this component. In this case, the component can be accessed through the `/temperatureconverter` URL.

Next, we have the HTML markup, which constructs the framework of our web page:

```
<h3>Temperature Converter</h3>
<p>Enter temperature in Celsius:</p>
<input type="number" @bind="celsiusTemp" @bind:event="oninput" />
<p>Temperature in Fahrenheit: @fahrenheitTemp</p>
```

Here, the `<h3>` tag provides a heading for our temperature converter. The `<p>` tag displays a prompt for the user to enter a temperature in Celsius. The `<input>` element is a field where the user can enter a numerical value, which is then bound to the `celsiusTemp` variable using the `@bind` directive. `@bind:event="oninput"` ensures that this binding is updated whenever the user modifies the input. Finally, another `<p>` tag is used to display the converted temperature in Fahrenheit, which is dynamically updated with the `@fahrenheitTemp` variable.

Moving on to Razor expressions, the `@` symbol signifies a transition from HTML to C# code, for instance:

```
<p>Temperature in Fahrenheit: @fahrenheitTemp</p>
```

In this line, `@fahrenheitTemp` inserts the value of the `fahrenheitTemp` variable directly into the HTML content. The `@bind` directive creates a two-way data binding between the input field and the `celsiusTemp` variable, ensuring that any change in the input field immediately updates the variable, and vice versa.

Lastly, we have the `@code` block, which houses the C# code that dictates the behavior of our component:

```
@code {
    private double celsiusTemp;
    private double fahrenheitTemp;

    protected override void OnParametersSet()
    {
        ConvertTemperature();
    }

    private void ConvertTemperature()
    {
        fahrenheitTemp = (celsiusTemp * 9.0 / 5.0) + 32;
    }
}
```

Within this block, we define two private variables, `celsiusTemp` and `fahrenheitTemp`, to store the temperatures in Celsius and Fahrenheit, respectively. The `OnParametersSet` method is a life cycle method that Blazor calls whenever parameters are set on the component. This method calls `ConvertTemperature()`, which performs the conversion from Celsius to Fahrenheit using this formula: `(celsiusTemp * 9.0 / 5.0) + 32`. This ensures that the temperature conversion is automatically recalculated whenever the input value changes.

By understanding each part of this Blazor component, you can appreciate how the combination of HTML, Razor expressions, and C# code work together to create a dynamic and responsive web application.

Component life cycle methods

The following list outlines the component life cycle's phases as they pertain to this component:

1. **Initialization**: Upon creation, Blazor boots up the component's parameters before setting up its state. In this component, the default value of `0` is used when initializing `CelsiusTemp` and `FahrenheitTemp`.
2. **Parameter setting**: Once the initialization process is completed, Blazor moves on to configuring the component's parameters. This step is of minimum significance as no parameters are being passed in this component.
3. **Rendering**: Once the parameters are configured, Blazor engages in the rendering phase of the component, during which the component's markup is processed. A render tree is generated that reflects the updated user interface. Within the context of this component, the initial render will showcase an input field and demonstrate the correlation between `0` degrees Celsius and its Fahrenheit equivalent, which is `32` degrees.
4. **After rendering**: Following the initial rendering, Blazor triggers the execution of the `OnAfterRender` and `OnAfterRenderAsync` life cycle methods. These methods are particularly beneficial for conducting operations that are reliant on the component's rendering phase (e.g., JavaScript interop calls). However, as this component does not override these methods, no operations are undertaken in this step.
5. **Parameter updating**: Upon detecting any modifications to the component's parameters performed by the parent component, Blazor repeats the parameter setting, rendering, and post-rendering stages. Moreover, a re-render is prompted whenever the `celsiusTemp` parameter is altered due to changes made by a user in the input field.
6. **Disposal**: Blazor executes its disposal procedure and releases any associated resources once the component is no longer required. However, as this component does not override the Dispose method, no particular actions are undertaken at this stage.

Using the component

A new page will need to be created to utilize this component. Let's create a new page under the name `TemperaturePage.razor`. The temperature converter component can be used on this page in this fashion:

```
@page "/temperaturepage"
<h1>Welcome to the Temperature Converter Page!</h1>
<TemperatureConverter />
```

```
@code {
    // Any additional C# code needed for this page
}
```

In this example, the tag for the temperature converter component is `<TemperatureConverter />`. Upon the identification of this tag by Blazor, the temperature converter component will be rendered in its place.

Keep in mind that the component must reside within the same project or a referenced project. If the element is located in a different namespace, `@using` must be included at the top of the file, specifying the component's namespace.

Additionally, ensure that the route defined in the `@page` directive (`/temperaturepage` in this case) is exclusive and does not overlap with any existing routes in the application.

Routing in Blazor

In Blazor, the mechanism through which a browser's URL is matched to that of a page in the application is referred to as **routing**. Blazor components can define their respective routes by using the `@page` directive located at the top of the component file.

For instance, the line at the top of the temperature converter component is as follows:

```
@page "/temperatureconverter"
```

This indicates that the temperature converter component can be seen upon accessing `/temperatureconverter` in the Blazor app.

Assuming that the temperature converter component needs to be utilized on another page, a new page needs to be created using the following method:

```
@page "/temperaturepage"
<h1>Welcome to the Temperature Converter Page!</h1>
<TemperatureConverter />
@code {
    // Any additional C# code needed for this page
}
```

In this example, the `@page` directive indicates how this page will be presented when navigating to `/temperaturepage` in the Blazor app. The temperature converter component is being utilized within this page, which means **Welcome to the Temperature Converter Page!** will be displayed on this page, with the temperature converter component situated underneath the heading.

Despite this being one of the simple examples, the routing system employed by Blazor is rather dynamic. Routes can be identified with parameters, optional parameters, and catch-all routes, among other things.

Setting up a sample API

In a Blazor application, the `HttpClient` service can call a web API. The following is a simple example of how this might be achieved. Create a Razor page named `Temperature.razor` and add the following code:

```
@page "/temperature"
@inject HttpClient Http
<h3>Current Temperature</h3>
<p>@temperature</p>
@code {
    private string temperature;

    protected override async Task OnInitializedAsync()
    {
        temperature = await Http.GetStringAsync("https://api.example.com/
temperature");
    }
}
```

In this demonstration, the `@inject` directive is utilized to incorporate the HttpClient service into the component. Following this, within the `OnInitializedAsync` life cycle method, a `GET` request is dispatched to the Web API by invoking the `GetStringAsync` method on the `HttpClient`. This thereby facilitates retrieving the current temperature as a string.

It is worth noting that `https://api.example.com/temperature` needs to be replaced with the actual URL of the web API that will be used. A possible replacement API is `https://api.weather.gov/`.

Additionally, it is vital to note that calling a web API from a Blazor application involves asynchronous operations. Therefore, employing the `async` and `await` keywords is necessary to manage the asynchronous code effectively.

Summary

Chapter 3 provided a thorough introduction to building a Blazor WebAssembly application, starting with the basic principles of Blazor and WebAssembly, their architecture, and how they collaborate. It guided the reader through setting up the development environment, creating a simple Blazor application, and understanding the structure and functionality of the `Program.cs` file. The chapter covered essential topics, such as adding services to the container, configuring the HTTP request pipeline, middleware configuration, and mapping Razor components. Additionally, it explained the entire structure of a Blazor solution, including components, pages, and configuration files, while providing a practical example of creating a temperature converter component with detailed code explanations.

As you have seen, mastering the fundamentals of Blazor, including its life cycle methods and event handling, lays a strong foundation for building robust web applications. With this solid base, we can now explore the next essential aspect of Blazor development: Razor components. In the upcoming chapter, you will discover how these versatile building blocks can be leveraged to create dynamic and interactive user interfaces, enhancing the functionality and usability of your web applications.

4

Using Razor Components and Data Binding

In this chapter, we will begin by examining the fundamental principles of Razor components, their architecture, and how they interact within ASP.NET Core. We will then delve into creating and managing Razor components, from setting up the development environment to advanced techniques and state management. By the end of this chapter, you will possess a comprehensive understanding of Razor components and be prepared to implement them in your projects. To achieve this, we'll cover the following topics:

- Understanding Razor components
- Creating Razor components
- Data binding in Razor components
- Managing dynamic data
- Advanced component techniques
- State management in Razor components
- Working with forms and validation
- Styling Razor components

Following the structure and content provided in this chapter will give you the necessary skills to build sophisticated, high-performance web applications using Razor components, empowering you to create top-notch solutions.

Technical requirements

This chapter requires access to **Visual Studio Community** (`https://visualstudio.microsoft.com/downloads/`) with the following workloads installed:

- ASP.NET and web development
- .NET Multi-platform App UI development
- .NET desktop development
- Universal Windows Platform Development

The code shown in the examples can be found here: `https://github.com/PacktPublishing/GUI-Programming-with-C-Sharp`.

Understanding Razor components

One of the features of ASP.NET Core is Razor components. These components enable developers to design interactive web applications by integrating C# and HTML. While most web applications traditionally run server-side, Razor components enable client-side running, giving the user a more seamless and highly responsive experience. These components are created to handle everything from essential updates to the **user interface** (**UI**) to intricate data interactions.

Benefits of using Razor components

Razor components offer several key benefits:

- **Performance**: Razor components run on WebAssembly, allowing for near-native performance in the browser
- **Reusability**: Components can be reused across different parts of an application, reducing code duplication and maintenance
- **Productivity**: Leveraging C# for client- and server-side development streamlines the development process
- **Interoperability**: Razor components can interact with JavaScript libraries and APIs, making them highly versatile
- **Consistency**: Using a single language (C#) for server- and client-side code improves consistency and reduces context-switching

Understanding the development workflow

Before beginning to code, it's essential to understand the development workflow for building Razor components. This typically involves the following steps:

1. **Creating a new project**: Developers can use Visual Studio or the .NET CLI to initiate a new ASP.NET Core project with Razor components.
2. **Adding components**: Incorporate .razor files and incorporate essential markup and logic to define Razor components within the project
3. **Building and running**: Build the project to compile the Razor components and run it locally to test any changes made.
4. **Debugging and testing**: Debugging tools and testing frameworks can be utilized to determine issues, fix them in the Razor components, and ensure they work as they are supposed to.

This development workflow can aid developers in effectively building and testing Razor components within the ASP.NET Core projects.

Creating Razor components

Adopting a structured approach, Razor components allow developers to craft modular and reusable UI elements suitable for seamless integration into web applications. To build effective and sustainable UIs, it is essential to grasp better the basic structure of Razor components and how component parameters are defined.

The basic structure of a Razor component

A Razor component typically consists of two main sections:

- **HTML markup**: This markup defines the structure and layout of the component's UI elements using standard HTML syntax. It may include placeholders for dynamic data and expressions to be evaluated by the Razor engine.
- **C# code block**: This block contains server-side C# code that is executed during the rendering process. It may include logic for data retrieval, manipulation, and conditional rendering based on component state or external inputs.

Here's an example of a simple Razor component that displays a greeting message:

```
<!-- GreetingComponent.razor -->
<div>
    <h1>Hello, @Name!</h1>
</div>
@code {
    [Parameter]
    public string Name { get; set; }
}
```

This example shows a component containing a `div` element. It contains an `h1` heading that displays a greeting message with the value of the `Name` parameter. The `@code` block defines a property named `Name`, and it serves as a parameter for the component.

Defining component parameters

Component parameters allow for dynamic customization of Razor components by passing data into them. Parameters can be defined within the component using the `@code` block and utilized to tailor the component's appearance and behavior, as illustrated in the following example:

```
<!-- GreetingComponent.razor -->
<div>
    <h1>@Message, @Name!</h1>
</div>
@code {
    [Parameter]
    public string Name { get; set; }

    [Parameter]
    public string Message { get; set; }
}
```

In this updated example, a new parameter named `Message` is added to the `GreetingComponent`. This parameter allows the greeting message displayed by the component to be personalized.

Reusing components across pages and applications

Among the many benefits of Razor components is their reusability. As components can be used throughout numerous pages within an application, everything remains consistent, and the risk of duplicating code is decreased.

A Razor component can be reused in any other part of the application by incorporating it within the markup of the desired page or component using the component's tag name, as shown in the following example:

```
<!-- Index.razor -->
@page "/index"
<div>
    <GreetingComponent Name="John" Message="Welcome to our website!" />
</div>
```

In this example, `GreetingComponent` has been included within the application's `Index` page, and values for the `Name` and `Message` parameters have been passed to customize the component's behavior.

In this section, we explored Razor components in ASP.NET Core, which enables developers to create interactive web applications by integrating C# and HTML for both server-side and client-side execution. This approach offers benefits such as enhanced performance, reusability, streamlined development, and interoperability with JavaScript. In the next section, we'll look at data binding and how to add data to Razor components.

Data binding in Razor components

Data binding is crucial in web development to allow communication between the component's UI and the underlying data model. Understanding the numerous methods of data binding supported by Razor components is fundamental for crafting interactive and adaptable UIs.

One-way data binding

Data from the component's state can be synchronized to the UI by using one-way data binding so that any changes in the data get reflected in the rendered output. This is attained through binding component properties to UI elements within the component's markup, as demonstrated in the following example.

This is the same as the sample page that is created by default:

```
<!-- CounterComponent.razor -->
<div>
    <h1>Current Count: @Count</h1>
    <button @onclick="IncrementCount">Increment</button>
</div>
@rendermode InteractiveServer
@code {
    private int Count { get; set; }

    private void IncrementCount()
    {
        Count++;
    }
}
```

This example shows the creation of a `CounterComponent` that displays a count value and a button to increment the count. The `Count` property serves as the component's state, and changes to this property trigger re-renders of the UI.

Two-way data binding

The component's state and UI input elements can have bidirectional synchronization through two-way data binding, which allows for a smooth interaction between the user and the application. This technique is more often than not used in form controls to bind input values to component properties. This technique is frequently used in form controls to bind input values to component properties, as shown in the following example:

```
<!-- TextInputComponent.razor -->
<div>
    <input type="text" @bind="Value" />
    <p>You entered: @Value</p>
</div>
@rendermode InteractiveServer
@code {
    private string Value { get; set; }
}
```

This example shows the creation of `TextInputComponent`, which enables text insertion into an input field. The input value is bound to the value property of the component using the `@bind` directive, which ensures that alterations in the input field are reflected in the component's state.

Event binding

Event binding enables components to respond to user interactions by triggering predefined actions or methods. Event handlers can be attached to UI elements within the component's markup using the `@` symbol followed by the event name, as demonstrated in the following example:

```
<!-- ButtonComponent.razor -->
<div>
    <button @onclick="OnClick">Click me</button>
</div>
@rendermode InteractiveServer
@code {
    private void OnClick()
    {
        // Handle button click event
    }
}
```

In this example, a button component has been created with a button element that triggers the `OnClick` method when clicked. Inside the `@code` block, the `OnClick` method is defined to handle the button click event and perform any necessary actions.

Managing dynamic data

Alongside static data binding, dynamic data binding is also supported by Razor components, where data is fetched from outer sources, such as APIs or databases. Components can, therefore, show data that may get modified over time, such as real-time updates or user-generated content. An example of this is shown here:

```
 <!-- WeatherForecastComponent.razor -->

<div>

    <h1>Weather Forecast</h1>

    @foreach (var forecast in Forecasts)
```

```
    {
        <p>@forecast.Date: @forecast.Temperature °F</p>
    }

</div>
@rendermode InteractiveServer

@code {

    public class WeatherForecast {
        public string Date { get; set; } = "";
        public double Temperature { get; set; } = 0;
    }

    public List<WeatherForecast> Forecasts { get; set; } = new();

    protected override async Task OnInitializedAsync()
    {
        Forecasts.AddRange(new List<WeatherForecast> {
            new WeatherForecast { Date = "Yesterday", Temperature = 72},
            new WeatherForecast { Date = "Today", Temperature = 79},
            new WeatherForecast { Date = "Tomorrow", Temperature = 75}});
    }
}
```

This example demonstrates the creation of `WeatherForecastComponent`. The component displays a list of weather forecasts retrieved from an external service. The `@foreach` loop repeats over the `Forecasts` collection and renders a paragraph element for each forecast.

Comprehending these data-binding methods is vital to constructing dynamic and interactive UIs with Razor components. Harnessing one-way data binding, two-way data binding, event binding, and dynamic data binding will allow developers to build engaging and responsive UIs that actively involve users and optimize their overall experience. In the next section, we explore advanced techniques in Razor components that enhance their flexibility and capabilities, including life cycle methods, data communication between components, templating, and performance optimization.

Advanced component techniques

In addition to basic functionality, Razor components support advanced techniques that enhance their flexibility and capabilities. These techniques include component life cycle methods, passing data between components, using templates, and optimizing component performance.

Component life cycle methods

By utilizing component life cycle methods, developers can insert custom actions or logic into different phases of a component's life cycle, such as initialization, rendering, and disposal. These methods allow components to make changes to the UI per any state modifications. Breakpoints can be used to see the sequence in which the events are called. An example is provided here:

```
<!-- LifecycleComponent.razor -->
<div>
    <h1>@Message</h1>
</div>

@rendermode InteractiveServer
@code {
    private string Message { get; set; }

    protected override void OnInitialized()
    {
        Message = "Component initialized";
    }

    protected override void OnParametersSet()
    {
        Message = "Component parameters set";
    }

    protected override void OnAfterRender(bool firstRender)
    {
        if (firstRender)
        {
            // Perform additional initialization after the component has
been rendered
        }
```

```
    }
}
```

This example creates `LifecycleComponent` and demonstrates how the life cycle methods are used to initialize the component, update its state according to parameters, and execute additional actions after rendering.

Passing data between components

Data can pass through parameters, events, or a shared state management system, which allows Razor components to communicate with each other. This facilitates the development of complex UIs designed with interconnected components that can interact with one another and exchange data as required. Here is the structure of the parent component:

```
<!-- ParentComponent.razor -->

<div>
    <ChildComponent @bind-Value="SharedValue" />
    <p>Shared Value: @SharedValue</p>
</div>

@rendermode InteractiveServer

@code {
    private string SharedValue { get; set; } = "";
}
```

Here is the structure of the child component:

```
<!-- ChildComponent.razor -->

<div>
    <input type="text" @bind="ValueProxy" @bind:event="oninput" />
</div>

@rendermode InteractiveServer

@code {
    [Parameter]
    public string Value { get; set; } = "";
```

```
    [Parameter]
    public EventCallback<string> ValueChanged { get; set; }

    private string ValueProxy
    {
        get => Value;
        set
        {
            Value = value;
            ValueChanged.InvokeAsync(value); // Notify ParentComponent
        }
    }
}
```

This example consists of `ParentComponent`, which includes `ChildComponent`. The parent component conveys a value to `ChildComponent` by utilizing a binding expression, and any modifications made to this value in `ChildComponent` are displayed in the parent component.

Using templates with Razor components

Templates are an effective means for structuring and organizing the layout of dynamic content within Razor components. Flexible and reusable components can be crafted by harnessing templates to adjust to various data sources and scenarios. An example is provided here:

```
<!-- ListComponent.razor -->
<div>
    @if (Items != null)
    {
        <ul>
            @foreach (var item in Items)
            {
                <li>@item</li>
            }
        </ul>
    }
    else
    {
        <p>No items to display</p>
```

```
    }
</div>

@rendermode InteractiveServer
@code {
    [Parameter]
    public List<string> Items { get; set; }
}
```

In this example, `ListComponent` has been created that renders a list of items passed as a parameter. Conditional rendering shows the items in a list if they exist or a message implying that no items are available.

Understanding these advanced component techniques is essential for building more complex and sophisticated UIs with Razor components. By mastering component life cycle methods, data-passing mechanisms, and template usage, developers can create highly customizable and reusable components that meet the needs of their applications. In the next section, we delve into state management in Razor components, covering various approaches from simple component-level state to more complex global state management solutions for sharing state across multiple components.

State management in Razor components

Managing the state is essential for building interactive web applications, and Razor components offer several approaches to state management. These approaches range from simple component-level state to more complex global state management solutions that enable sharing state across multiple components.

Handling state in single components

A component-level state allows individual components to remain independent, ensuring isolation and encapsulation. State changes within a component trigger re-renders of the affected UI elements, resulting in a responsive user experience. The following example demonstrates this concept:

```
<!-- CounterComponent.razor -->
<div>
    <h1>Current Count: @Count</h1>
    <button @onclick="IncrementCount">Increment</button>
```

```
</div>

@rendermode InteractiveServer
@code {
    private int Count { get; set; }

    private void IncrementCount()
    {
        Count++;
    }
}
```

In this example, `CounterComponent` has been created to maintain a count value at its state. The `IncrementCount` method can be triggered by button clicks, incrementing the count value, and further setting off a re-render of the UI to showcase the updated count.

Sharing state across multiple components

Some cases require states to be shared across multiple components within an application. The shared state management techniques can be used for components to synchronize their state and remain consistent throughout the application. The following example illustrates this approach. Here is the parent component:

```
<!-- ParentComponent.razor -->
<div>
    <ChildComponent @bind-Value="SharedValue" />
    <p>Shared Value: @SharedValue</p>
</div>

@rendermode InteractiveServer
@code {
    private string SharedValue { get; set; }
}
```

Here is the child component:

```
<!-- ChildComponent.razor -->
<div>
    <input type="text" @bind="Value" />
```

```
</div>

@rendermode InteractiveServer
@code {
    [Parameter]
    public string Value { get; set; }
}
```

This example shows `ParentComponent`, including `ChildComponent`. The former passes a shared value to the latter through a binding expression and allows both components to share the same state.

Using state management libraries

Developers can use third-party libraries, such as Flux or Redux, for more intricate state management requirements. These libraries provide advanced features and patterns to manage state in large-scale applications, including support for unchangeable data structures and time-travel debugging. The following example demonstrates a simple state management setup that could be extended with such libraries:

```
<!-- CounterComponent.razor -->
<div>
    <h1>Current Count: @counterState.Count</h1>
    <button @onclick="IncrementCount">Increment</button>
</div>

@code {
    private CounterState counterState { get; set; }

    protected override void OnInitialized()
    {
        counterState = new CounterState();
    }

    private void IncrementCount()
    {
        counterState.Count++;
    }
```

```
    public class CounterState
    {
        public int Count { get; set; }
    }
}
```

This example showcases the creation of `CounterComponent`, which manages its state by utilizing a nested class named `CounterState`. The component initializes an instance of `CounterState` in the `OnInitialized` life cycle method and updates its count value in response to user interactions.

Developing more reliable and scalable web applications with Razor components requires a strong comprehension of these state management techniques. Careful assessment of the application's requirements and using state management libraries where needed can allow developers to construct sustainable and practical components that facilitate a smooth user experience. In the next section, we explore how to work with forms and validation in Razor components, focusing on creating forms with strong validation capabilities to ensure that user input meets predefined criteria.

Working with forms and validation

One vital element of web applications is forms, which allow users to insert data and interact with the application. Razor components provide tools and techniques to craft forms with strong validation capabilities and guarantee that user input is correct and complies with predefined criteria.

Razor components simplify the procedure of constructing forms by offering abstractions for common form elements and behaviors. Developers can craft custom form components, such as text inputs, radio buttons, checkboxes, and select dropdowns, tailored to their application's particular needs.

Data validation is required to ensure that user input complies with predefined criteria. It also intercepts invalid data that is introduced to the server. Numerous validation methods are supported by Razor components, such as client-side and server-side validation, to ascertain data integrity and accuracy.

Displaying validation messages

Occurrences of validation make it necessary to provide feedback to the user to help them correct their input. Employing Razor components allows the developers to show validation messages with form fields, aid users in understanding the validation requirements, and correct any issues immediately.

Styling Razor components

A properly designed UI is reliant on both functionality and visual appeal. Razor components offer developers flexibility in styling, enabling them to implement CSS styles and frameworks to improve the appearance of their components and build visually appealing user experiences.

Applying CSS to components

CSS can be applied directly to Razor components using inline styles or referencing external stylesheets. This allows for precise control over the visual presentation of individual components, including layout, typography, colors, and spacing. The following example demonstrates the use of inline styles to style a button component:

```
<!-- ButtonComponent.razor -->
<div>
    <button style="background-color: #007bff; color: #fff; padding: 10px
20px; border: none; border-radius: 5px;" @onclick="ClickHandler">Click
me</button>
</div>
```

This example shows how inline CSS styles can be applied to personalize the appearance of `ButtonComponent`. The styles determine the background color, text color, border, border radius of the button element, and padding and create a visually appealing button design.

Scoped CSS for components

Scoped CSS is vital to ensure that styles determined within a component only apply to that component's markup. It helps to intercept any unintentional conflicts in styles and promotes modularity and encapsulation. Developers can determine specific styles for components through this approach without making unintended changes to other parts of the application. The following example demonstrates the use of scoped CSS within a Razor component:

```
<style>
@CardStyles
</style>

<!-- CardComponent.razor -->
<div class="card">
    <h2>Title</h2>
    <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>
</div>
```

```
@code {
    // Scoped CSS styles
    private string CardStyles => @"
        .card {
            background-color: #f8f9fa;
            border: 1px solid #dee2e6;
            border-radius: 5px;
            padding: 20px;
            box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
        }
    ";
}
```

In this example, scoped CSS styles have been defined for `CardComponent` using a C# property named `CardStyles`. The styles are encapsulated within the component and applied only to the markup rendered by the component, ensuring a consistent and isolated styling experience.

Summary

This chapter expanded on the use of Razor components, which are pivotal for building interactive web applications within ASP.NET Core. It started by exploring the architecture of Razor components, emphasizing their role in integrating C# and HTML to create seamless client-side experiences. The chapter highlighted the benefits of Razor components, such as performance, reusability, productivity, interoperability, and consistency.

Key topics covered included creating Razor components, where the fundamental structure of Razor components was detailed, including defining HTML markup and C# code blocks. It also explained how to create component parameters to enable dynamic customization. Various data binding techniques were discussed, such as one-way data binding for synchronizing component state with the UI, two-way data binding for bidirectional synchronization, and event binding to handle user interactions. The chapter demonstrated how to bind components to dynamic data sources, ensuring real-time updates and user-generated content display. Techniques such as component life cycle methods, passing data between components, and using templates were explored to enhance component flexibility and capability.

Different approaches to state management, from handling state in single components to sharing state across multiple components and using state management libraries, were covered to build interactive and responsive applications. The chapter explained building forms with strong validation capabilities using Razor components, ensuring user input correctness and compliance with predefined criteria. Methods for applying CSS to components, using scoped CSS, and integrating CSS frameworks, such as Bootstrap and Tailwind CSS, were discussed to enhance the visual appeal of components.

By the end of the chapter, readers are equipped with the necessary skills to build sophisticated, high-performance web applications using Razor components, from setting up the development environment to implementing advanced techniques and managing state effectively.

Having established a solid understanding of Razor components and their role in creating dynamic web applications, the next chapter will delve into another robust framework: **.NET Multi-platform App UI (.NET MAUI)**. This framework allows for building native, cross-platform desktop and mobile apps using C# and XAML. You will learn the basics of .NET MAUI, including its benefits, setup, project structure, and how it differs from Xamarin.Forms. Get ready to explore how to create simple cross-platform apps and utilize .NET MAUI's UI framework and coding conventions to extend your development capabilities across multiple platforms.

5

Introduction to .NET MAUI

This chapter will begin by exploring the fundamental principles of .NET **Multi-platform App UI (MAUI)**, including its architecture and how it allows developers to create cross-platform applications. Then, we'll walk through building a functional .NET MAUI application, from setting up the development environment to using core components such as C# and XAML to create a dynamic **user interface (UI)**. You'll learn how to bind data, access native device features, and handle platform-specific code, making your application interactive and responsive. By the end of this chapter, you'll have a deep understanding of creating and managing cross-platform applications using .NET MAUI, ensuring they run smoothly across different platforms.

To achieve this, we'll cover the following topics:

- .NET MAUI
- Building the UI with XAML
- Adding interactivity with C#
- Building a cross-platform application with .NET MAUI

Following the structure and content provided in this chapter will give you the necessary skills to build robust, cross-platform applications using .NET MAUI, allowing you to target Android, iOS, macOS, and Windows efficiently from a single code base.

Technical requirements

To complete this chapter, you need to be able to access **Visual Studio Community** (`https://visualstudio.microsoft.com/downloads/`) and have the following workloads installed:

- ASP.NET and web development
- .NET Multi-platform App UI Development
- .NET Desktop Development
- **Universal Windows Platform** (**UWP**) Development

The code for the examples provided in this chapter can be found in this book's GitHub repository: `https://github.com/PacktPublishing/GUI-Programming-with-C-Sharp`.

.NET MAUI

Microsoft developed the cross-platform framework .NET MAUI to help craft native applications for different platforms, including Windows, iOS, macOS, and Android, through the usage of a single shared code base. By allowing developers to harness C# and **Extensible Application Markup Language** (**XAML**), developers can craft applications within the .NET ecosystem with native performance and enhanced user experience.

.NET MAUI provides a combined set of APIs and tools that can streamline the construction of multi-platform applications and make the development procedure relatively more straightforward to help build, test, and deploy applications across various operation systems to ensure they work as intended.

Evolution from Xamarin.Forms

Xamarin.Forms is a well-known framework that was used to develop multi-platform applications, and .NET MAUI happens to be its evolution. The groundwork of sharing UI and business logic across platforms is quite laid back in Xamarin.Forms, whereas .NET MAUI offers a more modern and streamlined tactic. The following are some crucial improvements in .NET MAUI:

- **Single project structure**: Xamarin.Forms required developers to complete the lengthy task of making separate projects for each platform. However, with .NET MAUI, all platform-specific code and resources are combined into a single project, making the development procedure much more accessible.
- **Performance enhancements**: .NET MAUI has improved performance, offering much quicker startup times and smoother performance during runtime.

- **Handler architecture**: Replacing the renderer architecture in Xamarin.Forms and handlers in .NET MAUI offers a more flexible and efficient way to map cross-platform controls to native views.

Core components — .NET, C#, and XAML

The following are the core components of .NET MAUI:

- **.NET**: .NET is a free-of-charge, open source platform for development that facilitates multiple programming languages and frameworks. As such, it's the backbone of .NET MAUI. It offers an extensive collection of libraries and tools to developers to help them build a broad range of applications from web, desktop, mobile, and cloud-based solutions.
- **C#**: As a contemporary, object-oriented programming language, C# is utilized for formulating application logic in .NET MAUI. It provides strong typing, garbage collection, and a rich range of features, making it a powerful and flexible language for developing cross-platform applications.
- **XAML**: XAML, an XML-based language, defines the UI in .NET MAUI applications. It lets developers craft intricate and dynamic UIs with declarative syntax and separates the presentation layer from business logic.

Benefits of using .NET MAUI

One of the main advantages of .NET MAUI is its capability to construct applications using a single code base and run them across multiple platforms. This reduces the time needed for development and maintenance efforts since developers only have to formulate and update code in one place. .NET MAUI maintains consistency in functionality alongside user experience as code is shared across different platforms, simplifying the overall development procedure.

.NET MAUI offers access to a broad range of libraries, tools, and services as it's intensely consolidated with the .NET ecosystem, and it allows developers to utilize existing .NET skills and resources, which enhances productivity while facilitating smooth interoperability with other .NET technologies, such as the following:

- **ASP.NET Core**: For building backend services and APIs and web apps
- **Entity Framework Core**: For data access and **object-relational mapping (ORM)**
- **Azure**: For cloud services and deployment

A native look and feel is provided by the applications that are crafted with .NET MAUI due to the framework directly mapping cross-platform controls to native widgets on each platform. This tactic ensures that the performance of the applications is optimized and a responsive UI is provided to live up to the expectations of users who are more familiar with native applications. .NET MAUI also supports platform-specific personalization, which enables developers to customize the user experience to the distinct characteristics of each platform.

.NET MAUI supports a streamlined development workflow with features such as hot reload, enabling developers to see real-time changes without the need to restart the application. Notably, this feature improves productivity while decreasing cycles of development. Furthermore, project management is made more straightforward due to the single project structure employed by .NET MAUI, which combines all platform-specific code and resources in one place.

Use cases and applications

.NET MAUI is ideal for developing business applications requiring consistent experience across different platforms. For instance, to ensure that user experience is consistent in sales teams using different devices, a **customer relationship management** (**CRM**) system may harness .NET MAUI. Development time and costs are minimized due to the ability to share code and resources across platforms, which makes it easier to deploy and maintain business applications.

.NET MAUI's cross-platform capabilities are also beneficial for consumer applications such as social media platforms, multimedia applications, and messaging applications. Consistent UI/UX across devices improves user engagement and satisfaction. For example, a photo editing application can utilize .NET MAUI to offer a smooth experience on both desktop and mobile platforms, enabling users to switch between devices without facing any issues.

Integration with numerous internal systems and services is often an application requirement of enterprises. The ability to use existing .NET libraries and frameworks makes .NET MAUI a reasonable choice for enterprise-grade solutions.

While not traditionally used for high-performance games, .NET MAUI can be used for casual games and interactive applications. By integrating .NET MAUI with game development libraries, developers can create immersive experiences.

Creating a cross-platform application with .NET MAUI

Visual Studio offers a range of project templates upon starting a new .NET MAUI project to aid developers in beginning their projects quickly. These templates contain basic setups for various types of applications, such as single-page and multi-page applications and more intricate structures that incorporate numerous services and features.

Creating a new project

Follow these steps:

1. Open Visual Studio and select **Create a new project**.
2. Type `MAUI` to filter the available templates in the project templates search box.
3. Choose a template that suits your needs, such as **.NET MAUI App**.
4. Follow the prompts to name the project and select a location to save it.

Project structure and files

After creating a project, understanding its structure and the aims of each file and folder is crucial. A standard .NET MAUI project comprises the following (among other folders):

- `MainPage.xaml`: The main UI file for the application is written in XAML.
- `MainPage.xaml.cs`: The code-behind file for `MainPage.xaml` contains event handlers and logic.
- `App.xaml`: This file defines global resources and styles for the application.
- `App.xaml.cs`: This file contains the application life cycle methods, such as `OnStart`, `OnSleep`, and `OnResume`.
- **Platforms**:
    - `Android`: Contains platform-specific code for Android
    - `iOS`: Contains platform-specific code for iOS
    - `Windows`: Contains platform-specific code for Windows
    - `MacCatalyst`: Contains platform-specific code for macOS
- **Resources**:
    - `Images`: Stores images and icons used in the application
    - `Styles`: Defines application-wide styles and themes

With a solid grasp of .NET MAUI and its core features, we can now focus on creating user-friendly, cross-platform interfaces. In the next section, we'll explore the UI elements, layouts, and design patterns we can use to build responsive and visually appealing applications with .NET MAUI. So, let's dive into the essentials of UI development.

Building the UI with XAML

The UI for .NET MAUI applications is created using XAML. XAML provides a declarative approach to building UI elements, making it easier to design complex interfaces. Elements in XAML are described with XML-like tags, properties are set using attributes, and a hierarchical structure allows elements to be nested. Here's an example:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="MyApp.MainPage">
    <StackLayout>
        <Label Text="Welcome to .NET MAUI!"
               VerticalOptions="CenterAndExpand"
               HorizontalOptions="CenterAndExpand" />
    </StackLayout>
</ContentPage>
```

This XAML code defines a simple UI for a .NET MAUI application. It uses `ContentPage` as a container for application content. Inside the `VerticalStackLayout` class, `Label` displays a welcome message.

Layouts

Layouts are essential in determining how UI elements are arranged on the screen in a responsive and structured manner. In `StackLayout`, elements are arranged in a single direction, either vertically or horizontally, making it ideal for simple linear layouts. For instance, a vertical `StackLayout` will stack elements on top of each other, such as multiple labels or buttons. Here's how it looks in code:

```
<StackLayout Orientation="Vertical">
  <Label Text="First Label" />
  <Label Text="Second Label" />
</StackLayout>
```

For more complex layouts, `Grid` allows you to place elements in a table-like structure, using rows and columns to create organized designs that can adapt to various screen sizes. This is especially useful when you need to align multiple elements precisely in a grid formation.

On the other hand, `FlexLayout` provides more flexibility by letting you control how elements should wrap or align in multiple directions. This layout is great when you want dynamic, responsive designs where elements should be repositioned based on the available space. For example, you can use `FlexLayout` to justify elements with space between them in a row, as shown here:

```
<FlexLayout Direction="Row" JustifyContent="SpaceBetween">
    <Label Text="Left" />
    <Label Text="Center" />
    <Label Text="Right" />
</FlexLayout>
```

By using the right layout for your UI design, you ensure that your application adapts well to different screen sizes and orientations, providing a smooth user experience on all platforms. Whether for simple stacking, grid-based organization, or flexible dynamic arrangements, .NET MAUI's layouts give you the tools to create responsive and intuitive interfaces.

Controls

The numerous controls that are used throughout the framework are integral parts of the UI in .NET MAUI. So, let's look at some typical controls.

In a UI, `Button` represents a clickable element that triggers an action when pressed. For example, a button can be defined like this:

```
<Button Text="Click Me" Clicked="OnButtonClick" />
```

This button displays the text `Click Me` and triggers the `OnButtonClick` event when clicked.

The `Label` element is used to display static text on the screen. Here's an example of how it can be implemented:

```
<Label Text="Hello, World!" />
```

In this case, the label displays `"Hello, World!"`.

The `Entry` element provides a single-line text input field where users can type information. An example is shown here:

```
<Entry Placeholder="Enter text here" />
```

This entry field has a placeholder that prompts the user with `"Enter text here"`.

A `ListView` element displays a list of items and can be customized with templates. Here's an example of how to bind a list of items with a name and description:

```
<ListView ItemsSource="{Binding Items}">
    <ListView.ItemTemplate>
        <DataTemplate>
            <TextCell Text="{Binding Name}" Detail="{Binding Description}"
/>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
```

In this example, each item in the list is bound to a `Name` and `Description` property from the data source.

Styling and theming enhance the visual consistency of an application. The `Style` element allows you to define reusable settings for controls. For instance, the following code defines a style for labels. It's recommended that this code is added in a file under the `Styles` folder – for example, `Styles.xaml`:

```
<ResourceDictionary>
    <Style TargetType="Label">
        <Setter Property="TextColor" Value="Blue" />
        <Setter Property="FontSize" Value="20" />
    </Style>
</ResourceDictionary>
```

This style changes the text color of all labels to blue and sets their font size to 20.

Similarly, the `Theme` element allows you to define light and dark modes for the application. The following example shows how to apply a different background color, depending on the theme:

```
<ResourceDictionary>
    <Style TargetType="ContentPage" ApplyToDerivedTypes="True">
        <Setter Property="BackgroundColor" Value="{AppThemeBinding
Light=White, Dark=Black}" />
    </Style>
</ResourceDictionary>
```

In this case, the background color has been set to white in light mode and black in dark mode, enhancing the user experience based on the user's preferences.

With the UI in place, the next step is to make the application interactive. In the next section, we'll explore how to handle user actions such as button clicks, text input, and navigation using C#. We'll cover event handling, data binding, and commanding to create responsive, dynamic applications that respond to user input. So, let's dive into adding interactivity to your application.

Adding interactivity with C#

In .NET MAUI, event handling allows the application to respond to user interactions by executing event handlers, which are methods that are triggered when an event occurs. For example, a button click event can be handled by defining an event handler like this:

```
private void OnButtonClick(object sender, EventArgs e)
{
    DisplayAlert("Button Clicked", "You clicked the button!", "OK");
}
```

This method will display an alert when the button is clicked. To connect this event handler to a button, you can use the following XAML code:

```
<Button Text="Click Me" Clicked="OnButtonClick" />
```

This links the button's `Clicked` event to the `OnButtonClick` method, allowing the application to handle user clicks.

In addition to event handling, **data binding** in .NET MAUI connects UI elements to data sources, making it possible for the UI to update and reflect changes in the underlying data dynamically. To set up data binding, you must establish `BindingContext` for the page or control, as shown here:

```
BindingContext = new ViewModel();
```

This binds the view to a view model containing the data to be displayed. In XAML, you can then use binding syntax to connect a UI element to a property in the view model. For instance, we can use the following code to have `Label` display a bound property:

```
<Label Text="{Binding Name}" />
```

This binds the label's `Text` property to the view model's `Name` property, ensuring that any changes in the data are automatically reflected in the UI.

Commanding is another crucial concept in .NET MAUI, especially in the **Model-View-ViewModel (MVVM)** pattern, as it allows reusable actions to be defined. Commands are typically defined in the view model, as shown here. The view model code goes into the `ViewModels` folder, and the file is named after the object I'm referring to – that is, `MyPageViewModel.cs`:

```
public ICommand MyCommand { get; }

public ViewModel()
{
    MyCommand = new Command(ExecuteCommand);
}

private void ExecuteCommand()
{
    // Command logic here
}
```

This defines a command called `MyCommand` that executes the **ExecuteCommand** method when invoked. To use this command in the UI, you can bind it to a button in XAML, as follows:

```
<Button Text="Execute Command" Command="{Binding MyCommand}" />
```

When clicked, the button will execute the command bound to `MyCommand`, running the logic defined in the view model. This separation between the logic and the UI allows for better organization and reuse of code. Unlike handling a simple button click event in the code-behind using a Clicked event (e.g., `Clicked="OnButtonClicked"`), which ties the UI directly to the logic, commands follow the MVVM pattern by moving logic into the view model. This makes the application more testable, maintainable, and scalable. With commands, you also gain built-in support for enabling/disabling UI elements based on the command's state (via `CanExecute`), something that click events do not provide out of the box.

Building a cross-platform application with .NET MAUI

In this example, you'll learn how to build a cross-platform mobile application using .NET MAUI, which allows you to create applications that run on Android, iOS, and Windows using a single code base. The application will consist of several essential components, including a structured UI that separates the visual layer from the business logic, a mechanism for accessing platform-specific features such as device information, and practical techniques for debugging and testing across multiple platforms.

By following this step-by-step approach, you'll gain practical knowledge on managing data, binding it to UI elements, and accessing device features – all while ensuring that your application performs well across different operating systems.

As we progress, you'll create a simple application that displays a list of items, accesses the device name, and responds to user interactions such as button clicks and sensor data from the accelerometer. The application will also be tested on different platforms to ensure it runs smoothly. Along the way, we'll explore critical techniques such as defining models, implementing logic to handle data, and connecting the UI to these elements through binding. Finally, you'll learn how to access native APIs, use debugging tools, and optimize performance to build a well-rounded, functional application. Let's begin by defining the building blocks of your application.

In .NET MAUI, developers often strive to keep a clear separation between the UI and business logic to ensure that applications remain organized and easy to maintain. An excellent approach to achieving this is to structure your application using distinct layers: a model to represent the data, logic for handling it, and a view for displaying it to the user.

To start, the **Model** represents the data. For example, you can create a class to hold the information you want to display, such as items with a name and description:

```
public class Item
{
    public string Name { get; set; }
    public string Description { get; set; }
}
```

This simple class defines the structure for the data, allowing the application to store and manage items with a name and a description.

Next, you'll need a way to organize and manage the data. This is typically done with a **ViewModel** that contains both the data and the logic to interact with it. The view model holds collections of data, like so:

```
public class ViewModel
{
    public ObservableCollection<Item> Items { get; }

    public ViewModel()
    {
        Items = new ObservableCollection<Item>
```

```
        {
            new Item { Name = "Item 1", Description = "Description 1" },
            new Item { Name = "Item 2", Description = "Description 2" }
        };
    }
}
```

In this example, an `ObservableCollection` class of `Item` objects is created and initialized with two items. This allows real-time updates to be made to the UI if the data changes.

Once the data is in place, the **View** displays it to the user. In this case, `ListView` is used to present the items. The UI, defined in XAML, binds to the data and displays each item's name and description:

```
<ContentPage BindingContext="{Binding Source={StaticResource ViewModel}}">
    <ListView ItemsSource="{Binding Items}">
        <ListView.ItemTemplate>
            <DataTemplate>
                <TextCell Text="{Binding Name}" Detail="{Binding
                Description}" />
            </DataTemplate>
        </ListView.ItemTemplate>
    </ListView>
</ContentPage>
```

The `BindingContext` property links the view to the view model, while `ListView` binds to the Items collection. Each Item is displayed using a `TextCell` class that shows the Name and Description properties. In `{Binding Source={StaticResource ViewModel}}`, the `ViewModel` is a resource defined elsewhere in the XAML (typically in `App.xaml`) as a static resource. This tells the page to use that shared instance of the view model as the binding context, enabling the page to access its data and commands.

In addition to managing data, .NET MAUI makes it easy to access platform-specific functionality using **dependency injection (DI)**. Developers can define an interface (e.g., `IDeviceService`) and register platform-specific implementations (e.g., `DeviceService`) in the service container. This allows developers to call native APIs from shared code by defining an interface for platform-specific features.

This code should be placed in files named `IDeviceService.cs` and `DeviceService.cs` in the `Services` folder:

```
using Android.OS;

public interface IDeviceService
{
    string GetDeviceName();
}
```

Once the interface has been defined, you can implement it on each platform. For instance, on Android, you might implement the `IDeviceService` interface like this:

```
using Android.OS;

public class DeviceService : IDeviceService
{
    public string GetDeviceName()
    {
        return Android.OS.Build.Model;
    }
}
```

The `GetDeviceName` method returns the device model for Android devices. To use this service, you must register it with `DependencyService` and call it from your shared code:

```
DependencyService.Register<IDeviceService, DeviceService>();
var deviceName = DependencyService.Get<IDeviceService>().GetDeviceName();
```

This allows the application to access the device's name, regardless of the platform.

You may need to write platform-specific code directly in the shared project for certain scenarios. This can be done using compiler directives to ensure the code only runs on the appropriate platform. Here's an example:

```
string platformMessage;
#if ANDROID
platformMessage = "Running on Android";
#elif IOS
platformMessage = "Running on iOS";
#endif
```

These directives check which platform the code runs on and execute the relevant logic, providing a platform-specific message.

.NET MAUI Essentials offers another way to access cross-platform device features, such as `device information`, `connectivity`, and `sensors`. For instance, you can easily retrieve the device model and manufacturer using the following code:

```
var deviceModel = DeviceInfo.Model;
var manufacturer = DeviceInfo.Manufacturer;
```

Similarly, you can check the network connectivity status:

```
var isConnected = Connectivity.NetworkAccess == NetworkAccess.Internet;
```

To interact with device sensors, such as the accelerometer, you can use the `Accelerometer` class:

```
var accelerometer = Accelerometer.Default;
accelerometer.ReadingChanged += (s, e) =>
{
    var data = e.Reading;
    // Handle accelerometer data
};
accelerometer.Start(SensorSpeed.UI);
```

This enables your application to access and use real-time sensor data, adding interactivity based on device movement.

When developing cross-platform applications, it's essential to test them on all target platforms, such as Windows, Android, and iOS. You can run them on different platforms by selecting **Run** from the toolbar and choosing each of the environments.

Summary

In this chapter, you learned about the fundamental concepts of .NET MAUI, from its architecture to building cross-platform applications that run seamlessly on Android, iOS, macOS, and Windows. We explored how to use C# and XAML to design UIs, implement data binding, and manage events, providing a solid foundation for creating responsive and interactive applications. You also gained practical insights into platform-specific code using Dependency Services, and you learned techniques for debugging and testing your applications across different platforms to ensure consistency and performance. At this point, you should have a solid understanding of creating a basic .NET MAUI application.

In the next chapter, we'll dive deeper into advanced .NET MAUI development techniques. You'll learn how to implement custom renderers, manage complex data states, and integrate more sophisticated platform-specific features. This will enable you to build more robust, feature-rich applications that can handle greater complexity and deliver enhanced user experiences across all platforms. Stay tuned to elevate your cross-platform development skills to the next level.

6

Creating Native, Cross-Platform Apps with .NET MAUI

This chapter explores how to create native, cross-platform applications using .NET MAUI. As the successor to Xamarin.Forms, .NET MAUI offers a more streamlined development experience, allowing developers to write code once and deploy it across multiple platforms, including Android, iOS, Windows, and macOS. With this, developers can use C# and XAML to craft high-performance UIs and app logic while reusing a significant portion of the code base across platforms.

We also cover the benefits of utilizing .NET MAUI, such as code reusability, reduced development costs, and a consistent user experience across devices. Setting up the development environment, understanding the project structure, and employing critical components such as layouts, controls, and data binding are essential for building scalable applications.

As you move through the chapter, you will also learn how to style and theme your app, handle user inputs, integrate APIs and services, and implement accessibility features, ensuring a wide-reaching, inclusive application.

In this chapter, we will explore the following:

- Overview of the app and .NET MAUI
- Setting up the development environment
- Developing the app – creating the project, UI, logic, and events
- Implementing app logic and events
- Advanced UI design techniques
- Testing and deploying the app

Technical requirements

This chapter requires access to **Visual Studio Community** (`https://visualstudio.microsoft.com/downloads/`) with the following workloads installed:

- ASP.NET web development
- .NET Multi-platform App UI development
- .NET desktop development
- Universal Windows Platform development

The code shown in the examples can be found here: `https://github.com/PacktPublishing/GUI-Programming-with-C-Sharp`.

Overview of the app and .NET MAUI

.NET MAUI stands for **.NET Multi-platform App UI**. The main objective of this framework is to create native applications that operate across multiple platforms. This includes platforms such as Android, Windows, macOS, and iOS. It happens to be an evolution of Xamarin.Forms. A cohesive development experience is delivered as it leverages .NET 6. The effort and time required to create and maintain applications across multiple platforms are reduced significantly due to developers only being required to write the code once before deploying the application across various platforms.

By using .NET MAUI, developers can harness the capabilities of both C# and XAML to craft UIs and app logic while reusing a significant part of code across platforms. This framework requires only a single code base comprising the UI, business logic, and data access, making the development procedure of high-performance applications much simpler. A consistent user experience across different devices is also ensured.

Benefits of using .NET MAUI for cross-platform development

The following are some primary benefits of utilizing .NET MAUI for multi-platform development:

- **Code reusability**: The single code base is shared between different platforms. Hence, developers only have to write code once before deploying the application across the platforms.
- **Consistent user experience**: A single set of UI components is used across multiple platforms, ensuring a consistent user experience across devices.
- **Simplified maintenance**: Updating and maintaining the application across different platforms is much simpler. This is because developers no longer have to manage different platform codes.

- **Strong ecosystem**: The .NET ecosystem is very rich. It has extensive libraries, tools, and community support, giving developers ample resources for crafting their applications.
- **High performance**: Access native APIs and controls to create high-performance applications with smooth UIs.

As .NET MAUI is an evolution of Xamarin.Forms, it presents numerous improvements and new features for developers to use. Xamarin.Forms also leveraged C# and XAML to create cross-platform applications. However, it was also necessary to use separate projects for different platforms. On the other hand, .NET MAUI only requires a single project structure for multi-platform development. As such, the development process is more straightforward.

Comparisons between .NET MAUI and frameworks such as **Flutter** are often made to see which is better. The former usually garners attention due to its consolidation with the .NET ecosystem alongside leveraging the capabilities of C#. Developed by Google, Flutter utilizes the Dart language. It offers a different method of cross-platform development. Both frameworks have their advantages. The project's particular needs and the development team's familiarity with the respective frameworks are the deciding factors.

Now that we've explored the benefits and structure of .NET MAUI, it's time to move from theory to practice. Setting up the right development environment is the first step toward building your cross-platform application. In the following section, we'll walk through the tools, SDKs, and configurations needed to get started with .NET MAUI and ensure your project is ready for development across multiple platforms. Let's begin by setting up Visual Studio and configuring the essential resources.

Setting up the development environment

Launch Visual Studio and install the necessary pieces to run your MAUI project. This involves setting up the essential SDKs and emulators for different platforms.

.NET MAUI requires .NET 6 or later, which can be found on the .NET website.

Depending on the targeted platforms, the respective SDKs need to be installed:

- **Android**: Install the Android SDK and set up the Android Emulator:
    - Open Visual Studio and navigate to **Tools | Android | Android SDK Manager** to install and configure Android emulators
- **iOS**: Install Xcode (macOS only) and set up the iOS Simulators:
    - For iOS, open Xcode and configure the iOS Simulators

- **Windows**: Ensure the Windows SDK is installed
- **macOS**: Install Xcode

To verify the installation, create a new .NET MAUI project in Visual Studio to ensure that everything is set up correctly. Various emulators and simulators should support the project's construction and operation.

Understanding the basics of .NET MAUI

A .NET MAUI project follows a specific structure that organizes the code and resources needed for cross-platform development. Understanding this structure is crucial for effective development:

- **Solution file (.sln)**: The solution file contains the entire project and its configurations
- **Project file (.csproj)**: The project file defines the project configuration, dependencies, and target frameworks
- **Platforms folder**: This folder contains platform-specific code and resources for Windows, macOS, iOS, and Android
- **Resources folder**: This folder contains shared resources such as images, fonts, and styles
- **MainPage.xaml and MainPage.xaml.Cs**: The application's main entry points, where the UI and logic are defined
- **App.xaml and App.xaml.cs**: The application's root file that sets up the main page and application resources

Key components and namespaces

.NET MAUI applications utilize various components and namespaces to build UIs and handle application logic:

- **Components pages**: Represent screens or views in the application (e.g., `ContentPage` or `NavigationPage`)
- **Layouts**: Define the structure and arrangement of UI elements (e.g., `StackLayout` or `Grid`)
- **Controls**: Represent UI elements such as buttons, labels, and text inputs
- **Namespaces**: Contain the classes needed, some examples include:
    - `Microsoft.Maui.Controls`: Contains classes for building UIs
    - `Microsoft.Maui.Essentials`: Provides cross-platform APIs for accessing device features

Developing the app – creating the project, UI, logic, and events

To create a new .NET MAUI app, follow the steps outlined here:

1. **Open Visual Studio**: Launch Visual Studio from the Start menu or desktop shortcut.
2. **Create a new project**: Select **Create a new project** from the start page.
3. **Select a project template**: In the **Create a new project** dialog, search for `.NET MAUI App` and select the appropriate template.
4. **Configure project**: Enter the project name, location, and solution name. Click **Create** to generate the project.
5. **Choose platforms**: In the following dialog, select the version of .NET.
6. **Explore the project**: Once the project is created, explore the Solution Explorer to familiarize yourself with the project structure.

Designing the UI with XAML

.NET MAUI uses the declarative approach of XAML to design appealing UIs. It helps developers to craft and define UI element layouts in a clear and organized way. Due to XAML's power and adaptability, minimal coding effort is required to create complex UI designs.

Critical features of XAML include the following:

- **Declarative syntax**: Define UI elements and their properties in a readable XML format
- **Data binding**: Bind UI elements to data sources to create dynamic and responsive interfaces
- **Styles and templates**: Define reusable styles and templates to maintain a consistent look and feel across the application
- **Animations**: Create smooth animations and transitions to enhance the user experience

Using the Grid layout for responsive design

XAML's Grid layout is both powerful and adaptable. It allows the creation of intricate and responsive designs. A grid is organized into rows and columns, providing cells where UI elements can be placed.

To use the Grid layout, follow these steps:

1. **Define the grid**: Use the `<Grid>` element to define the grid layout.
2. Specify the number of rows and columns using the `<RowDefinition>` and `<ColumnDefinition>` elements.
3. **Add UI elements**: Use the grid to place UI elements within the grid `cells.Row` and `Grid.Column` properties.
4. **Span cells**: Use the `Grid.RowSpan` and `Grid.ColumnSpan` properties to span elements across multiple rows or columns.

Here is an example of a XAML Grid layout:

```
<Grid>
  <Grid.RowDefinitions>
    <RowDefinition Height="Auto" />
    <RowDefinition Height="*" />
  </Grid.RowDefinitions>
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="Auto" />
    <ColumnDefinition Width="*" />
  </Grid.ColumnDefinitions>

  <Label Grid.Row="0" Grid.Column="0" Text="Username:" />
  <Entry Grid.Row="0" Grid.Column="1" />

  <Label Grid.Row="1" Grid.Column="0" Text="Password:" />
  <Entry Grid.Row="1" Grid.Column="1" IsPassword="True" />
</Grid>
```

In this example, a simple login form is crafted with a Grid layout. Labels and entries are arranged in rows and columns.

Adding controls

Multiple controls can be added to feature on the UI. This helps in the creation of interactive applications using .NET MAUI. Some common controls include the following:

- To create clickable buttons, use the `<Button>` element:

  ```
  <Button Text="Submit" Clicked="OnSubmitClicked" />
  ```

- To display text, use the `<Label>` element:

```
<Label Text="Hello, World!" />
```

- To create text input fields, use the `<Entry>` element:

```
<Entry Placeholder="Enter your name" />
```

- To display a list of items, use the `<ListView>` element:

```
<ListView ItemsSource="{Binding Items}">
  <ListView.ItemTemplate>
    <DataTemplate>
      <TextCell Text="{Binding Name}" />
    </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

By combining and styling these controls, an enhanced UI can be developed for the application.

Styling and theming the app

Creating visually appealing applications is essential and largely depends on styling and theming. With .NET MAUI, developers can define various styles and themes using XAML, making it easier to maintain a consistent look and feel across the app.

To define reusable styles for controls, use the `<Style>` element:

```
<ResourceDictionary>
  <Style TargetType="Button">
    <Setter Property="BackgroundColor" Value="Blue" />
    <Setter Property="TextColor" Value="White" />
  </Style>
</ResourceDictionary>
```

To apply styles to controls, use the `Style` property:

```
<Button Text="Styled Button" Style="{StaticResource {x:Type Button}}" />
```

For theming, you can define different themes to apply various styles based on conditions such as light and dark mode:

```
<ResourceDictionary>
  <Style TargetType="ContentPage" x:Key="LightTheme">
```

```
    <Setter Property="BackgroundColor" Value="White" />
  </Style>
  <Style TargetType="ContentPage" x:Key="DarkTheme">
    <Setter Property="BackgroundColor" Value="Black" />
  </Style>
</ResourceDictionary>
```

This code defines two styles in a `ResourceDictionary` for theming a `ContentPage` in .NET MAUI. `LightTheme` sets the background color to white, while `DarkTheme` sets the background color to black. These themes can be applied to pages based on the user's preference, such as light or dark mode, allowing the app to dynamically adjust its appearance.

Once the UI is designed, the next step is to implement the app's logic and handle user interactions. In .NET MAUI, C# is used to manage user inputs, trigger events, and execute the underlying functionality of your app. This logic is typically written in the code-behind files associated with your XAML pages, ensuring smooth interaction between the UI and the app's operations.

Implementing app logic and events

In .NET MAUI, C# implements the application's operational logic, including handling user inputs, performing calculations, and interacting with data sources. Typically, this logic is written in code-behind files associated with the XAML pages.

Here's an example:

```
public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();
    }

    private void OnSubmitClicked(object sender, EventArgs e)
    {
        string username = usernameEntry.Text;
        string password = passwordEntry.Text;
        DisplayAlert("Login", $"Username: {username}\nPassword:
        {password}", "OK");
    }
}
```

This example shows how the `OnSubmitClicked` method handles the `Click` event of the `Submit` button to fetch input values. These values are then displayed in an alert.

Handling user inputs and events

Interactive applications rely heavily on handling user inputs and events. .NET MAUI provides a variety of events that developers can subscribe to and manage within their code.

To handle button clicks, use the `Clicked` event:

```
submitButton.Clicked += OnSubmitClicked;
```

To handle text input changes, use the `TextChanged` event:

```
usernameEntry.TextChanged += OnTextChanged;
```

To manage item selection in a `ListView`, use the `ItemSelected` event:

```
itemsListView.ItemSelected += OnItemSelected;
```

This code demonstrates how to handle events in .NET MAUI by linking user interactions with event handlers. The `Clicked` event for a button (`submitButton`) is connected to the `OnSubmitClicked` method, which triggers when the button is pressed. Similarly, the `TextChanged` event for an entry field (`usernameEntry`) is tied to `OnTextChanged`, and the `ItemSelected` event for a `ListView` (`itemsListView`) is linked to `OnItemSelected`, allowing the app to respond to changes or selections made by the user.

Data binding and the MVVM pattern

One of the most vital features of .NET MAUI is data binding, which allows developers to bind UI elements to data sources, creating a robust and responsive UI. The **Model-View-ViewModel (MVVM)** pattern is often implemented to separate the UI, business logic, and data systematically.

The **model** represents the data and business logic. Here's an example:

```
public class User
{
    public string Name { get; set; }
    public string Email { get; set; }
}
```

The **view model** acts as an intermediary between the view and the model. Here's an example:

```
public class UserViewModel : INotifyPropertyChanged
{
    private User _user;
    public User User
    {
        get { return _user; }
        set
        {
            _user = value;
            OnPropertyChanged(nameof(User));
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected virtual void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new
        PropertyChangedEventArgs(propertyName));
    }
}
```

The **view** represents the UI and is defined using XAML. Here's an example:

```
<ContentPage.BindingContext>
    <local:UserViewModel />
</ContentPage.BindingContext>
<StackLayout>
    <Label Text="{Binding User.Name}" />
    <Label Text="{Binding User.Email}" />
</StackLayout>
```

In the preceding code, the name `local` will get resolved to the name of your local project. By employing data binding and the MVVM pattern, developers can construct easily maintainable and testable applications.

Advanced UI design techniques

Custom controls and templates allow developers to create reusable UI elements, which help define the application's visual appearance consistently and flexibly.

Custom controls can be created by extending existing controls. Here's an example:

```
using Microsoft.Maui.Controls;
using Microsoft.Maui.Graphics;

public class CustomButton : Button
{
    public CustomButton()
    {
        BackgroundColor = Color.Blue;
        TextColor = Color.White;
    }
}
```

Control templates allow you to define the appearance of controls. Here's an example in XAML:

```
<ControlTemplate x:Key="CustomButtonTemplate">
    <StackLayout>
        <Label Text="{Binding Text, Source={RelativeSource
        TemplatedParent}}" TextColor="White" BackgroundColor="Blue" />
    </StackLayout>
</ControlTemplate>
<Button ControlTemplate="{StaticResource CustomButtonTemplate}"
Text="Custom Button" />
```

By crafting custom controls and templates like the preceding examples, developers can enhance the usability and appearance of their applications.

Animations and transitions

Animations and transitions enhance the visual appeal and user experience in .NET MAUI. The framework offers various ways to create simple and complex animations.

For **basic animations**, you can use built-in methods to animate properties. Here's an example:

```
await myButton.ScaleTo(1.5, 250, Easing.CubicInOut);
await myButton.ScaleTo(1, 250, Easing.CubicInOut);
```

For **keyframe animations**, you can create more complex animations by using keyframes. Here's an example:

```
var animation = new Animation();
animation.WithConcurrent((f) => myButton.Scale = f, 1, 1.5, Easing.
CubicInOut);
animation.WithConcurrent((f) => myButton.Opacity = f, 1, 0.5, Easing.
CubicInOut);
animation.Commit(this, "MyAnimation", length: 250, easing: Easing.
CubicInOut);
```

For **transitions**, you can animate changes between states. Here's an example:

```
var fadeOut = new Animation(v => myButton.Opacity = v, 1, 0);
var fadeIn = new Animation(v => myButton.Opacity = v, 0, 1);
fadeOut.Commit(this, "FadeOut", length: 500, finished: (v, c) => fadeIn.
Commit(this, "FadeIn", length: 500));
```

Simple and complex animations allow developers to create interactive and visually appealing applications with smooth transitions.

Accessibility features

.NET MAUI provides a range of features to help developers ensure their applications are accessible to all users worldwide.

To improve accessibility, developers can set **accessibility properties** on UI elements. Here's an example:

```
<Button Text="Submit" AutomationProperties.IsInAccessibleTree="true"
AutomationProperties.Name="Submit Button" />
```

For **screen reader support**, ensure the application works smoothly with screen readers. Here's an example:

```
<Label Text="Username" AutomationProperties.HelpText="Enter your username" />
```

To implement **keyboard navigation**, support keyboard shortcuts by adding keyboard accelerators. Here's an example:

```
using Windows.System;
using Microsoft.UI.Xaml.Controls;
using Microsoft.UI.Xaml.Input;

public MainPage()
{
    InitializeComponent();
    AddKeyboardAccelerator();
}

void AddKeyboardAccelerator()
{
    var accelerator = new KeyboardAccelerator { Key = VirtualKey.Enter };
    accelerator.Invoked += OnAcceleratorInvoked;
    this.KeyboardAccelerators.Add(accelerator);
}

private void OnAcceleratorInvoked(KeyboardAccelerator sender,
KeyboardAcceleratorInvokedEventArgs args)
{
    // Handle the Enter key press event
    args.Handled = true;
}
```

By utilizing these accessibility features, developers can ensure that their applications are inclusive and user-friendly and offer a better experience to all users, improving overall engagement.

After refining the UI with advanced design techniques, it's time to ensure the app performs as expected across different platforms. In the next section, we will focus on testing your application to identify and resolve any issues, followed by preparing it for deployment. Rigorous testing and smooth deployment are key steps to delivering a high-quality, cross-platform app. Let's explore the tools and methods for testing and deploying your app effectively.

Testing and deploying the app

To test the .NET MAUI application on various platforms, emulators and simulators for Android, iOS, and other target platforms need to be set up:

- Android Emulator:
    - Open Visual Studio and navigate to **Tools** | **Android** | **Android SDK Manager**
    - Install the required SDKs and system images
    - Configure and launch the Android emulator from **Tools** | **Android** | **Android Device Manager**
- iOS Simulator:
    - Install Xcode on the macOS machine
    - Open Xcode and navigate to **Xcode** | **Preferences** | **Components**
    - Install the required iOS Simulator
    - Launch the iOS Simulator from Xcode
- Windows and macOS:
    - Ensure the respective SDKs are installed
    - Set the target platform in Visual Studio and run the application

Proper operation across different devices and screen sizes is a significant aspect of applications. Emulators and simulators enable developers to assess the application's performance.

Configuring platform-specific settings

Different platforms require specific configurations to ensure optimal performance and compatibility.

For Android, you need to configure `AndroidManifest.xml` for permissions and settings. This can be done using the properties window or directly modifying the file. Here's an example of modifying the file directly:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
package="com.example.myapp">
    <application android:label="MyApp">
        <activity android:name="MainActivity" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
```

```
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

For iOS, configure the `Info.list` file for permissions and settings. Similar to the previous file, this can be modified through the properties window or directly on the file. Here's an example of modifying the file:

```
<dict>
    <key>CFBundleName</key>
    <string>MyApp</string>
    <key>NSCameraUsageDescription</key>
    <string>This app requires access to the camera.</string>
</dict>
```

You configure app settings and permissions within the project properties for Windows and macOS.

Developers configure platform-specific settings to ensure the application meets each platform's guidelines and requirements.

Running the app on Windows, macOS, iOS, and Android

The .NET MAUI application can be operated on various platforms once emulators are set up and configured.

To run the application on Windows, set the Windows project as the startup project and click the **Run** button in Visual Studio to launch the application on Windows. For macOS, set the macOS project as the startup project and click the **Run** button in Visual Studio to launch the application on macOS.

For iOS, set the iOS project as the startup project, select the desired simulator from the drop-down menu, and click the **Run** button in Visual Studio to launch the application on the iOS Simulator.

For Android, set the Android project as the startup project, select the desired emulator from the drop-down menu, and click the **Run** option in Visual Studio to deploy the application in the Android Emulator.

Deploying the app to app stores

Several steps are necessary before deploying the .NET MAUI application to app stores to ensure optimal performance and compliance with app store requirements.

To **Optimize the App**, remove unnecessary resources, and improve performance **Select Optimize the App to remove unnecessary resources and improve performance**. Next, **set the app version** by specifying the version number and build number in the project properties. Here's an example:

```
<PropertyGroup>
    <Version>1.0.0</Version>
    <AssemblyVersion>1.0.0.0</AssemblyVersion>
    <FileVersion>1.0.0.0</FileVersion>
</PropertyGroup>
```

For the **release build**, configure the settings to optimize the app for production. Here's an example of the `.csproj` file with the configuration:

```
<PropertyGroup Condition=" '$(Configuration)|$(Platform)' ==
'Release|AnyCPU' ">
    <Optimize>true</Optimize>
    <DebugType>pdbonly</DebugType>
    <PlatformTarget>AnyCPU</PlatformTarget>
    <DefineConstants>TRACE</DefineConstants>
    <ErrorReport>prompt</ErrorReport>
    <WarningLevel>4</WarningLevel>
</PropertyGroup>
```

By following these steps, the application will be adequately prepared for deployment, ensuring it meets the quality and performance standards required by app stores.

Creating app packages

Creating app packages involves generating the necessary files for deployment to app stores.

For Android, create an APK or AAB file for deployment using the following command in the **Command-Line Interface (CLI)** or a Visual Studio terminal on the root directory for the project:

```
dotnet build -t:SignAndroidPackage -c Release -f net6.0-android
```

For iOS, generate an IPA file for deployment with this command:

```
dotnet build -c Release -f net6.0-ios
```

For Windows and macOS, create the respective app packages using these commands:

```
dotnet publish -c Release -f net6.0-windows
dotnet publish -c Release -f net6.0-macOS
```

Creating these app packages makes the application ready for submission to app stores.

Submitting to app stores

To submit an application to app stores, you must meet each platform's submission guidelines and requirements:

- Google Play Store (Android):
    - Create a developer account on the Google Play Console
    - Upload the APK or AAB file and provide the necessary information (app details, screenshots, etc.)
    - Submit the app for review and publication
- Apple App Store (iOS):
    - Create a developer account on the Apple Developer Program
    - Use Xcode to upload the IPA file to App Store Connect
    - Provide the necessary information (app details, screenshots, etc.)
    - Submit the app for review and publication
- Microsoft Store (Windows):
    - Create a developer account on the Microsoft Partner Center
    - Upload the app package and provide the necessary information (app details, screenshots, etc.)
    - Submit the app for review and publication
- Mac App Store (macOS):
    - Create a developer account on the Apple Developer Program
    - Use Xcode to upload the app package to App Store Connect
    - Provide the necessary information (app details, screenshots, etc.)
    - Submit the app for review and publication

Abiding by the submission process for each app store ensures the application is available and accessible to a broad audience.

Summary

This chapter provided a comprehensive guide to developing cross-platform applications using .NET MAUI, focusing on critical aspects such as setting up the development environment, designing UIs with XAML, handling app logic, and implementing accessibility features. It also covered essential steps for testing and deploying applications across Android, iOS, Windows, and macOS platforms. Developers can streamline creating native apps by leveraging a single code base and targeting multiple platforms, ultimately reducing development time and costs.

In the next chapter, we will explore the use of the MVVM pattern and data binding in .NET MAUI. These are crucial for maintaining a clean separation between the UI and business logic, allowing for scalable, maintainable applications.

7

Implementing MVVM and Data Binding in .NET MAUI

This chapter focuses on the **Model-View-ViewModel** (**MVVM**) design pattern and the role of data binding in building dynamic, maintainable, and interactive applications using .NET MAUI. The MVVM pattern is essential for separating the user interface from business logic, allowing developers to create clean and testable code. In .NET MAUI, data binding simplifies synchronizing the view model with the user interface, reducing manual updates and enabling real-time interaction.

We will explore key concepts such as **one-way** and **two-way data binding**, implementing commands for handling user interactions, and using `ObservableCollection` for managing dynamic data collections. Additionally, we'll cover advanced techniques such as value converters to transform data and commands to manage user input more effectively.

By the end of this chapter, you will have a comprehensive understanding of how MVVM and data binding work together in .NET MAUI to create responsive applications. These skills will form the foundation for crafting high-performance, scalable apps that deliver an optimal user experience across platforms.

Upon completing this chapter, developers will gain a more robust understanding of how MVVM and data binding are used to create maintainable, scalable, and testable applications using .NET MAUI. They will also have gained knowledge and skills for constructing cross-platform applications that fulfill current requirements and are adaptable to future requirements.

This chapter aims to be a comprehensive guide, providing developers with theoretical insights and practical skills. Whether the developer is familiar with or new to .NET MAUI, the concepts and techniques covered here will enhance their ability to build high-quality applications. This journey will aid in mastering MVVM and data binding in .NET MAUI and unlock the full potential of cross-platform development skills.

In this chapter, we will look at the following topics:

- Introduction to MVVM and data binding in .NET MAUI
- Data binding concepts
- Creating a view model and defining bindings
- Basic data binding
- Enhancing user experience

Technical requirements

This chapter requires access to **Visual Studio Community** (`https://visualstudio.microsoft.com/downloads/`) with the following workloads installed:

- ASP.NET and web development
- .NET Multi-platform App UI Development
- .NET desktop development
- Universal Windows Platform Development

The code shown in the examples can be found here: `https://github.com/PacktPublishing/GUI-Programming-with-C-Sharp`.

Introduction to MVVM and data binding in .NET MAUI

Microsoft's multi-platform applications framework is .NET MAUI. Only a single code base is required to craft native applications that work on Windows, Android, macOS, and iOS. Harnessing the capabilities of C# and XAML using .NET MAUI ensures that user interfaces are immersive and responsive.

At the heart of creating practical .NET MAUI applications lies the MVVM pattern. MVVM divides the application into three interconnected components:

- **Model**
- **View**
- **View model**

A significant concept called **data binding** helps define the communication between the view and the view model. It enables the automatic synchronization of properties and commands in the view model with the UI elements in the view. This synchronization enables the construction of dynamic and interactive interfaces with the usage of minimal boilerplate code.

This chapter will explore the complexities of utilizing the MVVM pattern and data binding in .NET MAUI. Beginning with the basics, a comprehensive introduction to MVVM and data binding will be offered. The creation of a simple view model will be discussed, followed by binding it to a view to establish a solid base for more advanced topics.

After that, advanced concepts such as binding modes, converters, commands, and collections will be introduced. Developers can create more sophisticated applications with numerous features by understanding these topics. Binding modes can control the data flow between the view and the view model. Data transformation is enabled during binding through the utilization of converters. Commands provide a way to handle user interactions, and collections allow developers to manage and display lists of data effectively.

To apply theoretical concepts, this section will guide the development of a cross-platform book list application. The application of MVVM and data binding principles for creating a real-world application will also be showcased. Setting up the project, as well as building and viewing models, will also be explored. Views will be designed, and data binding and commands will be implemented. Testing and debugging tactics will also be discussed to ascertain the application's reliability and robustness.

What are MVVM and data binding?

The MVVM pattern is a renowned framework. It is designed to aid in segregating various aspects of development tasks within an application. It is commonly used to develop user interfaces, particularly in frameworks such as Xamarin, WPF, and .NET MAUI. An application is divided into three main components by MVVM.

The **model** represents the application's data and business logic, handling all operations related to data access, manipulation, and business rules. It ensures data integrity and implements validation and business processes that are essential for the application. For instance, the model would manage transactions and accounts and perform financial calculations in a financial application, thus acting as the core of the application's functionality.

The **view** dictates the application's visual presentation, controlling its structure, layout, and appearance. It encompasses all the UI elements, such as buttons, labels, text boxes, and other controls that the user interacts with. The primary responsibility of the view is to present data to the user and handle interactions. For example, in a weather application, the view would display the current weather conditions and forecast information and provide buttons for refreshing data, ensuring an intuitive user experience.

The **view model** mediates between the model and the view, abstracting the data and commands that the view needs. It manages the presentation logic and ensures proper state management, formatting the data from the model in a way that the view can use effectively. The view model also processes user input and commands, ensuring the application responds appropriately. For example, in a task management application, the view model would handle tasks, process user commands to add or update tasks, and interact with the model to ensure the task registry reflects the user's actions.

Comparison with other design patterns (MVC and MVP)

Model-View-Controller (**MVC**) divides an application into three parts: model, view, and controller. The controller handles user input, updates the model, and alters the view accordingly. In MVC, the controller directly manipulates both the view and the model, resulting in a tighter coupling between these components. This can make maintaining and testing the application more challenging. MVC is commonly used in web applications, where the controller processes HTTP requests and responses.

Model-View-Presenter (**MVP**) also divides the application into three components: model, view, and presenter. In this architecture, the presenter acts as a bridge between the model and the view. It handles user input and updates the view, mediating all interactions between the model and the view. This leads to a less tightly integrated design than MVC, though it still requires manual updates to the view. MVP is often used in mobile and desktop applications, providing a better separation of concerns than MVC.

MVVM introduces the view model to separate the view from the model through data binding. This allows automatic synchronization between the view and view model, reducing the need for manual updates. MVVM provides a more flexible and testable architecture than MVC and MVP. It simplifies UI updates and reduces boilerplate code through automatic data binding. MVVM is widely used in modern UI frameworks such as WPF, Xamarin, and .NET MAUI, which are ideal for managing complex data interactions.

Benefits of using MVVM in app development

There are various essential benefits provided by MVVM to promote the creation of applications that are maintainable, testable, and scalable:

- **Separation of concerns**:
    - MVVM enables a defined partition between the user interface (view), the business logic (view model), and the data (model).
    - The code is more organized and readable due to this partition. As such, it is easier to manage and update each component independently. Developers can work on various parts of the application. This will not cause any disruption in each other's work.
    - For example, development teams are able to focus on their areas of expertise, such as user interface design, business logic, and data management, when constructing a large-scale enterprise application. This enhances the overall efficiency of the development processes.
- **Testability**:
    - By decoupling the UI logic (view model) from the UI controls (view), MVVM enhances the ability to perform unit testing on the presentation logic without requiring the actual UI components.
    - This improves the overall testability of the application, as developers can write automated tests for the view model to verify business logic and data manipulation independently.
    - For example, in a healthcare application, developers can test the logic for calculating patient doses in the view model without interacting with the user interface.
- **Scalability**:
    - MVVM promotes a modular structure where each component can be developed, tested, and maintained separately.
    - This modularity is crucial for large applications where different teams or developers may work on different parts of the code base simultaneously. It also allows for easier integration of new features and updates.
    - For example, in an e-commerce platform, new features such as payment gateways or recommendation systems can be added independently of the existing UI and business logic.

After understanding the fundamental principles of the MVVM pattern and data binding in .NET MAUI, it's now time to explore how these concepts can be practically applied to build real-world applications. In the following sections, we will delve into the implementation process, beginning with setting up a development environment to ensure an efficient and smooth workflow. From there, we'll transition to creating a basic .NET MAUI project, laying the groundwork for incorporating MVVM and data binding techniques to manage user interactions and data presentation.

Data binding concepts

Data binding plays one of the most vital roles in MVVM. Owing to data binding, the data between the view and the view model is synchronized automatically. The UI elements in the view are linked to properties in the view model through this concept, which facilitates a smooth flow of data and commands.

In **one-way binding**, the target property, usually in the view, is updated when the source property in the view model changes. This type of binding is beneficial for read-only data displays, where the UI needs to reflect changes in the underlying data without allowing user modifications. A typical example is displaying the current date and time on a label that updates every second.

Two-way binding synchronizes changes between the source property in the view model and the target property in the view. Changes in the view or the view model are automatically reflected in both directions. This type of binding is often used in forms and interactive UI elements, where user input needs to be reflected in the view model, and vice versa. For instance, a textbox for entering user information updates the view model as the user types, and any changes in the view model are immediately reflected in the textbox.

In **one-time binding**, the target property is updated only once when the view is initialized and does not track any subsequent changes in the source property. This can be useful for displaying static data that does not change during the application's life cycle, improving performance. An example would be showing a static welcome message or instructional text on the UI.

Data binding facilitates communication between the view and the view model by establishing a direct connection between the properties and the view model UI elements. In the MVVM framework, this integration ensures that data is synchronized automatically without requiring extensive coding or constant manual updates. When data in the view model changes, it is seamlessly reflected in the view, streamlining the development process and improving efficiency.

Implementation in .NET MAUI

Extensible Application Markup Language (XAML) is a markup language used to define and structure user interfaces. In .NET MAUI, XAML is primarily utilized for data binding, where properties in the view model are connected to controls in the XAML markup. The binding mode (such as one-way or two-way) and any necessary converters or validators for data transformation are specified within the XAML.

Consider a simple .NET MAUI application where the view model contains a property for the user's name. This example is intended to show the elements of the project; you will learn how to run it in the *Creating a view model and defining bindings* section.

View model (UserViewModel.cs)

In this file, we define a `UserViewModel` that implements the `INotifyPropertyChanged` interface, enabling data binding between the UI and the view model in .NET MAUI. The `UserName` property is updated and notifies the view when its value changes, ensuring real-time synchronization between the view model and the view:

```
public class UserViewModel : INotifyPropertyChanged
{
    private string _userName;
    public string UserName
    {
        get => _userName;
        set
        {
            if (_userName != value)
            {
                _userName = value;
                OnPropertyChanged();
            }
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;

    protected void OnPropertyChanged([CallerMemberName] string
                                     propertyName = null)
```

```
    {
        PropertyChanged?.Invoke(this, new
                                PropertyChangedEventArgs(propertyName));
    }
}
```

View (MainPage.xaml)

In this file, we define a simple user interface in .NET MAUI, where the user's input is bound to a `UserName` property in `UserViewModel`. The `Entry` control uses two-way binding to synchronize the input with the view model, while `Label` displays the value using one-way binding. Make sure you replace `YourNamespace` with the one you are using if you are using a different name for the namespace:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:YourNamespace"
             x:Class="YourNamespace.MainPage">

    <ContentPage.BindingContext>
        <local:UserViewModel />
    </ContentPage.BindingContext>

    <StackLayout>
        <Label Text="Enter your name:" />
        <Entry Text="{Binding UserName, Mode=TwoWay}" />
        <Label Text="{Binding UserName, Mode=OneWay}" />
    </StackLayout>
</ContentPage>
```

In this example, **two-way binding** is used to bind the `Entry` control's `Text` property to the `UserName` property in `UserViewModel`. When the user types into the `Entry` control, the `UserName` property in the view model is automatically updated, and any changes to the `UserName` property in the view model are reflected in the `Entry` control. Additionally, a `Label` control is bound to the `UserName` property using **one-way binding**, meaning the label will display the current value of `UserName` and update whenever the value changes, but it won't affect the view model directly.

This approach showcases how XAML and data binding in .NET MAUI simplify the interaction between the UI and the data layer, enabling automatic synchronization and reducing the need for manual updates.

Now that we have explored the fundamental concepts of data binding and its implementation within .NET MAUI, let's shift our focus to creating the view model and binding it to the view. Setting up the view model ensures that data and user interactions flow seamlessly between the user interface and the business logic. By following these next steps, we will establish the foundation for a well-structured, maintainable application using MVVM principles.

Creating a view model and defining bindings

Before starting, it is fundamental to configure an **integrated development environment (IDE)** to create cross-platform applications using .NET MAUI. The ideal choice for this is Visual Studio. These are the steps for setting up the development environment:

1. **Install Visual Studio**: The latest Visual Studio version is on Microsoft's official website. Download and install it from there. During installation, make sure to select the workloads for .NET MAUI. The workloads include components for building and running .NET MAUI applications on various platforms.
2. **Install the .NET SDK**: The .NET SDK should be automatically installed by Visual Studio. If not, download it from the .NET download page. The .NET SDK contains the required runtime and libraries to build and run .NET applications.
3. **Enable Developer mode**: On Windows, go to **Settings**. Then, enable **Developer mode**. This step is vital for deploying and testing apps on both local and remote devices. Go to **Settings** first. Then, go to **Update & Security** and then navigate to **For developers**. Finally, select **Developer mode**.
4. **Set up emulators**: If you are developing for Android or iOS, install the corresponding emulator via Visual Studio's installer. This allows developers to test their applications on different devices and screen sizes. In Visual Studio, go to **Tools**. Then, go to **Android Emulator Manager** and **Tools**, and then to **iOS Simulator** to set up and manage emulators.
5. **Create a new project**: Open Visual Studio. Then, select **Create a new project** and choose the **.NET MAUI App** template to start a new project. This template provides the basic structure and files needed for a .NET MAUI application.

The correct setup of the environment ensures that developers can develop, test, and debug the application smoothly across various platforms. If the setup is appropriate, time can be saved, and issues from misconfigured environments can be prevented.

Creating a simple .NET MAUI app

After configuring the IDE, a basic .NET MAUI app needs to be constructed. Here is an example that can be used as a guide for the basic setup and framework of a .NET MAUI project:

1. **New project setup**:
    a. Open Visual Studio, and select **Create a new project**.
    b. Choose the **.NET MAUI App** template and click **Next**.
    c. Configure the project by providing a name, location, and solution name. Click **Create**. This initializes a new .NET MAUI project with the necessary files and folders.
2. **Project structure**:
    a. **Platforms folder**: This folder contains platform-specific code for Android, iOS, macOS, and Windows. Each subfolder includes files and resources tailored for the respective platform.
    b. **Resources folder**: This folder stores application resources such as images, fonts, and styles. These resources are shared across all platforms and can be referenced in XAML and C# code.
    c. **MainPage.xaml**: The main user interface file using XAML. It defines the layout and elements of the initial screen presented to the user.
    d. **App.xaml and App.xaml.cs**: Define application-level resources and startup logic. `App.xaml` contains shared resources such as styles and templates, while `App.xaml.cs` includes the application's initialization code.
3. **Running the app**:
    a. The target platform (Android, iOS, macOS, or Windows) needs to be chosen from the run configurations. This can be done using the drop-down menu in the Visual Studio toolbar.
    b. To build and deploy the app, click the **Run** button. Visual Studio will compile the code and then deploy it to the selected emulator or device before launching the application.

This simple setup provides a base for creating more intricate applications. The following steps involve adding functionality and implementing MVVM.

Creating the view model

In the MVVM pattern, the view (user interface) and the model (business logic) must be connected, achieved through the view model. The following steps outline how to create a view model and bind it to the view in .NET MAUI.

Define a ViewModel class

Add a new class file, `MainViewModel.cs`, to the project. This file will contain the view model logic for the application's main page, as shown here:

```
using System.ComponentModel;

public class MainViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    protected virtual void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new
        PropertyChangedEventArgs(propertyName));
    }
}
```

In the first part, we define a new `ViewModel` class named `MainViewModel`. This class implements the `INotifyPropertyChanged` interface, which is essential for enabling data binding in .NET MAUI. The `PropertyChanged` event is raised whenever a property in the view model changes, ensuring that the view is updated automatically. The `OnPropertyChanged` method is responsible for invoking this event and notifying the UI of any changes to bound properties.

With the basic structure of the view model in place, the next step is to add the necessary properties that will be bound to UI elements in the XAML file. This allows the data and commands in the view model to dynamically interact with the user interface.

Implement properties

Next, properties are added to the view model that represent the data and commands used by the view. These properties will be bound to UI elements in the XAML file. Here's an example of a `Title` property:

```
private string _title;

public string Title
{
    get => _title;
    set
    {
        if (_title != value)
        {
            _title = value;
            OnPropertyChanged(nameof(Title));
        }
    }
}
```

In this code, we add a `Title` property to the view model, which will be used by the view to display data. The property is backed by a private `field _title`, and in the setter, we check whether the new value is different from the current value. If it is, we update the field and call `OnPropertyChanged` to notify the view of the change. This mechanism ensures that the user interface stays in sync with the view model.

Now that the properties have been implemented, the next step is to initialize them with default values in the view model's constructor. This ensures that the UI has relevant data to display when it first loads. Following this, we will explore how to bind these properties to the UI elements in XAML to create a fully functional, dynamic interface.

Initialize data

In the view model's constructor, initialize any default values or data the view might need when it is first displayed. For instance, the following code sets a default title:

```
public MainViewModel()
{
    Title = "Hello, .NET MAUI!";
}
```

By defining the view model and its properties, you create a structure to which the view can bind, allowing dynamic data updates to the UI.

Defining the view

The view in .NET MAUI is typically defined using XAML. Here's how to create the view and bind it to the view model.

Open `MainPage.xaml`, which contains the UI definition for the application's main page. Define the layout and elements using XAML. For example, create a label that binds to the `Title` property of the view model:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="YourNamespace.MainPage">

    <StackLayout>
        <Label Text="{Binding Title}"
               VerticalOptions="CenterAndExpand"
               HorizontalOptions="CenterAndExpand" />
    </StackLayout>
</ContentPage>
```

In the first part, we define the view using XAML in `MainPage.xaml`, which describes the user interface elements and layout. The `Label` element is bound to the `Title` property of the view model using the `{Binding Title}` syntax. This binding allows the text of the label to automatically display the value of the `Title` property from the view model, ensuring that the UI reflects any updates to the data.

Once the view is defined, the next step is to connect it to the view model. This is achieved by setting `BindingContext` in the `MainPage.xaml.cs` file to an instance of `MainViewModel`, enabling the UI elements to interact with the view model's properties and ensuring a dynamic, data-driven interface.

Binding to the view model

In the code-behind file, `MainPage.xaml.cs`, set `BindingContext` to an instance of `MainViewModel`. This connects the view to the view model and enables data binding:

```
public partial class MainPage : ContentPage
{
```

```
    public MainPage()
    {
        InitializeComponent();
        BindingContext = new MainViewModel();
    }
}
```

By binding the view to the view model, the UI automatically updates when the view model's properties change. This ensures a dynamic and interactive user interface that reacts to real-time user input and data changes.

With the view and view model successfully connected, we now have a functional application that demonstrates the basic principles of MVVM and data binding. However, to unlock the full potential of MVVM and enhance the user experience, it is essential to dive deeper into the core data binding techniques.

In the next section, we will explore primary data binding approaches such as one-way and two-way binding, as well as introduce advanced concepts such as converters, commands, and handling collections. These features will provide more control over data flow, improve interactivity, and enable the development of more sophisticated and dynamic applications.

Basic data binding

The core concept of MVVM is **data binding**, which enables automatic synchronization between the user interface and the underlying data. One-way binding connects a UI element to a property in the view model. The user interface reflects any updates made in the view model, but the UI itself cannot modify the data. This technique is useful for displaying data that doesn't require user input or modifications.

Two-way binding ensures that changes in the UI are reflected in the `ViewModel` property, enabling user input. This is essential for interactive elements such as forms where user input needs to be captured and processed.

Standard UI controls such as `TextBox`, `Label`, and `Button` can be bound to `ViewModel` properties and commands. This allows the UI to reflect the state of the view model dynamically and allows user interactions to trigger actions in the view model.

The `INotifyPropertyChanged` interface ensures that the view is updated when a property in the view model changes. This mechanism enables data binding, informing the view when the underlying data has been modified.

These primary data binding techniques are essential to the MVVM pattern, allowing for a dynamic and responsive user interface. A solid understanding of these concepts is necessary for building complex .NET MAUI applications using MVVM.

The following section will explore **advanced data binding techniques**, including binding modes, converters, commands, and collections. These advanced techniques will enhance your applications' functionality and user experience, providing more control over data manipulation and user interaction.

When and how to use each mode

Choosing the appropriate binding mode is crucial for ensuring that your application behaves as expected and efficiently handles data updates. Different modes are suited to different use cases, depending on whether the data should only be displayed, updated by the user, or remain static throughout the application's life cycle. By selecting the right binding mode, you can optimize the data flow between the view and the view model, reduce unnecessary updates, and ensure that the user interface reacts appropriately to changes in the underlying data. Here are the most commonly used binding modes and their ideal scenarios:

- `OneWay`: Use for static or read-only data that the user will not change
- `TwoWay`: Use for interactive elements where user input must be captured and reflected in the view model
- `OneWayToSource`: Use for scenarios where user actions update the view model without feedback to the UI
- `OneTime`: This displays data that does not change after initial binding, such as application settings or constants

Understanding these modes can ensure that the application behaves correctly and efficiently, minimizing unnecessary data flow and enhancing performance.

Converters

Value converters in .NET MAUI are essential for transforming data between the source and target in data bindings. They are instrumental when the source property's data type or format does not match the expectations of the target control.

A value converter in .NET MAUI implements the `IValueConverter` interface, which defines two methods: `Convert`, which transforms the source data before it is passed to the target, and `ConvertBack`, which transforms the target data back to the source format. The `ConvertBack` method is often not implemented if two-way binding is not required.

To create a value converter, define a new class that implements the `IValueConverter` interface. For example, `BoolToVisibilityConverter` converts a Boolean value to visibility states:

```
using System;
using System.Globalization;
using Microsoft.Maui.Controls;

public class BoolToVisibilityConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter,
                          CultureInfo culture)
    {
        if (value is bool boolValue)
        {
            return boolValue ? Visibility.Visible : Visibility.Collapsed;
        }
        return Visibility.Collapsed;
    }

    public object ConvertBack(object value, Type targetType,
                              object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }
}
```

Next, register the converter in the XAML file by adding it to the `Resources` section. This allows the converter to be reused throughout the page:

```
<ContentPage.Resources>
    <local:BoolToVisibilityConverter x:Key="BoolToVisibilityConverter" />
</ContentPage.Resources>
```

Finally, use the converter in a binding expression within the XAML file to transform data between the view model and the UI:

```
<Label Text="Visible or Not"
       IsVisible="{Binding IsItemVisible, Converter={StaticResource
BoolToVisibilityConverter}}" />
```

By using value converters, you can customize the presentation of data, ensuring that the UI reflects the underlying data model accurately and intuitively. This technique is beneficial when the data format or type differs between the view model and the view.

Commands

User interactions, such as button clicks in .NET MAUI, are managed using commands, adhering to the MVVM pattern. Commands provide a way to separate UI interaction logic from the business logic, enhancing maintainability. The `ICommand` interface defines the structure for implementing commands, ensuring a consistent approach.

The `ICommand` interface includes two main methods: `Execute`, which defines what happens when the command is invoked, and `CanExecute`, which determines whether the command can be executed. This method turns UI elements on or off based on the application's state.

To implement commands in the view model, start by defining a `command` property. You can use the built-in `Command` class or create a custom command implementation. For example, in this view model, `ShowMessageCommand` is defined to handle button clicks:

```
using System.Windows.Input;
using Microsoft.Maui.Controls;

public class MainViewModel
{
    public ICommand ShowMessageCommand { get; }

    public MainViewModel()
    {
        ShowMessageCommand = new Command(ShowMessage);
    }

    private void ShowMessage()
    {
        // Logic to execute when the command is invoked
        Application.Current.MainPage.DisplayAlert("Message", "Hello, .NET
                                                   MAUI!", "OK");
    }
}
```

Next, bind this command to a button in the XAML file, allowing the user to trigger the command with a button click:

```
<Button Text="Show Message"
        Command="{Binding ShowMessageCommand}" />
```

Commands can be used for various UI actions beyond simple button clicks. For example, they can handle gestures such as swipes or taps, or respond to list item selections. By binding UI actions to commands, you keep your UI logic separate from business logic, making your application more maintainable and testable. This structure ensures a cleaner separation of concerns and a more modular design.

Collections

Handling collections of data in .NET MAUI often requires dynamic lists that are displayed and updated in the user interface. The `ObservableCollection<T>` class helps manage and bind such dynamic data lists, especially in MVVM architectures. `ObservableCollection` automatically alerts when items are added, removed, or changed, making it ideal for data binding in UIs.

To start, define an `ObservableCollection` property in the view model. This allows dynamic data to be maintained and updated:

```
using System.Collections.ObjectModel;

public class MainViewModel
{
    public ObservableCollection<Book> Books { get; }

    public MainViewModel()
    {
        Books = new ObservableCollection<Book>
        {
            new Book { Title = "Book 1", Author = "Author 1" },
            new Book { Title = "Book 2", Author = "Author 2" }
        };
    }
}
```

Next, `ObservableCollection` is bound to a list control in the XAML file, such as `ListView` or `CollectionView`. The data from `ObservableCollection` will be automatically reflected in the UI, and any changes to the collection will trigger updates in the bound control:

```
<CollectionView ItemsSource="{Binding Books}">
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <StackLayout>
                <Label Text="{Binding Title}" />
                <Label Text="{Binding Author}" />
            </StackLayout>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
```

This approach ensures that the UI stays in sync with any changes to the collection, providing a dynamic and responsive user interface for displaying lists of data. `ObservableCollection` makes adding or removing items from the collection easy, and the changes will be automatically reflected in the UI without requiring manual updates.

Binding collections to UI controls enables dynamic and responsive user interfaces. Here are some typical controls:

- `ListView`: A list of items with built-in support for item selection and scrolling is showcased
- `CollectionView`: A more flexible and performant control that displays data collections supporting layouts such as grids and horizontal lists
- `BindableLayout`: Allows binding collections to any layout control, enabling custom layouts and item templates

Once you've implemented converters, commands, and collections to manage data flow and user interactions effectively, it's time to focus on enhancing the overall user experience. An excellent user interface isn't just functional but also engaging, intuitive, and visually appealing.

In the next section, we'll explore ways to improve user experience through design consistency, responsive layouts, and subtle animations. Additionally, we'll cover best practices for maintaining a clean and scalable code base, ensuring that your application remains efficient and easy to manage as it grows in complexity. By following these guidelines, you'll be able to create a polished, user-friendly application that performs seamlessly across multiple platforms.

Enhancing user experience

An excellent user experience goes beyond just functional code. Here are some tips and best practices to improve an app's user interface when using MVVM and data binding:

- **Consistent design**: Ensure the application uses a consistent design language across all platforms. Implement styles and templates to standardize UI elements, creating a cohesive and polished look.
- **Responsive layouts**: Support for different screen sizes and orientations is essential, as users access applications on various devices. Utilize adaptive controls such as `Grid` and `FlexLayout` to create flexible and responsive designs that seamlessly adjust to different screen dimensions.
- **Animations and transitions**: Incorporating animations and transitions can provide visual feedback and enhance user engagement. .NET MAUI offers built-in support for animations, which can be triggered through data binding to improve the overall user experience.

Here are the best practices for maintainable and scalable code:

- **Separation of concerns**: Adhering to the MVVM pattern ensures the separation of the user interface from the business logic. Keeping responsibilities distinct leads to more maintainable and testable code.
- **Reusable components**: Avoid code duplication by crafting reusable components and controls. Use user and custom controls for commonly used UI elements, allowing you to streamline development and maintain consistency.
- **Error handling**: Implement robust error handling and validation within the view model. Provide clear feedback to the user when errors occur or input is invalid, ensuring a smoother and more user-friendly experience.
- **Performance optimization**: Ensure optimal performance by optimizing data binding and UI updates. Use asynchronous commands and data loading techniques to prevent the UI from becoming unresponsive due to long-running tasks.

Following these tips and best practices, developers can create well-functioning applications that deliver an immersive user experience.

This section explored advanced data binding techniques in .NET MAUI, including binding modes, converters, commands, and collections. These techniques enable the development of dynamic, interactive, and maintainable applications. Mastering MVVM and data binding allows developers to build scalable applications with a seamless user experience.

Summary

This chapter provided an in-depth exploration of the MVVM pattern and data binding in .NET MAUI, focusing on how these concepts enable the creation of dynamic, interactive, and maintainable applications. We began by discussing the fundamentals of MVVM and its components (model, view, and view model) and then delved into basic and advanced data binding techniques, including one-way and two-way binding, value converters, commands, and collections such as `ObservableCollection`. These concepts allow seamless synchronization between the user interface and the underlying data, reducing manual updates and ensuring a responsive user experience.

Additionally, we covered essential best practices for crafting well-functioning applications, such as maintaining a consistent design, optimizing performance, and adhering to the separation of concerns. By mastering these techniques, developers can create scalable, maintainable, high-performance applications in .NET MAUI.

In the next chapter, we will shift focus to WinUI 3 and Visual Studio, where we will explore how to build modern desktop applications using the WinUI 3 framework. You will learn how to set up your development environment, work with WinUI controls, and understand how this powerful UI framework integrates with Visual Studio for Windows development.

8

Getting Started with WinUI 3 and Visual Studio

In this chapter, we will begin by exploring the fundamental concepts behind WinUI 3, its architecture, and how it interacts with the Windows operating system to create modern and responsive applications. We will then guide you through setting up your development environment with Visual Studio and configuring it for WinUI 3 projects. By the end of this chapter, you will have a comprehensive understanding of how to create and manage WinUI 3 applications, from basic UI design to more advanced techniques such as integrating Windows APIs. To achieve this, we will cover the following topics:

- Understanding WinUI 3 and its benefits
- Real-world applications and use cases
- Setting up the environment
- Installing and configuring WinUI 3
- Creating a new WinUI 3 project
- Building a WinUI 3 app
- Data binding and the MVVM pattern in WinUI 3

Following the structure and content in this chapter will equip you with the necessary skills to build efficient, visually appealing, high-performance Windows applications using WinUI 3.

Technical requirements

This chapter requires access to **Visual Studio Community** (`https://visualstudio.microsoft.com/downloads/`) with the following workloads installed:

- ASP.NET and web development
- .NET Multi-platform App UI Development
- .NET desktop development
- Universal Windows Platform Development

The code shown in the examples can be found here: `https://github.com/PacktPublishing/GUI-Programming-with-C-Sharp`.

Understanding WinUI 3 and its benefits

Windows UI Library 3 (**WinUI 3**) is a cutting-edge UI framework tailored exclusively for Windows applications. An extensive toolkit and controls equip developers to create user interfaces that are not only modern but also interactive and visually pleasing. WinUI 3 is focused on offering a cohesive and flexible framework suitable for both desktop and **Universal Windows Platform** (**UWP**) applications. It aims to empower developers to construct efficient applications that adhere to the latest design standards set by Microsoft.

WinUI 3 is not only focused on visual appeal. It is designed to offer a solid groundwork that leverages the native functionalities of the Windows operating system. Developers can, therefore, fully empower Windows' full capabilities. This results in the creation of applications that are not only faster and more efficient but also more integrated with the underlying OS features.

To understand the significance of WinUI 3, it's essential to look at its evolution. The inception of UWP marked the beginning of the journey. It sought to establish a standardized platform for application creation that could be deployed across all Windows 10 devices, including desktops, tablets, phones, and IoT devices. While revolutionary in its scope, UWP encountered challenges regarding adaptability and performance in desktop applications.

An evolution of UWP was introduced as WinUI 2. It was built on the same foundation as UWP but offered extra controls and features unavailable in the base UWP framework. In terms of design and usability, WinUI 2 brought many enhancements. It also aligned more closely with the **Fluent design system**. However, the issue remained that it was still integrated with the UWP platform and, therefore, its inherent limitations.

The UI framework of WinUI 3 is disassociated from its underlying platform. Thus, it represents a major advancement, as WinUI 3 can be utilized with both UWP and Win32 applications. As such, a versatile solution is provided for various types of applications. In this manner, the limitations identified in earlier versions are addressed, and WinUI 3 provides a more inclusive and future-ready approach to the development of Windows applications.

One of the standout features of WinUI 3 is its native performance. In contrast to other frameworks dependent on additional layers or abstraction, WinUI 3 is designed for direct integration with the Windows operating system. Therefore, applications crafted with WinUI 3 can harness the full potential of underlying hardware and system resources, as ascertained by the aforementioned direct approach. This results in faster load times, smoother animations, and enhanced responsiveness for users.

Additionally, WinUI 3 provides extensive system integration, enabling applications to take advantage of native Windows features. These features include touch and pen input, high DPI support, and advanced graphics capabilities. Efficient application functionality and smooth integration within the Windows environment are ascertained, thus delivering a unified and cohesive user experience.

The foundational framework of WinUI 3 is built around the principles of the Fluent design system. It directs the framework's visual and interactive features. Fluent emphasizes five fundamental principles, as follows:

- **Light**: Fluent uses light and shadow to create a sense of hierarchy and focus. Elements are highlighted or subdued based on their importance and interaction states, naturally guiding users' attention.
- **Depth**: Layers and depth create a three-dimensional feel, making interfaces more engaging and easier to navigate.
- **Motion**: The thoughtful use of motion and animations can provide context and feedback to users, making interactions more understandable and enjoyable.
- **Material**: Fluent employs a range of materials and textures to create a tactile and immersive experience. These can include transparency, blur effects, and surface reflections.
- **Scale**: Fluent's responsiveness and flexibility greatly benefit developers. This is because it ensures that interfaces look good and function smoothly across devices with various screen sizes and orientations.

The synergy of these principles creates interfaces. Aesthetic attractiveness and ease of use are both prioritized in this case. By adhering to these principles, WinUI 3 allows developers to create applications that look modern and polished and provide a superior user experience.

WinUI 3 offers an array of UI controls and components. It consists of essential elements such as buttons and textboxes and more intricate components such as data grids and media players. Developers can personalize the appearance and behavior of the controls to suit their applications. This is because the controls are designed to be personalized.

The diversity of controls in WinUI 3 empowers developers. This aids in the creation of sophisticated and feature-laden applications, reducing dependency on third-party libraries or custom implementations. Hence, the development process speeds up and ascertains consistency and reliability throughout various parts of the application.

A critical challenge in software development is ensuring the applications' ongoing compatibility with future versions of the platform they are developed on. WinUI 3 addresses this through the provision of a framework built for forward compatibility. This means that applications developed with WinUI 3 will be able to seamlessly transition to future iterations of Windows, reducing the necessity for extensive revisions or modifications.

Furthermore, WinUI 3 is constantly being developed. Microsoft is dedicated to adding new features and improvements. This continual development guarantees that WinUI 3 stays at the forefront of UI technology and empowers developers with the required tools for the creation of contemporary and future-proof applications.

WinUI 3 versus WPF – key differences and advantages

Various frameworks exist for creating desktop applications. **Windows Presentation Foundation (WPF)** is among the most adopted choices. It has a wide range of features and a huge community of developers. However, in comparison to WinUI 3, WPF does have some limitations:

- **Performance**: WinUI 3 offers superior performance due to its native integration with the Windows operating system. Despite being powerful, WPF is reliant on the .NET Framework. This can introduce additional overhead.
- **Modern design**: WinUI 3 incorporates the Fluent design system, which is more modern and visually appealing compared to the design principles used in WPF.
- **Future-proofing**: WinUI 3 was crafted with future compatibility in mind. This ensured that applications built with it would continue to function appropriately on future versions of Windows. WPF, an older framework, may not receive the same level of support and updates in the future.

WinUI 3 versus UWP

UWP was designed to provide a cohesive platform for creating applications that work across all Windows 10 devices. Despite being a step forward, UWP had issues addressed by WinUI 3:

- **Flexibility**: WinUI 3 separates the UI framework and the underlying platform. This allows WinUI 3 to be utilized with both UWP and Win32 applications. As such, it offers more flexibility than UWP, which is bound to its platform.
- **Performance**: WinUI 3 offers improved performance through native integration with the Windows operating system, while UWP applications may experience some performance overhead due to their abstraction layers.
- **Control set**: WinUI 3 provides a richer set of controls and components than UWP, allowing developers to build more feature-rich applications.

WinUI 3 versus WinForms

There are many frameworks for creating Windows applications. One of the oldest of these frameworks is **Windows Forms (WinForms)**. It is often praised for its simplicity and straightforward approach, as well as its ease of use. However, it is recognized to have areas where it does not match up to WinUI 3.

- **Design and aesthetics**: WinUI 3's Fluent design system provides a more modern and visually attractive user interface, which is considered much better than the dated look of WinForms applications.
- **Performance**: Enhanced performance is achieved due to WinUI 3's native integration with Windows. Again, this is better than WinForms, which relies on the .NET Framework.
- **Feature set**: WinUI 3 provides an extensive and contemporary set of controls and components, making constructing sophisticated applications easier.

Real-world applications and use cases

WinUI 3 is highly versatile, making it suitable for building various applications, from essential utilities to complex enterprise software. For example, productivity tools such as text editors, spreadsheet software, and project management systems can utilize WinUI 3's robust performance and modern UI components. Its advanced graphics capabilities and responsive design are ideal for multimedia applications such as media players and photo editing software. Additionally, WinUI 3's rich feature set and future-proofing are especially beneficial for business and enterprise software, enhancing **customer relationship management (CRM)** systems, accounting platforms, and other business-critical applications.

WinUI 3 offers enhanced performance and modern UI design, making it an excellent choice for developing healthcare applications such as medical record systems and patient management software, ultimately improving the user experience for healthcare professionals. In the finance sector, WinUI 3's secure and efficient framework is ideal for banking and financial applications, enabling the creation of robust and reliable software for managing financial transactions and data. WinUI 3 can be leveraged in education to build e-learning platforms and other educational tools that foster engaging and interactive environments for students and educators.

Case study 1 – healthcare management system

Implementing WinUI 3 in a healthcare management system can deliver a streamlined experience to doctors and nurses. This is due to the easy retrieval of patient records, efficient appointment management, and accessible communication with other healthcare professionals. The interface will benefit from the Fluent design system's intuitive layout, and the enhanced performance of WinUI 3 will ensure that the large amounts of data do not cause issues with functionality.

Case study 2 – financial trading platform

A financial trading platform can use WinUI 3 to offer traders real-time data updates, advanced chartering capabilities, and a responsive user interface. WinUI 3's powerful performance ensures fast and effective trade execution, and its modern UI components contribute to a professional and polished visual presentation.

Case study 3 – educational e-learning application

An e-learning application developed with WinUI 3 can enhance students' engagement. Features such as multimedia content, quizzes, and progress tracking can do this. The Fluent design system aids in the construction of a user-friendly and visually attractive interface. The native integration with Windows ensures seamless performance across various devices.

Setting up the environment

Meeting the suggested hardware and software requirements is essential. It aids in giving an optimized development workflow with WinUI 3. These specifications maximize performance and allocate necessary resources for development and testing tasks.

These are the hardware requirements:

- **Processor**: WinUI 3 development requires a multi-core processor with at least 2 GHz clock speed. The recommended processor is Intel i5 or higher.

- **RAM**: A minimum of 8 GB of RAM is recommended. However, 16 GB or more is recommended to handle larger projects and run additional applications simultaneously without performance degradation.
- **Storage**: You need at least 20 GB of free disk space to install Visual Studio and related components. An SSD is preferred for faster read/write speeds.
- **Graphics**: A DirectX 11 compatible graphics card is recommended to leverage hardware acceleration features in WinUI 3.

Here are the software requirements:

- **Operating system**: WinUI 3 development requires Windows 10 version 1903 (build 18362) or later. Windows 11 is also compatible. Keep operating systems updated with the latest patches.
- **.NET Framework**: Developers need to use at least .NET 6 for development with WinUI 3.
- **Visual Studio**: Opting for Visual Studio 2022 or later is advisable for WinUI 3 development, given its extensive support and bundled tools specialized for Windows application development.

The following software is also required:

- **Windows SDK**: WinUI 3 development requires the latest version of the Windows SDK, which comprises the required headers, libraries, and tools to construct Windows applications.

Compatible operating systems and versions

WinUI 3 is crafted to function seamlessly on the latest Windows releases, mainly Windows 10 and Windows 11. This section provides an in-depth exploration of their compatibility and the benefits associated with utilizing these operating systems:

Windows 10:

- **Version**: Compatibility of WinUI 3 extends to Windows 10 1903 (build 18362) and later versions. Availability of the latest features and benefits from enhanced security updates is therefore ensured.
- **Benefits**: Windows 10 provides a stable and widely adopted platform alongside extensive documentation and community support. This platform also comprises powerful security features and broad hardware compatibility.

Windows 11:

- **Version**: WinUI 3 is entirely compatible with Windows 11. This brings extra enhancements and a contemporary interface.
- **Benefits**: Windows 11 is designed to deliver enhanced system performance, modernize the user interface, and better integrate with contemporary hardware. Some of the new, noteworthy additions are a new **Start** menu, snap layouts for multi-tasking, and enhanced virtual desktop functionality.

Visual Studio workloads needed for WinUI 3:

- **.NET desktop development**: This workload includes the necessary tools for building Windows applications. NET.
- **Universal Windows Platform development**: This workload includes the Windows SDK and tools for building UWP applications.
- **Desktop development with C++ (optional)**: If the project requires C++ development, select this workload as well.

Visual Studio individual components needed:

- **Windows 10 SDK (latest)**: Required for building and running WinUI 3 applications.
- **.NET 6 Runtime**: This is required for running applications built with the latest .NET framework.

Once you've ensured that the appropriate hardware and software requirements are met and Visual Studio is correctly configured with the necessary workloads and components for WinUI 3, you are ready to begin the actual development process. The next crucial step is to install and configure the specific WinUI 3 project templates in Visual Studio, which will streamline the setup and provide a solid foundation for building your Windows applications.

With the environment fully prepared, we can now move on to installing these templates and verifying their installation, followed by creating your first WinUI 3 project to understand its structure and settings.

Installing and configuring WinUI 3

Proper project templates must be installed in Visual Studio to begin developing with WinUI 3. These templates offer a preconfigured setup, simplifying the procedure of crafting a new application with WinUI 3.

Here's how to install the necessary templates and ensure your environment is fully prepared:

- **Launch the Installer**: Open the Visual Studio Installer from either the **Start** menu or the search bar.
- **Modify installation**: Locate the installed version of Visual Studio in the Visual Studio Installer and click on the **Modify** button.
- **Add workloads**: Select the **Universal Windows Platform development** workload. This includes the necessary components for WinUI 3 development.

Install WinUI 3 templates:

1. **Launch Visual Studio**: After modifying the installation, open Visual Studio.
2. **Open the Extensions menu**: Go to **Extensions** > **Manage Extensions**.
3. **Search for WinUI 3**: In the **Online** tab, search for `WinUI 3 Project Templates`.
4. **Install templates**: Find the WinUI 3 project templates and click **Download**. Follow the prompts to install the templates.
5. A system rebook may be necessary to complete the installation.

Verify installation:

1. **Create a new project**: Go to the **File** dropdown. Now, navigate to the **New** menu. Locate and select **Project** to create a new project.
2. **Search for WinUI 3**: To ensure the correct installation of the templates, search for `WinUI 3` in the new project dialog.

Once the necessary templates are installed and verified, you're ready to move forward with creating your first WinUI 3 project. This involves setting up the project within Visual Studio, where you'll configure key details such as the project name, location, and solution settings.

Creating a new WinUI 3 project

Setting up a new project in Visual Studio involves numerous steps, especially for WinUI 3 development. A detailed guide is outlined here:

1. Using either the **Start** menu or the desktop, launch Visual Studio.
2. Click **Create a new project** on the start screen.

3. Search for WinUI 3. In the project template search box, type `WinUI 3` to filter the available templates.
4. Select the template. Select **Blank App, Packaged (WinUI 3 in Desktop)** for a basic template to get started. Click **Next**.
5. Type in a name for the project (e.g., `MyFirstWinUI3App`).
6. Specify the location where the project should be saved.
7. A name for the solution should be specified. This could be the same as the project name.
8. Make sure that .NET 6 or the most recent version of .NET is selected.
9. Click **Create**.

Understanding project templates and initial settings

When a new WinUI 3 project is created, the selected template includes predefined settings and files that provide a foundation for the application. Understanding these initial settings and templates is crucial for efficient development:

Project templates:

- **Blank App (Packaged)**: This minimal template includes the essential files and settings for a WinUI 3 application. It is ideal for starting simple projects or learning the basics of WinUI 3.
- **Navigation Pane (Packaged)**: This template has a basic navigation structure, including a navigation view control. It is useful for applications with multiple pages or sections.
- **Code-Behind** or **MVVM**: Developers have to select between code-behind (event handling in the code-behind file) or MVVM patterns, depending on their architectural preference.

Initial settings:

- **Target Version**: Specifies the minimum Windows version required to run the application.
- **Configurations**: This includes debugging and releasing configurations for building and deploying the application.
- **References**: The project references essential libraries and packages, including WinUI and .NET.

Understanding these templates and settings helps developers make informed decisions when starting a new project and ensures that they effectively leverage the provided structure.

Understanding the WinUI 3 project structure

Logical organization of files and a clear separation of concerns is the default project structure in a WinUI 3 project. Here is an outline of the typical project structure:

- `MyFirstWinUI3App` (project):
    - `Assets` (folder): Contains image assets and other resources
    - `Strings` (folder): Stores localization resources
    - `App.xaml`: Defines application-level resources and settings
    - `App.xaml.cs`: Contains the code-behind application initialization
    - `MainWindow.xaml`: The main window's XAML layout
    - `MainWindow.xaml.cs`: Code-behind for the main window

Understanding the responsibilities and objectives of each file and folder in the project structure is essential for effective development:

- The `Assets` folder: Stores images, icons, and other visual assets used in the application.
- `Assets/logo.png`: Example image asset.
- `App. xaml`: Defines application-level resources, styles, and templates. Contains the `<Application>` element, which serves as the application's root.
- `App.xaml.cs`: This is the code-behind file for `App.xaml`. It initializes the application, handles startup events, and sets the main window.
- `MainWindow.xaml`: This file defines the layout and structure of the main window using XAML and contains UI elements such as buttons, text blocks, and containers.
- `MainWindow.xaml.cs`: This is the code-behind file for `MainWindow.xaml`. It handles events, manages the UI logic, and interacts with the XAML-defined elements.

Choosing between packaged and unpackaged app types

A choice can be made between packaged and unpackaged app types when developing an application with WinUI 3. Each type has distinct characteristics that influence how the application is deployed and executed:

Packaged apps are distributed as MSIX packages, which include all necessary files and dependencies for the application. These apps are installed through the Microsoft Store or by sideloading the MSIX package.

The primary advantages of packaged apps include simplified deployment with a streamlined installation process that offers built-in support for updates, enhanced security through a sandboxed environment, and better integration with Windows features such as notifications and live tiles. On the other hand, unpackaged apps are distributed as loose files and installed manually or through traditional installers. They offer greater flexibility by allowing direct modification of files and provide compatibility with legacy systems and non-native software.

Building a WinUI 3 app

Creating a simple WinUI 3 application involves several steps, beginning with setting up a new project. To start, launch Visual Studio and create a new WinUI 3 project. If you require additional guidance on this process, refer to section 3.1 for detailed instructions on setting up your project environment.

Once the project is created, the next step is to design the **user interface (UI)**. Open the `MainWindow.xaml` file, where the main window's layout is defined using XAML. For instance, you can create a simple design with a textbox for user input, a button to trigger an event, and a text block to display a greeting. The XAML code defines and structures these elements in a grid, allowing for a clean, centered layout. Here's an example of how the design could look:

```
<Grid>
    <StackPanel HorizontalAlignment="Center" VerticalAlignment="Center">
        <TextBlock Text="Enter Your Name:" />
        <TextBox x:Name="NameTextBox" Width="200" Margin="5" />
        <Button Content="Greet Me" Click="OnGreetButtonClick" />
        <TextBlock x:Name="GreetingTextBlock" FontSize="16" Margin="5" />
    </StackPanel>
</Grid>
```

This simple UI allows users to input their name, click a button, and see a personalized greeting displayed in the text block.

After designing the UI, you must implement event handling to make the application interactive. Open the `MainWindow.xaml.cs` file, which acts as the code behind the XAML layout. In this file, you will define an event handler that responds to the button click event.

The following code retrieves the user's input from the textbox and updates the text block with a personalized greeting when the button is clicked:

```
private void OnGreetButtonClick(object sender, RoutedEventArgs e)
{
    string name = NameTextBox.Text;
    GreetingTextBlock.Text = $"Hello, {name}!";
}
```

This interaction lets the application respond dynamically to user input, creating a simple but effective user experience.

The process of adding and configuring controls in XAML is straightforward. For instance, `TextBox` is used to accept user input, as shown here:

```
<TextBox x:Name="NameTextBox" Width="200" Margin="5" />
```

`Button` triggers the event handler when the button is clicked:

```
<Button Content="Greet Me" Click="OnGreetButtonClick" />
```

Finally, `TextBlock` displays the output, such as the personalized greeting:

```
<TextBlock x:Name="GreetingTextBlock" FontSize="16" Margin="5" />
```

In the XAML file, `Button` is defined with an event that activates the `OnGreetButtonClick` method when the user clicks it:

```
<Button Content="Greet Me" Click="OnGreetButtonClick" />
```

The corresponding code-behind method retrieves the user's name from the textbox and updates `TextBlock` with a greeting message:

```
private void OnGreetButtonClick(object sender, RoutedEventArgs e)
{
    string name = NameTextBox.Text;
    GreetingTextBlock.Text = $"Hello, {name}!";
}
```

This simple interaction highlights how WinUI 3 enables developers to create applications that dynamically respond to user input and update the UI.

By following these steps and incorporating the provided code, you can build a basic yet functional WinUI 3 application demonstrating the power of event handling and dynamic UI updates.

Running and debugging a WinUI 3 app

Running and debugging applications are two significant steps to verify proper functionality during development. Here is how to do it.

This is how to run the application:

1. **Build the project**: Click **Build**. Now, click **Build Solution**. This will compile the project.
2. **Start debugging**: Click **Debug**, followed by **Start Debugging** (or press *F5*). This will run the application.

This is how to debug the application:

1. **Set breakpoints**: Click in the margin next to the line numbers. This will set breakpoints in the code.
2. **Step through the code**: Use the debugging toolbar to review the code, inspect variables, and evaluate expressions.
3. **Use Watch windows**: Use the **Watch** windows to monitor variables and expressions while debugging.

Issues can be identified and fixed by running and debugging the application. This will verify that the application works as intended.

After successfully running and debugging your WinUI 3 application to ensure everything functions correctly, the next step is to focus on implementing a structured approach to managing your app's user interface and business logic. This is where the **Model-View-ViewModel** (**MVVM**) pattern comes into play. By separating the concerns of data, user interface, and logic, MVVM makes it easier to manage and scale your application. In the following section, we'll explore how to implement data binding and the MVVM pattern in your WinUI 3 app, starting with defining the model.

Data binding and the MVVM pattern in WinUI 3

The MVVM pattern is widely recognized for its ability to organize complex software projects effectively. It achieves this by separating the user interface (view) from the business logic (view model) and the data (model), allowing for a clear division of development concerns. The first step in implementing the MVVM pattern in a WinUI 3 application is to define the model. For example, you can create a model class representing the data structure. The following code demonstrates how to define a simple `User` class:

```
public class User
{
```

```
    public string Name { get; set; }
    public int Age { get; set; }
}
```

Once the model is defined, the next step is to create the view model. The view model handles the business logic and implements the `INotifyPropertyChanged` interface for data binding purposes. In this example, the `UserViewModel` class manages the `User` object, and any changes to the properties are communicated to the view using the `OnPropertyChanged` method:

```
public class UserViewModel : INotifyPropertyChanged
{
    private User user;

    public UserViewModel()
    {
        user = new User { Name = "John Doe", Age = 30 };
    }

    public string Name
    {
        get => user.Name;
        set
        {
            if (user.Name != value)
            {
                user.Name = value;
                OnPropertyChanged(nameof(Name));
            }
        }
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new
        PropertyChangedEventArgs(propertyName));
    }
}
```

The final step is to bind the view to the view model. This is achieved by setting `DataContext` of the view to an instance of `UserViewModel` in the code-behind. The UI elements in the XAML file are then bound to the properties of the view model, shown here:

```
public MainWindow()
{
    InitializeComponent();
    RootGrid.DataContext = new UserViewModel();
}
```

In the XAML file, you can bind `TextBox` to the `Name` property of the view model:

```
<TextBox Text="{Binding Name}" Width="200" Margin="5" />
```

This approach separates the UI and business logic, making the development process more flexible and easier to maintain. The MVVM pattern also simplifies testing and future maintenance, improving the overall scalability of the application.

Enhancing the user experience with animations and transitions can make the application more engaging and responsive. In WinUI 3, you can use XAML to define animations. For example, you can use a Storyboard to animate the width of a rectangle. In the following XAML code, a button triggers an animation that increases the width of the rectangle:

```
<StackPanel>
    <Button Content="Animate" Click="OnAnimateButtonClick" />
    <Rectangle x:Name="AnimatedRectangle" Width="100" Height="100"
    Fill="Blue" />
    <Storyboard x:Key="RectangleAnimation">
        <DoubleAnimation
            Storyboard.TargetName="AnimatedRectangle"
            Storyboard.TargetProperty="Width"
            From="100" To="200" Duration="0:0:2" />
    </Storyboard>
</StackPanel>
```

In the code-behind, you can trigger the animation using the following method:

```
private void OnAnimateButtonClick(object sender, RoutedEventArgs e)
{
    Storyboard storyboard = (Storyboard)Resources["RectangleAnimation"];
    storyboard.Begin();
}
```

Animations and transitions provide visual feedback and create smooth interactions, which is essential for enhancing user engagement.

Integrating a WinUI 3 application with Windows APIs and services can expand its functionality and improve the user experience. For example, you can use the File Picker API to allow users to select files from their system. After adding the necessary references to the Windows SDK libraries, the following code demonstrates how to implement the file picker functionality in the code-behind:

```
private async void OnOpenFileButtonClick(object sender, RoutedEventArgs e)
{
    var picker = new Windows.Storage.Pickers.FileOpenPicker();
    picker.ViewMode = Windows.Storage.Pickers.PickerViewMode.Thumbnail;
    picker.SuggestedStartLocation = Windows.Storage.Pickers.
PickerLocationId.PicturesLibrary;
    picker.FileTypeFilter.Add(".jpg");
    picker.FileTypeFilter.Add(".jpeg");
    picker.FileTypeFilter.Add(".png");

    Windows.Storage.StorageFile file = await picker.PickSingleFileAsync();
    if (file != null)
    {
        // Handle the selected file
    }
    else
    {
        // No file was selected
    }
}
```

By incorporating Windows APIs, you can leverage the full capabilities of the Windows platform, enhancing user interaction through advanced features.

This chapter has provided a comprehensive guide to creating a WinUI 3 project in Visual Studio, from setting up the project structure to implementing advanced features such as MVVM, animations, and Windows API integration. Following the steps and examples discussed, developers can create powerful and responsive WinUI 3 applications that fully utilize modern Windows features. Whether you are a beginner or an experienced developer, this chapter is a valuable resource for your development journey.

Summary

In this chapter, we explored the fundamental aspects of WinUI 3 and its role in modern Windows application development. We began by understanding WinUI 3's architecture and integration with the Windows operating system, highlighting its performance benefits and native capabilities. We then guided you through setting up a WinUI 3 development environment in Visual Studio. We covered essential concepts such as the Fluent design system and its application in creating modern, responsive user interfaces.

By walking through the process of building a simple WinUI 3 application, you learned how to design user interfaces with XAML, manage events, and handle state effectively. We also delved into best practices for leveraging Windows-specific features and APIs to enhance your application's functionality and user experience.

This foundational knowledge prepares you to take the next step in your development journey. In the following chapter, we will dive deeper into constructing feature-rich desktop applications. You will learn advanced UI design techniques, integrate more complex components, and build a fully functional, modern, desktop application from start to finish.

9

Building a Modern Desktop App with WinUI 3

This chapter will cover advanced controls in WinUI 3 that enhance user experience and design, including NavigationView, ListView, and GridView. These controls will enable us to build feature-rich Windows applications with intuitive and aesthetically pleasing interfaces. We start by exploring NavigationView, which allows for seamless navigation between different pages or sections of an application. We will then integrate a ListView to dynamically display a collection of data items, such as courses, in a scrollable format. Following that, we will implement a GridView to present data in a structured, grid-like layout, ideal for scenarios such as product catalogs or course selection interfaces. Finally, we will examine TreeView, which helps display hierarchical data such as course content, enabling users to expand or collapse nodes as needed.

Throughout the chapter, you will learn how to create a responsive and interactive desktop app, leveraging the powerful capabilities of WinUI 3. We will cover how these controls work together to create a modern desktop app with enhanced data presentation and navigation features. This chapter will conclude with a project where we combine all these elements to build a fully functional and visually engaging application.

To achieve this, we will cover the following topics:

- Advanced WinUI 3 controls
- Using ListViews
- Using GridViews

Technical requirements

This chapter requires access to **Visual Studio Community** (`https://visualstudio.microsoft.com/downloads/`) with the following workloads installed:

- ASP.NET and web development
- .NET multi-platform app UI development
- .NET desktop development
- Windows application development

The code shown in the examples can be found at `https://github.com/PacktPublishing/GUI-Programming-with-C-Sharp`.

Advanced WinUI 3 controls

Providing a user-friendly and visually appealing interface is crucial in modern application development. WinUI 3 offers several advanced controls that allow developers to build applications with sophisticated layouts and interactive elements. These controls enhance the user experience by making navigation seamless, improving data presentation, and ensuring that applications are responsive across various device types.

This section will explore some of the most important controls available in WinUI 3, including NavigationView, ListView, GridView, and TreeView. These controls are vital in creating structured, navigable, and scalable applications.

We will begin with NavigationView, which lets users quickly move between different application pages. This control includes built-in responsiveness, ensuring an optimal user experience on various screen sizes.

By understanding and utilizing these advanced controls, you'll be able to create feature-rich Windows applications that are functional, intuitive, and aesthetically pleasing. Let's dive into the NavigationView example to see how to incorporate it into your application.

Learning management system example

In this example, we explore using the NavigationView control in a WinUI application to create a basic layout for a **Learning Management System** (**LMS**). NavigationView is a modern and flexible control that allows developers to develop side navigation menus, providing a user-friendly experience for navigating between different sections of an application.

This LMS example demonstrates how to create a structured navigation menu with options such as **Home**, **Profile**, **Favorite Courses**, **Contact**, and **Settings**. Alongside the navigation menu, the example includes a title section and a welcome message inside a styled container.

The example is designed to achieve the following:

- **Create a responsive navigation pane**: The navigation pane allows users to switch between different sections of the LMS
- **Present the main content**: The main content area contains a title and a welcome message styled with different fonts, colors, and layout properties
- **Enhance usability and design**: The design creates an intuitive and visually appealing interface using icons and appropriate layout settings

Throughout the example, we utilize critical WPF controls such as NavigationView, NavigationViewItem, StackPanel, TextBlock, and Border to create a simple yet elegant **User Interface (UI)** that mimics the structure of a modern learning platform. This structure could include additional functionality for navigating between different pages in a complete LMS application. Here's a breakdown of the code.

Step 0 – create a new WinUI project

In Visual Studio, create a new project, filter by C#, Windows, and WinUI, and create a blank app. Then, open the `MainWindow.xaml` file.

Step 1 – define the NavigationView control

The NavigationView control is used to create a navigation pane, where users can navigate between different sections such as **Home**, **Profile**, **Favorite Courses**, and so on. This control helps to structure your app's UI in a modern and user-friendly way:

```
<NavigationView x:Name="MyNavigationView"
                IsPaneOpen="True"
                IsBackButtonVisible="Collapsed"
                PaneDisplayMode="Left"
                VerticalAlignment="Stretch"
                HorizontalAlignment="Stretch">
```

- `x:Name="MyNavigationView"`: Names the control to reference in the code behind
- `IsPaneOpen="True"`: The pane (navigation menu) is open by default
- `IsBackButtonVisible="Collapsed"`: Hides the back button since this example doesn't need it

- `PaneDisplayMode="Left"`: The navigation pane is displayed on the left
- `VerticalAlignment="Stretch"` and `HorizontalAlignment="Stretch"`: Ensure the NavigationView control stretches to fill available space vertically and horizontally

Step 2 — add navigation items (menu)

Within the NavigationView control, we define several menu items using NavigationViewItem. Each item represents a different section of the LMS (**Home**, **Profile**, **Favorite Courses**, etc.):

```
<NavigationView.MenuItems>
    <NavigationViewItem Icon="Home" Content="Home" Tag="home" />
    <NavigationViewItem Icon="Contact" Content="Profile" Tag="profile"/>
    <NavigationViewItem Icon="Bookmarks" Content="Favorite Courses"
Tag="courses" />
    <NavigationViewItem Icon="Phone" Content="Contact" Tag="contact" />
    <NavigationViewItem Icon="Setting" Content="Settings" Tag="settings" />
</NavigationView.MenuItems>
```

Each `NavigationViewItem` represents a section of the LMS:

- `Icon`: Specifies the icon for the item (e.g., **Home** icon for the **Home** section)
- `Content`: The label that will be shown (e.g., **Home** or **Profile**)
- `Tag`: A unique identifier for each item (used for programmatic navigation)

Step 3 — display the main content

Inside the NavigationView control, a StackPanel is used to hold the page's main content, including a title and a welcome message inside a border:

```
<StackPanel Orientation="Vertical" VerticalAlignment="Stretch"
HorizontalAlignment="Stretch">
    <TextBlock
        Text="Learning Management System"
        FontSize="32"
        FontWeight="Bold"
        Foreground="#182980"
        Margin="0 0 50 0"
        HorizontalAlignment="Center"
    />
```

- **StackPanel**: A container for arranging the elements vertically:
  - `Orientation="Vertical"`: Aligns elements vertically
  - `VerticalAlignment="Stretch"` and `HorizontalAlignment="Stretch"`: Ensures the StackPanel stretches to fit the available space
- **TextBlock**: Displays the **Learning Management System** title:
  - `Text="Learning Management System"`: The title text
  - `FontSize="32"`: Sets the font size
  - `FontWeight="Bold"`: Makes the text bold
  - `Foreground="#182980"`: Sets the text color
  - `HorizontalAlignment="Center"`: Centers the title horizontally
  - `Margin="0 0 50 0"`: Adds spacing below the title

Step 4 – add a border with welcome text

The Border control wraps the welcome message inside a styled container with a black background and padding:

```
<Border Background="Black" Padding="10">
    <TextBlock
       TextWrapping="Wrap"
       FontSize="16"
       Margin="10"
       LineHeight="24"
       Foreground="#ffffff">
        Welcome to LMS! Here, you can find the details of your favorite
educational courses, grades, and more.
        It is designed to support students in personalized learning, help
them manage their work effectively, and
        seek guidance from educators in difficult situations.
    </TextBlock>
</Border>

</StackPanel>
</NavigationView>
```

- **Border**: A container with a border or background around its child element:
  - `Background="Black"`: Sets the background color of the border to black
  - `Padding="10"`: Adds padding inside the border to separate the text from the border's edges
- **TextBlock**: Displays the welcome message:
  - `TextWrapping="Wrap"`: Ensures the text wraps to the next line if it exceeds the available width
  - `FontSize="16"`: Sets the size of the text
  - `Margin="10"`: Adds a margin around the text
  - `LineHeight="24"`: Sets the height of each line of text for better readability
  - `Foreground="#ffffff"`: Sets the text color to white for contrast against the black background

When selecting the "hamburger" control, the menu will display the following:

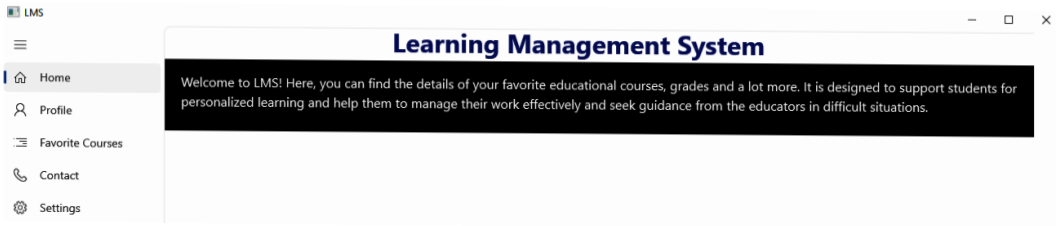


Figure 9.1 – LMS menu

After setting up a welcoming message using the Border control, we can now enhance the navigation experience by adding another interactive element. The next step introduces a feature that will help users explore their favorite courses with ease.

ListView

In this section, we extend the *NavigationView LMS* example by adding a new feature: a dynamically populated list of favorite courses using the ListView control. ListView is a versatile control in WPF that allows us to display a collection of data items, such as courses, in a scrollable list format. By integrating it into the existing navigation structure, we allow users to view their favorite courses directly within the **Favorite Courses** section of the app.

The goal of this section is to demonstrate the following:

- **Seamless navigation**: The navigation menu allows users to switch between different sections (such as **Home**, **Profile**, or **Favorite Courses**)
- **Dynamic content**: The courses list is populated dynamically from the code behind, which binds the list of courses to the ListView control, ensuring flexibility in content updates
- **User-friendly UI**: By combining NavigationView and ListView, we create a structured and interactive interface where users can easily access their favorite courses

This feature is an important addition to the app, enhancing user interaction by allowing users to navigate through different parts of the app and view personalized course data in a clean, organized format.

First, we need to modify the NavigationView control from the previous example to include an additional section for **Favorite Courses**. We achieve this by adding a new menu item within `NavigationView.MenuItems`. This step ensures users can navigate to the **Favorite Courses** section from the side pane:

```
<NavigationView x:Name="MyNavigationView"
                IsPaneOpen="True"
                IsBackButtonVisible="Collapsed"
                PaneDisplayMode="Left"
                VerticalAlignment="Stretch"
                HorizontalAlignment="Stretch">
    <!-- Navigation Menu Items -->
    <NavigationView.MenuItems>
        <NavigationViewItem Icon="Home" Content="Home" Tag="home" />
        <NavigationViewItem Icon="Contact" Content="Profile"
Tag="profile"/>
        <NavigationViewItem Icon="Bookmarks" Content="Favorite Courses"
Tag="courses" />
        <NavigationViewItem Icon="Phone" Content="Contact" Tag="contact"
/>
        <NavigationViewItem Icon="Setting" Content="Settings"
Tag="settings" />
    </NavigationView.MenuItems>

    <!-- Main Content -->
    <StackPanel x:Name="ContentPanel" Orientation="Vertical"
VerticalAlignment="Stretch" HorizontalAlignment="Stretch">
```

```
        <!-- Placeholder for dynamic content -->
    </StackPanel>
</NavigationView>
```

In this code, we've added a `courses` tag to the existing **Favorite Courses** section. The `StackPanel` element named `ContentPanel` will serve as the container for dynamic content that changes based on the user's navigation selection.

Adding the ListView control for Favorite Courses

Now that we've created a menu option for **Favorite Courses**, we can define the layout for the section. The ListView control, which will hold the list of favorite courses, is initially hidden using the `Visibility="Collapsed"` property. This panel will become visible when the user selects the **Favorite Courses** option from the navigation pane. To achieve this, replace this code:

```
    <!-- Main Content -->
    <StackPanel x:Name="ContentPanel" Orientation="Vertical"
VerticalAlignment="Stretch" HorizontalAlignment="Stretch">
        <!-- Placeholder for dynamic content -->
    </StackPanel>
```

In its place, insert this code:

```
<StackPanel x:Name="FavoriteCoursesPanel" Visibility="Collapsed">
    <TextBlock
        Text="My Favorite Courses"
        FontSize="32"
        FontWeight="Bold"
        Foreground="#182980"
        Margin="0 0 50 0"
    />
    <ListView x:Name="FavoriteCoursesList">
        <ListView.Items>
            <TextBlock Text="Course 1" />
            <TextBlock Text="Course 2" />
            <TextBlock Text="Course 3" />
            <TextBlock Text="Course 4" />
            <TextBlock Text="Course 5" />
        </ListView.Items>
    </ListView>
</StackPanel>
```

Here, we use a StackPanel that contains a title (**My Favorite Courses**), followed by a ListView control with several static course items. The `FavoriteCoursesPanel` is initially hidden from view, but when the user selects **Favorite Courses**, this panel will display the list of their favorite courses.

Displaying Favorite Courses dynamically in the code behind

To make the app more interactive, we can dynamically populate the list of favorite courses using the code-behind in `MainWindow.xaml.cs`. We will add a method that fills the ListView with dynamic data when the user selects the **Favorite Courses** section in the navigation pane:

```
public MainWindow()
{
    InitializeComponent();
    MyNavigationView.SelectionChanged += MyNavigationView_SelectionChanged;
    ShowFavoriteCourses();
}
```

In the `MainWindow` constructor, we initialize the app by calling `InitializeComponent()` and registering an event handler for `MyNavigationView.SelectionChanged`. This event triggers whenever the user selects a different section from the navigation pane. Additionally, `DisplayFavoriteCourses()` is called to set up the initial display for the favorite courses:

```
// Handle NavigationView selection changes
private void MyNavigationView_SelectionChanged(NavigationView sender,
NavigationViewSelectionChangedEventArgs args)
{
    var selectedTag = (args.SelectedItem as NavigationViewItem).Tag.
ToString();

    if (selectedTag == "courses")
    {
        ShowFavoriteCourses();
    }
    else
    {
        HideAllPanels();
    }
}
```

In the `MyNavigationView_SelectionChanged` event handler, the app checks the selected section using the `Tag` property of the selected item. If the **Favorite Courses** section is chosen (identified by the `courses` tag value), it calls `ShowFavoriteCourses`. If another section is selected, it calls `HideAllPanels` to hide all other panels:

```
// Display the Favorite Courses section
private void ShowFavoriteCourses()
{
    HideAllPanels();  // Ensure other panels are hidden
    FavoriteCoursesPanel.Visibility = Visibility.Visible;
    var courses = new[] { "OOP", "Data Structures", "Operating Systems",
    "App Development", "Database" };
    FavoriteCoursesList.ItemsSource = courses;
}
```

The `ShowFavoriteCourses` method manages the visibility of the **Favorite Courses** panel. It first hides all other panels by calling `HideAllPanels`, then makes the `FavoriteCoursesPanel` visible. `FavoriteCoursesList` is populated with a list of course names using the `ItemsSource` property, creating a dynamic and scrollable list for the user:

```
// Hide all content panels
private void HideAllPanels()
{
    FavoriteCoursesPanel.Visibility = Visibility.Collapsed;
}
```

The `HideAllPanels` method is a utility function that collapses the visibility of all content panels, ensuring that only the selected section is visible at any given time.

This example demonstrates how to combine the NavigationView and ListView controls to create a flexible, interactive app. It allows users to navigate between different sections and view dynamic data in a structured, scrollable list. The **Favorite Courses** section is seamlessly integrated into the overall app layout, offering a clean and engaging user experience. This structure is easily expandable for future enhancements, such as including additional sections or integrating more complex data sources.

Here is how the UI looks with the added code:

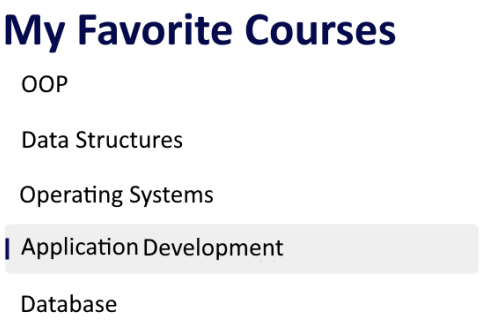


Figure 9.2 – Panel view

Now that we've successfully integrated a dynamic **Favorite Courses** section using ListView, let's take it a step further by enhancing how data is displayed. While ListView is perfect for simple, linear data presentation, there are scenarios where a more structured layout with multiple columns can offer a clearer view.

GridView

The GridView control in WinUI 3 provides a powerful way to display data in a grid layout with multiple columns, offering logical content divisions. This makes it especially useful for presenting structured data such as product catalogs, image galleries, or course listings. In this example, we will implement a **Favorite Course Selector app**, displaying a grid of favorite courses and allowing the user to select one. Each course will be visually represented as an individual item within the grid.

The first element in our layout is a `TextBlock` that displays the **Favorite Course Selector** title. This title is formatted to be visually prominent, with a font size of 32, bold weight, and the color blue (#182980). The `HorizontalAlignment="Center"` property ensures that the title is centered horizontally in the application window. A margin is added below the title to provide some spacing between the title and the following content.

Next, we define the GridView, which will contain the course items. The GridView is configured to stretch horizontally and vertically within the window, making it responsive to changes in the window size. By setting `SelectionMode="Single"`, we ensure that only one item can be selected inside the GridView.We created five StackPanel elements for the items collection, each representing a different course.

Every StackPanel element has a fixed width and height, with a light gray background and margins to separate the items visually. Inside each StackPanel element is a TextBlock containing the course name centered horizontally using the `HorizontalAlignment="Center"` property. This ensures the course names are consistently aligned in the middle of their respective grid items:

```
<TextBlock
    Text="Favorite Course Selector"
    FontSize="32"
    FontWeight="Bold"
    Foreground="#182980"
    Margin="0 0 50 0"
    HorizontalAlignment="Center"
/>
<Grid>
    <GridView x:Name="FavoriteCoursesGrid"
      HorizontalAlignment="Stretch"
      VerticalAlignment="Stretch"
      SelectionMode="Single"
      Margin="0">
        <GridView.Items>
            <StackPanel Width="200" Height="50" Background="LightGray"
Margin="10">
                <TextBlock Text="Application Development"
HorizontalAlignment="Center"/>
            </StackPanel>
            <StackPanel Width="200" Height="50" Background="LightGray"
Margin="10">
                <TextBlock Text="OOP" HorizontalAlignment="Center"/>
            </StackPanel>
            <StackPanel Width="200" Height="50" Background="LightGray"
Margin="10">
                <TextBlock Text="Data Structures"
HorizontalAlignment="Center"/>
            </StackPanel>
            <StackPanel Width="200" Height="50" Background="LightGray"
Margin="10">
                <TextBlock Text="Operating Systems"
HorizontalAlignment="Center"/>
```

```
            </StackPanel>
            <StackPanel Width="200" Height="50" Background="LightGray"
Margin="10">
                <TextBlock Text="Database" HorizontalAlignment="Center"/>
            </StackPanel>
        </GridView.Items>
    </GridView>
</Grid>
```

A GridView is particularly effective in this scenario because it provides a clear, structured way to present the course options. The grid format makes it easy for users to differentiate between the courses and make a visual selection. The individual StackPanel elements allow us to customize the appearance of each grid item, ensuring a consistent layout and design. Using TextBlock elements inside the StackPanel elements, the course names are displayed straightforwardly, centered in each grid item.

This GridView example illustrates how to create a visually structured layout for a favorite course selector app in WinUI 3. Using a combination of TextBlock, StackPanel, and GridView, we have presented a list of courses in a grid format, allowing for easy selection by the user. The flexibility of the GridView control makes it an ideal choice for applications that require a structured presentation of items, such as course selectors, product listings, or image galleries. The layout is fully responsive, with horizontal and vertical stretching, ensuring it adapts well to different window sizes. This approach provides a clean, user-friendly interface that can be easily extended to incorporate more features, such as additional course details or interactivity upon selection.

Here is the screen generated by this code.

Figure 9.3 – Grid view

The screen shows the **Favorite Course Selector** interface with a grid of course options such as **Application Development**, **OOP**, **Data Structures**, **Operating Systems**, and **Database**. One of the options, **Application Development**, is highlighted, indicating it has been selected.

Summary

In this chapter, we explored the process of developing modern desktop applications using WinUI 3, focusing on advanced UI controls and responsive design techniques. We began by introducing the NavigationView control, which allows for seamless navigation between different sections of an application, providing users with an intuitive way to move through various features and pages.

Next, we implemented the ListView control, which is ideal for displaying data collections in a scrollable format, making it helpful in presenting lists such as courses or other groupings. We then explored the GridView control, which offers a grid-like structure to display data in an organized, multi-column layout. This control is particularly effective for applications that require structured data presentation, such as product catalogs or course selectors.

The next chapter will build upon this foundation by exploring XAML controls and data binding. We will examine how WinUI 3 enables robust data binding techniques that connect your application's UI to the underlying data models, allowing for more dynamic and interactive applications.

10

Advanced WinUI

Creating visually engaging, responsive, and high-performing applications is essential in modern desktop application development. This chapter focuses on techniques and best practices for optimizing WinUI 3 applications, enhancing user experience, and ensuring smooth, efficient interactions. The topics covered include performance optimization, control customization, data binding, the **Model-View-ViewModel** (**MVVM**) architectural pattern, and essential profiling and debugging techniques.

We'll begin by exploring **Control Customization and Building Complex Interfaces**. Customizing controls allows developers to design visually appealing UIs that enhance user interaction. We'll look at how properties such as background, padding, margin, font size, weight, and foreground can be modified to create a polished interface. These properties give you flexibility in styling elements to match your app's theme and improve readability and structure, ensuring an intuitive experience for users.

Next, we'll dive into **Data Binding and MVVM**. The MVVM pattern decouples the UI from the application's business logic, creating a more organized and maintainable code structure. By binding UI elements directly to data and business logic in the ViewModel, MVVM allows developers to create more dynamic applications where changes in the model layer are automatically reflected in the view. This section includes practical examples of implementing data binding in WinUI 3 and demonstrates how the MVVM pattern simplifies data handling, ensuring consistent, real-time updates across the application.

We then cover **Optimizing Application Performance**, detailing critical strategies for reducing load times, optimizing memory usage, and minimizing UI latency. Techniques such as lazy loading, asynchronous data loading, and efficient data binding reduce resource usage, allowing applications to run smoothly even with complex features. This section provides insight into managing resources, structuring layouts, and selecting animations to create fast, responsive user interfaces.

The chapter explores profiling and debugging tools to further enhance application quality. Profiling is essential for tracking performance metrics, such as memory consumption and response times, which help identify bottlenecks. Debugging techniques, including hot reload, breakpoints, conditional breakpoints, and memory leak detection, provide developers with tools to catch and resolve issues early in development. Developers can build robust, error-free applications that deliver a seamless user experience by effectively using these techniques.

In this chapter, you'll comprehensively understand advanced WinUI 3 development. You will learn how to optimize performance, organize code with MVVM, customize controls for improved user interaction, and use profiling and debugging tools to ensure smooth and reliable applications. Each topic is complemented by practical examples and techniques that empower you to create high-quality, efficient, and maintainable WinUI 3 applications.

Technical requirements

This chapter requires access to **Visual Studio Community** (`https://visualstudio.microsoft.com/downloads/`) with the following workloads installed:

- ASP.NET and web development
- .NET multi-platform app UI development
- .NET desktop development
- Windows application development

The code shown in the examples can be found here: `https://github.com/PacktPublishing/GUI-Programming-with-C-Sharp`.

TreeViews

This section demonstrates how to use TreeView in a WinUI 3 application to display hierarchical data, such as folder structures or family trees. TreeView is a versatile control that allows users to expand and collapse nodes, revealing or hiding child items under parent nodes.

Since TreeView doesn't support direct content binding, we define a function to populate the tree nodes. In this example, we'll create a course content display for different subjects. To set up the TreeView, open `MainWindow.xaml` and add the following code within a `StackPanel`:

```
<TextBlock
    Text="Course Content Details"
    FontSize="32"
    FontWeight="Bold"
    Foreground="#182980"
    Margin="0 0 50 0"
    HorizontalAlignment="Center"
/>
<Grid>
    <TreeView x:Name="CourseContentTree" Margin="20" />
</Grid>
```

Next, in `MainWindow.xaml.cs`, create a `DisplayCourseContent` method to add course topics and call it in the `MainWindow` constructor:

```
public MainWindow()
{
    this.InitializeComponent();
    DisplayCourseContent();
}

private void DisplayCourseContent()
{
    TreeViewNode oop = new TreeViewNode { Content = "Object Oriented
Programming" };
    oop.Children.Add(new TreeViewNode { Content = "Constructors and
Destructors" });
    oop.Children.Add(new TreeViewNode { Content = "Inheritance" });
    oop.Children.Add(new TreeViewNode { Content = "Polymorphism" });
    oop.Children.Add(new TreeViewNode { Content = "Composition" });
    oop.Children.Add(new TreeViewNode { Content = "Aggregation" });

    TreeViewNode dsa = new TreeViewNode { Content = "Data Structures" };
    dsa.Children.Add(new TreeViewNode { Content = "Linked List" });
    dsa.Children.Add(new TreeViewNode { Content = "Queue" });
```

```
    dsa.Children.Add(new TreeViewNode { Content = "Stack" });

    TreeViewNode desktopApps = new TreeViewNode { Content = "Desktop
Applications" };
    desktopApps.Children.Add(new TreeViewNode { Content = "C#" });
    desktopApps.Children.Add(new TreeViewNode { Content = "WinUI 3" });
    desktopApps.Children.Add(new TreeViewNode { Content = ".NET" });

    CourseContentTree.RootNodes.Add(oop);
    CourseContentTree.RootNodes.Add(dsa);
    CourseContentTree.RootNodes.Add(desktopApps);
}
```

To organize and present course content effectively, we implement the following features:

- **Course topics display**: We create a tree structure that includes different subjects and topics under each
- **Dynamic population**: The `DisplayCourseContent` function defines each course topic as a TreeViewNode and organizes subtopics as child nodes
- **Node management**: The root nodes (OOP, data structures, and desktop applications) are added to `CourseContentTree.RootNodes`, setting up a collapsible, expandable tree

This structure allows users to navigate various course topics by expanding or collapsing nodes, creating an organized, easy-to-follow display of course content.

Once this is run, the screen should look like this:

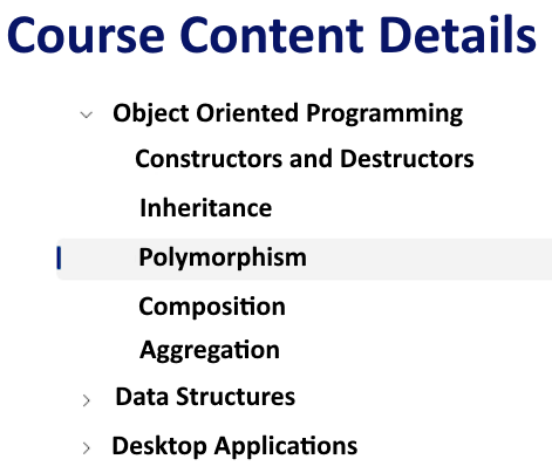


Figure 10.1 – TreeView

Customizing controls and building complex interfaces

Customizing controls in WinUI 3 enables the development of visually appealing and user-friendly interfaces. By modifying control properties such as background, padding, margin, and font size, developers can design interfaces that enhance the user experience and highlight important application elements.

WinUI 3 offers several properties for customizing the appearance and behavior of UI elements. Here's an overview of commonly used properties and examples to demonstrate their impact.

The Background property

The `Background` property changes the background color of an element, helping it stand out visually or align with the application's theme:

```
<Border Background="#b0d3f7">
    <TextBlock>
       Welcome to LMS! Here, you can find the details of your favorite
educational courses, grades, and more.
    </TextBlock>
</Border>
```

In this example, the border control's background color is set to a soft blue (`#b0d3f7`), giving it a calm, inviting look. The screen should look as follows:

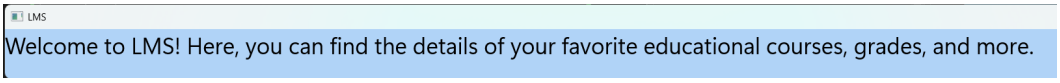


Figure 10.2 – Border and TextBlock

The Padding property

The `Padding` property controls the space between content and an element's borders, allowing content to "breathe" within the UI:

- `Padding="50"` applies equal padding (50) on all four sides
- `Padding="50 100"` applies 50 pixels on the left and right and 100 pixels at the top and bottom
- `Padding="50 100 150 200"` applies padding individually: 50 on the left, 100 at the top, 150 on the right, and 200 at the bottom

Here's an example with padding customization:

```
<Border Background="#b0d3f7" Padding="40 20">
    <TextBlock>
        Welcome to LMS! Here, you can find the details of your favorite
educational courses, grades, and more.
    </TextBlock>
</Border>
```

Before setting padding, text may look cramped. Adding padding makes the text more comfortable to read, as it's no longer too close to the edges of the border. The screen should look as follows:

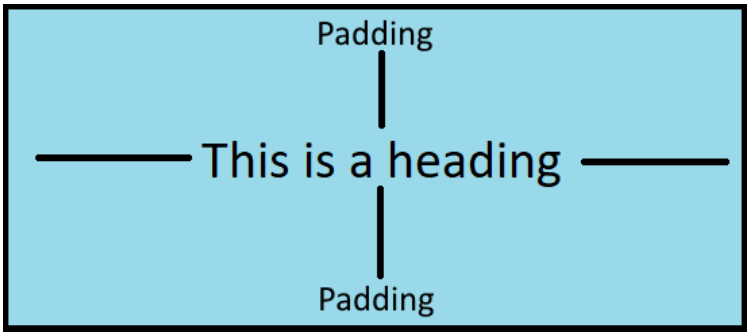


Figure 10.3 – Padding Property

The Margin property

`Margin` determines the space outside an element, affecting its positioning relative to surrounding elements. It helps create a visually balanced layout:

- `Margin="50"` applies equal margins on all four sides
- `Margin="50 100"` sets 50 pixels on the left and right and 100 pixels at the top and bottom
- `Margin="50 100 150 200"` specifies each side individually: `50` for the left, `100` for the top, `150` for the right, and `200` for the bottom

Here's an example with `Margin` applied:

```
<Border Background="#b0d3f7" Padding="40 20" Margin="50 25">
    <TextBlock>
        Welcome to LMS! Here, you can find the details of your favorite
educational courses, grades, and more.
    </TextBlock>
</Border>
```

Applying margin spaces to the border control away from other UI elements makes the interface look less crowded. The screen looks as follows:

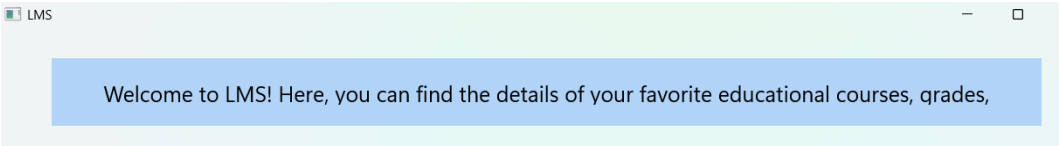


Figure 10.4 – Applying Margins

The FontSize property

The `FontSize` property adjusts the size of the text within a control, making it larger or smaller based on the desired emphasis.

Here's an example:

```
<Border Background="#b0d3f7" Padding="40 20" Margin="50 25">
    <TextBlock FontSize="20">
        Welcome to LMS! Here, you can find the details of your favorite
educational courses, grades, and more.
    </TextBlock>
</Border>
```

With `FontSize="20"`, the text is set to be larger than the default, making it more readable and prominent. The screen looks as follows:

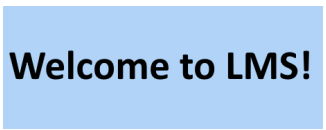


Figure 10.5 – Increasing Font Size

The FontWeight property

`FontWeight` adjusts the thickness of the text, allowing for visual emphasis.

Here's an example:

```
<Border Background="#b0d3f7" Padding="40 20" Margin="50 25">
    <TextBlock FontSize="20" FontWeight="Bold">
        Welcome to LMS! Here, you can find the details of your favorite
educational courses, grades, and more.
    </TextBlock>
```

```
</Border>
```

Setting `FontWeight="Bold"` makes the text thicker, giving it a bolder appearance to draw attention. The screen looks as follows:

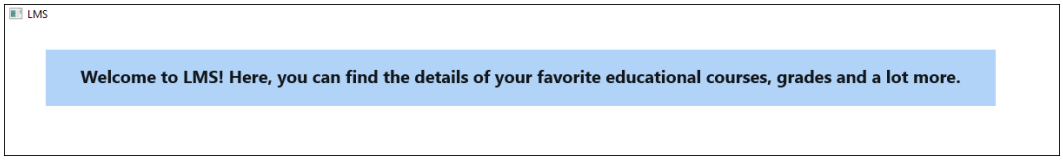


Figure 10.6 – Applying Font Weight

The Foreground property

The `Foreground` property changes the color of the text, enhancing the readability and matching the application's color scheme.

Here's an example:

```
<Border Background="#b0d3f7" Padding="40 20" Margin="50 25">
    <TextBlock FontSize="20" FontWeight="Bold" Foreground="#182980">
        Welcome to LMS! Here, you can find the details of your favorite
educational courses, grades, and more.
    </TextBlock>
</Border>
```

Here, `Foreground="#182980"` sets the text color to a deep blue, making it stand out against the light background. The screen looks as follows:

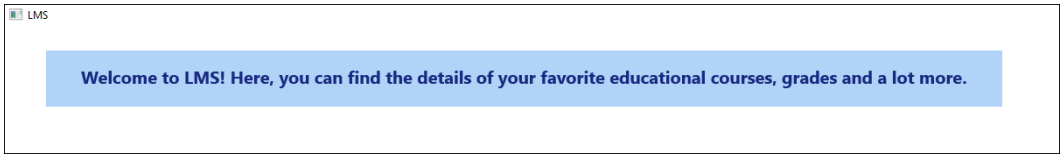


Figure 10.7 – Changing Foreground Color

Data binding and MVVM – deep Dive into the MVVM pattern to decouple UI and business logic

Data binding is essential in linking the **user interface (UI)** to the underlying business logic, making it easier to manage and manipulate data-driven UIs. The MVVM pattern is an architectural design that separates UI concerns from business logic, resulting in cleaner, more maintainable code.

Understanding MVVM layers

MVVM divides application components into three essential layers, each with a specific role:

- **Model**: This layer contains the application's business logic and data. It operates independently of the UI and is reusable across multiple applications without modification.
- **View**: The view represents the UI and is defined in XAML markup. It presents data visually and responds to user interactions through event handlers.
- **ViewModel**: Acting as a bridge between the Model and View, the ViewModel binds data from the Model to the View and notifies the View of any updates. It can also format data (such as sorting) before passing it to the View without modifying the underlying Model.

MVVM example – course details app

Let's create an example app to display course details using MVVM. This example will consist of a course model, a course view model for data binding, and a UI in XAML.

Step 1: Define the Model class (Course)

First, create a `Course.cs` file with a simple class representing course details. Make sure the namespace matches the one in your application:

```
namespace WinUIThreeDemo
{
    public class Course
    {
        public string CourseName { get; set; } = "Desktop Application
Development";
        public string CourseDescription { get; set; } = "Win UI 3, .NET,
Java";
    }
}
```

In this class, the `CourseName` and `CourseDescription` properties define the course's data that will be displayed in the UI.

Step 2: Create the ViewModel class (CourseViewModel)

Next, add a `CourseViewModel.cs` file to handle data binding and updates between the Model and View. This class implements the `INotifyPropertyChanged` interface, allowing it to notify the UI of changes in data:

```
using System.ComponentModel;

namespace WinUIThreeDemo
{
    public class CourseViewModel : INotifyPropertyChanged
    {
        private Course _course;

        public CourseViewModel()
        {
            _course = new Course();
        }

        public string CourseName
        {
            get => _course.CourseName;
            set
            {
                if (_course.CourseName != value)
                {
                    _course.CourseName = value;
                    OnPropertyChanged(nameof(CourseName));
                }
            }
        }

        public string CourseDescription
        {
            get => _course.CourseDescription;
            set
            {
                if (_course.CourseDescription != value)
```

```
                {
                    _course.CourseDescription = value;
                    OnPropertyChanged(nameof(CourseDescription));
                }
            }
        }

        public event PropertyChangedEventHandler PropertyChanged;

        protected void OnPropertyChanged(string propertyName)
        {
            PropertyChanged?.Invoke(this, new
            PropertyChangedEventArgs(propertyName));
        }
    }
}
```

To support responsive data binding and automatic UI updates, we implement the following:

- INotifyPropertyChanged: This interface raises property change notifications, making data-binding responsive
- Property Wrappers: The `CourseName` and `CourseDescription` properties are wrapped with change detection to update the UI whenever they are modified

Step 3: Update the View (MainWindow.xaml)

In the `MainWindow.xaml` file, set up the UI to display course details using data binding. We'll bind the ViewModel to `Grid.DataContext` to establish the connection.

Replace the existing `StackPanel` code in `MainWindow.xaml` with the following:

```
<Grid Background="#b0d3f7">
    <Grid.DataContext>
        <local:CourseViewModel />
    </Grid.DataContext>

    <StackPanel Orientation="Vertical" HorizontalAlignment="Center"
VerticalAlignment="Center" Background="#182980">
        <TextBlock Text="Course Details"
                   FontSize="24"
```

```
                    Margin="20"
                    Width="220"
                    FontWeight="Bold"
                    Foreground="White"
            />

            <StackPanel Orientation="Horizontal" HorizontalAlignment="Left"
VerticalAlignment="Center">
                <TextBlock Text="Course Name:"
                           FontSize="20"
                           Margin="20"
                           Width="220"
                           FontWeight="Bold"
                           Foreground="White"
                />
                <TextBlock Text="{Binding CourseName}"
                           FontSize="20"
                           Margin="20"
                           Foreground="White"
                />
            </StackPanel>

            <StackPanel Orientation="Horizontal" HorizontalAlignment="Left"
VerticalAlignment="Center">
                <TextBlock Text="Course Description:"
                           FontSize="20"
                           Margin="20"
                           Width="220"
                           FontWeight="Bold"
                           Foreground="White"
                />
                <TextBlock Text="{Binding CourseDescription}"
                           FontSize="20"
                           Margin="20"
                           Foreground="White"
                />
            </StackPanel>
        </StackPanel>
    </Grid>
```

To connect the View to the ViewModel and enable dynamic UI updates, we implement the following:

- **DataContext Binding**: `Grid.DataContext` binds to `CourseViewModel`, connecting the View to the ViewModel's data
- **Data Binding**: Each `TextBlock` in the View binds to the `CourseName` and `CourseDescription` properties in the ViewModel, which dynamically updates as data changes

In this example, the MVVM pattern allows us to manage data and UI separately, making the application easier to maintain and scale. By binding the ViewModel to the View using data binding, any changes in the Model layer automatically update the View, and the View can send user input back to the ViewModel.

This approach makes the code base modular, enabling business logic reuse across multiple UIs. The generated UI should look like this:

Figure 10.8 – Course Details UI

Optimizing app performance

Implementing performance optimization techniques is crucial for creating responsive and efficient applications. This involves reducing the load time, managing memory efficiently, minimizing UI latency, and employing profiling and debugging tools to ensure smooth user experiences. The following are some techniques that improve application performance.

Load time reduction

Implementing lazy loading allows static parts of the app to load first, giving users immediate access to the interface, while complex components and logic load afterward. This approach enhances the perceived speed of the app, as users don't have to wait for the entire application to load at once. Asynchronous functions should be used to load data without blocking the main thread, ensuring a smoother experience.

Here are some example practices:

- **Lazy Loading**: Load static UI components upfront and defer complex functionalities
- **Asynchronous Operations**: Use async functions to avoid blocking threads during data load
- **Resource Management**: Release unused resources to maintain efficient memory use

Memory optimization

Effective use of data binding and user controls can significantly reduce memory usage. In WinUI 3, prefer using x:Bind over Binding, as it provides better performance by leveraging compile-time binding. Additionally, it releases resources when they're no longer needed to prevent memory leaks and keep the application efficient.

Here are some example practices:

- **Efficient Data Binding**: Use `x:Bind` instead of Binding for faster binding
- **Resource Disposal**: Release resources such as event handlers to free up memory

Reducing UI latency

Minimize the nesting of layout elements, as deeply nested elements can increase the rendering time. Avoid using complex or unnecessary animations that could slow down the interface, especially on lower-end devices. A streamlined layout with minimal nesting ensures quick response times and smooth interactions.

Here are some example practices:

- **Simplified Layouts**: Limit layout nesting to reduce the processing time
- **Selective Animations**: Use animations sparingly to maintain responsiveness

Profiling and debugging techniques for ensuring smooth user experiences

Preserving resources and identifying performance bottlenecks are essential to maintaining a smooth, responsive application. Profiling and debugging techniques help track performance and detect issues early, allowing for more efficient and reliable application development.

Profiling monitors the application's performance, tracking memory usage, CPU load, and response times. Use the Performance Profiler in Visual Studio to examine resource usage and performance metrics. In Visual Studio, navigate to **Debug > Performance Profiler** to select and start profiling tools. Once the application runs for a set duration, please close it to view detailed performance data.

Debugging techniques

Debugging tools help developers detect and fix errors, enhancing app stability and functionality. Essential debugging techniques include the following:

- **Hot Reload**: This feature allows instant visualization of code changes without rebuilding the app, saving time and improving developer productivity.
- **Breakpoints**: Enable stopping the app execution at specific lines to inspect variables and program flow, making identifying and resolving bugs easier.
- **Conditional Breakpoints**: These advanced breakpoints pause execution only when certain conditions are met, helping isolate specific issues without manually sifting through each line.
- **Memory Leak Detection**: Tools such as .NET Memory Profiler identify cases where resources aren't released correctly. Memory leaks occur when objects that are no longer needed remain in memory, causing excessive usage and potential crashes.

Here are some example practices:

- **Hot Reload**: Modify code with instant feedback without rebuilding
- **Breakpoints**: Inspect variable states at designated points in the code
- **Conditional Breakpoints**: Trigger breakpoints only under specific conditions
- **Memory Profilers**: Use tools to detect and fix memory leaks, improving overall efficiency

Developers can significantly improve application performance by employing best practices for load time reduction, memory optimization, UI responsiveness, and profiling and debugging techniques. These strategies help create a smoother user experience, reduce resource costs, and enhance the app's robustness and efficiency.

Summary

This final chapter explored essential techniques for building efficient, responsive, and visually appealing WinUI 3 applications. We began by examining control customization, demonstrating how properties such as background, padding, margin, and font size enhance the user interface and help create a polished, intuitive experience. Customizing these properties allows developers to create interfaces that align with the application's theme and improve readability and user engagement.

Next, we discussed the importance of data binding and the **Model-View-ViewModel (MVVM)** pattern, a powerful approach for organizing code and separating the UI from business logic. Using MVVM, developers can efficiently manage data updates and interactions, providing a clean, maintainable structure that allows real-time data binding between the ViewModel and the View. This pattern ensures that applications remain modular, easy to maintain, and capable of handling dynamic data.

We also covered performance optimization techniques to ensure applications run smoothly and efficiently, including load time reduction, memory management, and UI latency reduction. By using techniques such as lazy loading, asynchronous data handling, and efficient data binding with x:Bind, developers can significantly enhance performance and create a more seamless user experience.

Finally, we explored profiling and debugging techniques, focusing on tools that allow developers to track performance metrics, detect memory leaks, and quickly resolve issues. Profiling tools and debugging techniques such as hot reload, breakpoints, and memory leak detection provide valuable insights and enable developers to ensure that applications are reliable and efficient.

As we conclude this book, you now have the tools, knowledge, and techniques necessary to create sophisticated, user-friendly, and high-performing WinUI 3 applications. This book has provided a comprehensive guide to mastering modern desktop application development with Blazor, .Net MAUI, and WinUI 3, from control customization and responsive design to effective data handling and performance optimization. With these skills, you are well-equipped to tackle future challenges and deliver exceptional application user experiences.

packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

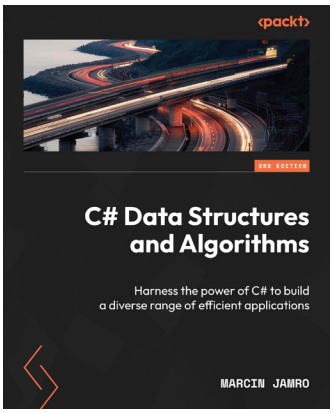
Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

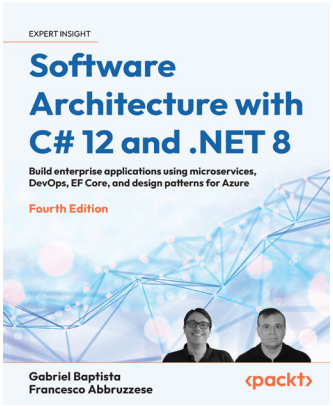


C# Data Structures and Algorithms

Marcin Jamro

ISBN: 978-1-80324-827-1

- Understand the fundamentals of algorithms and their classification
- Store data using arrays and lists, and explore various ways to sort arrays
- Build enhanced applications with stacks, queues, hashtables, dictionaries, and sets
- Create efficient applications with tree-related algorithms, such as for searching in a binary search tree
- Boost solution efficiency with graphs, including finding the shortest path in the graph
- Implement algorithms solving Tower of Hanoi and Sudoku games, generating fractals, and even guessing the title of this book



Software Architecture with C# 12 and .NET 8

Gabriel Baptista, Francesco Abbruzzese

ISBN: 978-1-80512-765-9

- Program and maintain Azure DevOps and explore GitHub Projects
- Manage software requirements to design functional and non-functional needs
- Apply architectural approaches such as layered architecture and domain-driven design
- Make effective choices between cloud-based and data storage solutions
- Implement resilient frontend microservices, worker microservices, and distributed transactions
- Understand when to use test-driven development (TDD) and alternative approaches
- Choose the best option for cloud development, from IaaS to Serverless

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packt.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Now you've finished *GUI Programming with C#*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

`https://packt.link/r/1835882552`

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don’t worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don’t stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily.

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below:

`https://packt.link/free-ebook/9781835882542`

2. Submit your proof of purchase.
3. That’s it! We’ll send your free PDF and other benefits to your email directly.

Index

A

accessibility properties 102

advanced controls, WinUI 3 150

advanced UI design techniques
- accessibility features 102, 103
- animations 102
- control templates 101
- custom controls 101
- transitions 101

animations 102

application
- creating 47-51

ASP.NET Core 77

Azure 77

B

Background property 167

binding modes
- OneTime 123
- OneWay 123
- OneWayToSource 123
- TwoWay 123

Blazor 10, 19, 20
- advantages 23
- and WebAssembly 26, 27
- app, bootstrapping 22
- GUIs, creating 10
- Hello World app, creating 12-14
- loading steps 21
- management of interactive UI 22, 23
- management of user interactions 22, 23
- .NET runtime, loading 21

Blazor application
- App.razor 29
- _Imports.razor 29
- improving 28-31
- Pages folder 29
- Program.cs 28
- project file 28
- Shared folder 29
- Startup.cs 28
- wwwroot folder 28

Blazor solution
- files 44-47

Border 154

C

C# 2, 77
- event-driven programming 3
- object-oriented programming 3
- rich library support 2

collections 126, 127

commanding 84

Command-Line Interface (CLI) 106

commands 125, 126

component
- using 52

component life cycle methods
after rendering 52
disposal 52
initialization 52
parameter setting 52
parameter updating 52
rendering 52
control templates 101
converters 123, 124
core components, .NET MAUI
C# 77
.NET 77
XAML 77
course details app, MVVM example 171
Model class (Course), defining 171
View (MainWindow.xaml), updating 173, 175
ViewModel class (CourseViewModel), creating 172, 173
cross-platform application
creating, with .NET MAUI 78, 84-88
cross-site request forgery (CSRF) attacks 44
cross-site scripting (XSS) attacks 38
custom controls 101
customer relationship management (CRM) 78, 135

D

data binding 83, 111, 114, 122, 171
event binding 63
in Razor components 61
one-way 61, 62, 114
two-way 62, 114
data validation
validation messages, displaying 71
working with 71
debugging 32
information, displaying for 32, 33
debugging tools
using, in Visual Studio 35, 36
dependency injection (DI) 86
development environment
setting up, steps 117
development environment, .NET MAUI
setting up 93, 94
diagnostic tools
using 33
Document Object Model (DOM) 23
dynamic data
managing 63, 64

E

Entity Framework Core 77
environment, WinUI 3
compatible operating systems and versions 137, 138
hardware requirements 136, 137
setting up 136
software requirements 137
errors
handling 37
event binding 63
event handling 83
ExecuteCommand method 84
Extensible Application Markup Language (XAML) 11, 76, 77, 80, 115
features 95

F

Favorite Courses
displaying, dynamically 157-159
ListView control, adding for 156, 157
Favorite Course Selector app 159
Fluent design system 132
Fluent principles
depth 133
light 133
material 133
motion 133
scale 133
Flutter 93
FontSize property 169
FontWeight property 169
Foreground property 170
forms
working with 71

G

Graphical User Interfaces (GUIs) 1, 2
accessibility 1
competitive advantage 1
throughput 1
user experience 1
GridView 150, 159, 161

H

Hello World app
creating, in Blazor 12-14
creating, in .NET MAUI 14-16
creating, in WinUI 3 17, 18
HTTP Strict Transport Security (HSTS) 43

I

integrated development environment (IDE) 117

K

keyboard navigation 103
keyframe animations 102

L

Learning Management System (LMS) 150
Learning Management System (LMS) example 151
border, adding with welcome text 153, 154
main content, displaying 152, 153
navigation items (menu), adding 152
NavigationView control, defining 151, 152
new WinUI project, creating 151
ListView 150, 154-156
adding, for Favorite Courses 156, 157

M

maintainable and scalable code
best practices 128
Margin property 168
Model 85, 111
Model-View-Controller (MVC) 112
Model-View-Presenter (MVP) 112
Model-View-ViewModel (MVVM) pattern 84, 99, 100, 109, 111, 112, 144-147, 163
benefits, in in app development 113
Model 171
View 171
ViewModel 171

N

NavigationView 150

.NET 77

.NET MAUI 10, 11, 75, 76, 92, 110
- advantages 77, 78
- app logic, implementing 98, 99
- benefits, for cross-platform development 92, 93
- components 94
- core components 77
- cross-platform application, creating 78, 84-88
- crucial improvements 76, 77
- data binding 99, 100
- development environment, setting up 93, 94
- GUIs, creating 11
- Hello World app, creating 14-16
- MVVM pattern 99, 100
- namespaces 94
- project, creating 79
- project, files 79
- project, structure 79
- structure 94
- use cases and applications 78
- user inputs and events, handling 99

.NET MAUI application
- creating 118
- deploying 104
- deploying, to app stores 106
- packages, creating 106
- platform-specific settings, configuring 104, 105
- running, on Android 105
- running, on iOS 105
- running, on Windows 105
- submitting, to app stores 107
- testing 104

.NET MAUI project development 95
- app, styling 97, 98
- app, theming 97, 98
- controls, adding 96, 97
- Grid layout, using for responsive design 95, 96
- UI, designing with XAML 95

O

object-relational mapping (ORM) 77

one-way data binding 61, 62, 109, 114, 116

P

Padding property 167, 168

performance optimization techniques 175
- load time reduction 176
- memory optimization 176
- profiling and debugging techniques 177
- UI latency, reducing 176

Program.cs file 42-44

R

Razor 47
- best practices and tips 38

Razor components 58
- benefits 58
- component parameters, defining 60
- creating 59
- data binding 61
- development workflow 59
- reusing, in pages and applications 61
- structure 59, 60

Razor components, advanced techniques 65
component life cycle methods 65, 66
data passing, between components 66, 67
templates, using 67, 68
Razor components, sections
C# code block 59
HTML markup 59
Razor components, state management 68
state, handling in single components 68
state, sharing in multiple components 69, 70
third-party libraries, using 70, 71
Razor components styles 72
CSS styles, applying to 72
scoped CSS styles, using 72, 73
real-world use cases, WinUI 3
educational e-learning application 136
financial trading platform 136
healthcare management system 136
routing 53, 54
Running Pages
issues 34

S

sample API
setting up 54
screen reader support 102
ServerInfo helper 33
StackPanel 153

T

TextBlock 153, 154
transitions 102
TreeView 150, 164-166
troubleshooting 32
two-way data binding 62, 109, 114, 116

U

UI, .NET MAUI applications 80
controls 81, 82
layouts 80, 81
styling 82
theming 82
Universal Windows Platform (UWP) 132
versus WinUI 3 135
user experience
enhancing 128
user interface (UI) 58, 142

V

View 86, 112
defining 121
View (MainPage.xaml) 116, 117
view model 112
binding to 121, 122
creating 119
data, initializing 120
properties, implementing 120
ViewModel 85
ViewModel class
defining 119
View model (UserViewModel.cs) 115
Visual Studio
blank Windows Form 7
controls, adding to Form 7-9
debugging tools, using 35, 36
Designer window 6
Editor 4

installing 3
Properties window 5
Solution Explorer 4
Toolbox 5
URL 3
using, to create GUI in C# 6
Visual Studio Code 3
Visual Studio Community 3
Visual Studio Enterprise 3
Visual Studio Professional 3

W

wasm 19
WebAssembly 10, 19, 24
and Blazor 26, 27
benefits 26
working 25
Windows 10
benefits 137
version 137
Windows 11
benefits 138
version 138
Windows Forms (WinForms) 10
versus WinUI 3 135
Windows Presentation Foundation (WPF) 10
versus WinUI 3 134
Windows UI Library 3 (WinUI 3) 132
features 133
installing 139
real-world applications and use cases 135
templates, installing 139
Visual Studio individual components 138
Visual Studio needed workloads 138
Windows UI Library 3 (WinUI 3) app
building 142, 143
data binding 145-147
debugging 144
MVVM pattern 144-147
running 144
versus UWP 135
versus WinForms 135
versus WPF 134
WinUI 3 10, 11
advanced controls 150
GUIs, creating 11
Hello World app, creating 17, 18
WinUI 3 project
creating 139, 140
initial settings 140
packaged and unpackaged app type, selecting between 141
structure 141
templates 140

X

Xamarin.Forms 76